

HyperTrace CloudAegis

Autonomous Real-Time Cloud Cost Intelligence & Infrastructure Protection Platform

Correlating live infrastructure telemetry, real-time \$/hour cost velocity, rolling behavioural baselines, and runtime security context to execute policy-governed autonomous remediation.

MAJOR PROJECT • COMPREHENSIVE TECHNICAL DOSSIER & BUILD GUIDE

Author: Manas | Final Documentation | 2026

Specification: 4 Parts • 30 Chapters • 14 Implementation Milestones • 5 Appendices • 17 Architecture Figures

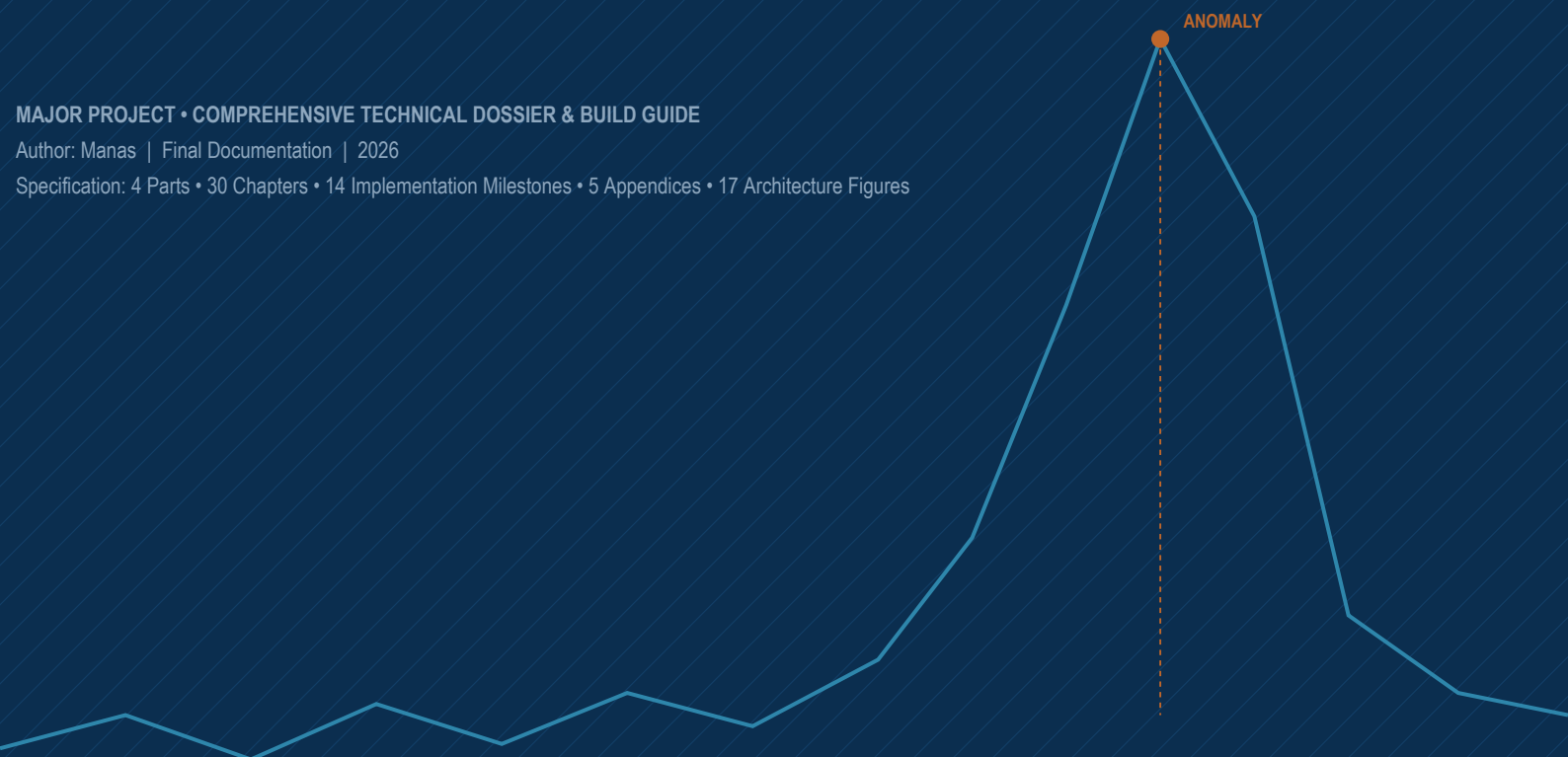


TABLE OF CONTENTS

Table of Contents

Part I: The Problem, The Gap, and The Concept	p. 3
1. Executive Summary	4
2. The Problem, In Full Detail	4
2.1 Why this problem exists	4
2.2 Concrete failure scenarios this project targets	5
2.3 The three properties that make this class of incident distinctive	5
3. How HyperTrace Functions — The Four-Phase Engine	5
Phase 1 — Observe	6
Phase 2 — Understand: converting technical data into money	6
Phase 3 — Decide: behavioural and anomaly analysis	7
Phase 4 — Act: policy-driven autonomous remediation	7
4. System Architecture Overview	8
4.1 Why an event bus instead of the collector writing straight to the database	9
4.2 The single-writer principle	9
5. Backend Design — Services, Data Model, APIs	9
5.1 Service decomposition	9
5.2 Data model	9
5.3 Key API endpoints	10
5.4 Non-functional backend concerns	10
6. Frontend Design — Screens, Components, UX	10
6.1 Primary screens	11
6.2 Frontend architecture	11
7. Competitive Landscape	11
7.1 Category A — Cloud-native monitoring and observability	11
7.2 Category B — Cloud billing and FinOps visibility platforms	12
7.3 Category C — Autonomous cost-optimization and cloud governance	12
7.4 Category D — Runtime and cloud security tools	12
8. Direct Feature Matrix	13
9. What Genuinely Makes HyperTrace Different (and What Doesn't)	14
9.1 The honest positioning	14
9.2 What should not be claimed	14
10. Literature Survey and Research Gap	15
Research gap, stated for a report	15
11. Drawbacks, Risks and Honest Limitations	15
12. Complete Requirements Specification	17
12.1 Functional requirements	18
12.2 Non-functional requirements	18

12.3 Scope boundaries	18
13. Technology Stack	19
14. Core Algorithms	19
14.1 Moving-average and EWMA baseline	19
14.2 Z-score anomaly detection, the core explainable detector	20
14.3 Joint decision logic — the actual differentiator	20
14.4 Policy evaluation	21
15. Presentation Defense Kit	21
16. Conclusion to Part I	23
Part II: Detailed Architecture & Systems Design	p. 24
17. Architecture Principles and Constraints	25
17.1 Seven principles	25
17.2 Hard constraints	25
18. The Four Views	26
18.1 System context	26
18.2 Container view	26
18.3 Component view — Cost Intelligence Service	28
18.4 Component view — Behaviour Analysis Service	28
19. Data Flow: Hot Path and Cold Path	28
19.1 Hot path latency budget	29
19.2 What is deliberately excluded from the hot path	29
19.3 The incident lifecycle, end to end	30
19.4 The cold path and replay	30
20. Event and Topic Design	31
20.1 Topic layout	31
20.2 Why partition by service, and where ordering actually matters	31
20.3 Core event schemas	32
20.4 Idempotency and delivery semantics	33
21. The Data Model in Full	34
21.1 Dimension tables	34
21.2 Hypertables	35
21.3 Continuous aggregate for the dashboard	36
21.4 State, decision and audit tables	37
21.5 The cardinality problem and the chosen compromise	39
22. The Cost Model, Precisely	40
22.1 Direct cost of a workload	40
22.2 Shared and unallocated cluster cost	40
22.3 Cost per unit of work	41
22.4 A fully worked example	41
22.5 Reconciliation against the authoritative bill	42

23. Baselines and State Management	42
23.1 The 168-bucket scheme	42
23.2 The update rule	43
23.3 Warm-up and the maturity threshold	43
23.4 Gaps, missing data and aging	44
23.5 Legitimate step changes	44
23.6 Where the state lives and how it survives restart	44
24. Detection and Classification	45
24.1 What is scored	45
24.2 Thresholds, and why these numbers	45
24.3 The confidence score	46
24.4 Classification and the evidence each one requires	46
24.5 The trade being made, stated plainly	47
25. Policy, Actions and Reversibility	47
25.1 Policy as data	47
25.2 The action ladder	48
25.3 Conflict resolution and precedence	49
25.4 Safety rails on the executor	50
26. Autonomy, Trust and Human Authority	50
26.1 The trust ladder	50
26.2 The anomaly lifecycle	51
26.3 What remains a human decision, permanently	52
27. Failure Modes and Graceful Degradation	52
27.1 Component failure matrix	53
27.2 The three properties this table demonstrates	53
27.3 Failure modes with no clean answer	54
28. Security, Tenancy and Self-Observability	54
28.1 Authentication, authorisation and roles	54
28.2 Tenant isolation	54
28.3 Hardening the platform's own footprint	55
28.4 Observing the observer	55
29. Performance, Scale and Capacity	56
29.1 Data volume arithmetic	56
29.2 Where the compute goes	56
29.3 Sizing the local demonstration	57
29.4 The first bottleneck, and the honest ceiling	57
Part III: Hands-On Build Guide: Kind & Local Stack	p. 58
30. Before You Start	59
30.1 What you need on the machine	59
30.2 The mental model	59
30.3 What you are going to build	59

30.4 The route through the milestones	60
30.5 Repository layout	61
Milestone 1: A Kubernetes cluster on your laptop	61
Milestone 2: The workload you are going to watch	64
Milestone 3: Metrics collection with Prometheus	69
Milestone 4: The database and the schema	72
Milestone 5: The event bus	75
Milestone 6: The collector	78
Milestone 7: The cost engine	82
Milestone 8: The behaviour engine	87
Milestone 9: The policy engine	93
Milestone 10: The executor and the audit trail	96
Milestone 11: The API layer	100
Milestone 12: The dashboard	105
Milestone 13: Four demonstrations	110
Milestone 14: Measure it, then write down what is true	112
31. What you have built, and what you have not	115
31.1 What is genuinely working	115
31.2 What is simulated, and what that costs you	115
31.3 The honest limitations to state before you are asked	115
31.4 If you have another two weeks	116
Part IV: Appendices, Reference & Viva Defense	p. 117
A. Appendix A - The repository, end to end	118
A.2 The Makefile that replaces every command in Part III	119
A.3 If Kubernetes will not cooperate	120
B. Appendix B - PromQL cookbook	121
B.1 The collector contract	121
B.2 Queries for when something looks wrong	121
C. Appendix C - Glossary for the impatient	121
C.1 Infrastructure	122
C.2 Data and streaming	122
C.3 Detection and cost	123
D. Appendix D - Fourteen weeks, and what to cut	123
D.1 What each week must end with	124
D.2 Cut in this order	124
E. Appendix E - Definition of done	124
E.1 Functional	124
E.2 Non-functional and evidential	125

F. Appendix F - Questions you will be asked	125
F.1 On novelty	125
F.2 On the mathematics	126
F.3 On safety	126
F.4 On engineering choices	126
F.5 On honesty	127

The Problem, The Gap, and The Concept

The complete technical dossier as originally submitted, expanded to sixteen chapters. Covers why modern cloud elasticity makes runaway spend invisible, how HyperTrace correlates four signals into a policy-gated decision, and how it compares against existing commercial tools.

Chapters 1–3	Executive summary, failure scenarios, and the four-phase engine
Chapters 4–6	System architecture, backend services, and frontend UX
Chapters 7–8	Competitive landscape and direct feature matrix
Chapters 9–11	Honest positioning, academic literature, and risk analysis
Chapters 12–16	Requirements, math formulas, and concrete algorithms

THE ONE-PARAGRAPH VERSION

1. Executive Summary

HyperTrace is a monitoring-and-control platform that sits above a company's existing cloud infrastructure — AWS, GCP, Azure, or self-managed Kubernetes — and continuously answers one question that no single existing tool answers on its own: **is the money being spent by this infrastructure, right now, actually justified by what the business is getting in return, and if not, should the system act automatically?**

It answers that question by pulling together four signal types that today live in four separate products — infrastructure telemetry (CPU, memory, network, disk), live cost estimation, behavioural and anomaly analysis, and security signals — into one correlated decision. Where organizational policy allows, it then executes an automatic containment action: throttle, freeze-scaling, quarantine, or terminate, rather than only raising an alert and waiting for a human to investigate. The difference between those two behaviours is the entire project. A tool that alerts converts a financial problem into a notification; a tool that acts converts it into a bounded, logged, reversible event.

The premise underneath the whole system is a claim about signals rather than a claim about software: that **cost velocity** — the rate of change of spend, not its absolute level — is a first-class operational signal that deserves the same real-time treatment the industry already gives to error rates and security events. Absolute spend is a finance question, and it is answered monthly. The rate of spend is an engineering question, and it should be answered in seconds.

This document provides: an unsparing comparison against every category of real, currently-shipping product that overlaps with this idea, with vendors named; a concrete backend and frontend architecture; a full requirements specification; a justified technology stack; and — critically for a strict evaluation panel — an honest list of the project's weaknesses together with how each is addressed. A panel that asks *why not just use CloudWatch plus Cost Explorer plus GuardDuty?* is the single most likely failure point of this defense if the answer is not already prepared. Part I answers it. Part II supplies the engineering detail behind the answer, and Part III builds a working system that demonstrates it on a laptop.

THE CLAIM, STATED ONCE, PRECISELY

The defensible claim is narrow and should be stated narrowly: no shipping product joins live cost, live telemetry, behavioural baselines, and security signals into a single policy-governed automatic decision. That is an integration and orchestration gap, not the invention of a new primitive. Every chapter that follows is written to support that claim and no larger one.

WHY THIS GAP EXISTS AT ALL

2. The Problem, In Full Detail

2.1 Why this problem exists

Modern applications at the scale of Netflix, Swiggy, Flipkart or Uber are decomposed into tens or hundreds of independent microservices: login, payment, order, recommendation, notification, image processing, search, analytics, and so on. Each service independently consumes billable cloud resources — CPU, memory, storage, network egress, managed databases, load balancers, GPUs, and serverless invocations. The bill is therefore not a property of the application; it is an emergent property of hundreds of independent consumption decisions made by software, continuously, without human review.

Cloud elasticity is what makes this dangerous. Auto-scaling is designed to absorb demand spikes automatically, and it does that job well. But the auto-scaler cannot distinguish demand from defect. It observes that CPU is high and adds capacity; it has no opinion about *why* CPU is high. That single missing judgement is precisely the feature that turns a small software defect into a large, invisible financial event. Elasticity converts a bug that would once have caused an outage — loud, obvious, instantly escalated — into a bug that causes a silent, uncapped invoice.

2.2 Concrete failure scenarios this project targets

The four scenarios below are not hypothetical categories invented to justify a project. Each is a well-documented class of cloud incident, and each shares the same structural property: existing tooling cannot see it, for a specific and explicable reason.

Scenario	Technical signature	Why it is invisible today
Runaway retry or infinite loop	One request silently fans out into 10 □ 100 □ 100,000 internal calls; CPU climbs; the auto-scaler launches dozens of new nodes	The application keeps working and customers see no errors, so nobody investigates until the monthly invoice arrives
Orphaned or forgotten resources	Test or staging servers — say 200 instances — left running for weeks after the experiment that needed them ended	No functional impact, no alert threshold crossed; a purely organizational “nobody noticed” failure that no monitor is watching for
Cryptojacking	Attacker gains shell access and installs a miner; CPU reaches 95–99%; no data exfiltration and no malware signature match	Security tools are tuned to detect data theft or known malware signatures, not compute abuse that looks entirely legitimate to the kernel
Traffic-blind auto-scaling	CPU crosses a scaling threshold even though real user traffic is flat, so the cluster grows to serve work no customer requested	Auto-scalers scale on resource metrics, never on whether the <i>cause</i> of the metric is legitimate demand

The common thread is worth stating explicitly, because it is the justification for the entire project. In every one of these cases the system keeps functioning, so uptime monitoring stays green. No security signature is matched, so the security operations tooling stays quiet. And the cost becomes visible only on a delayed, aggregated bill, so finance discovers the problem weeks later, when the money is already spent and the evidence needed to explain it has aged out of the metrics retention window. Three independent monitoring systems all report “normal” while a fourth, unmonitored quantity — the rate of spend — goes vertical.

2.3 The three properties that make this class of incident distinctive

It helps to name the properties, because they determine what a detector must look like. First, these incidents are **silent**: no user-visible symptom is produced, so no service-level objective is breached and no page fires. Second, they are **signature-free**: nothing about the behaviour matches a known pattern of attack, because in three of the four cases there is no attacker at all. Third, and most importantly, they are **latent in the financial dimension**: the only monitored quantity that moves decisively is one that no real-time system is observing.

A detector for silent, signature-free, financially-latent incidents cannot be built from thresholds on a single metric, because the only metric that moves reliably is one that nobody computes in real time. It must be built by computing that metric first — live cost — and then reasoning about it in relation to the business signal that is supposed to explain it. That is the shape of the four-phase engine described next.

THE TIMESCALE MISMATCH

One number frames the whole problem. Cloud billing data on AWS lands through Cost and Usage Reports with a lag typically measured in hours, sometimes up to a day. An auto-scaling group can triple in size in ten minutes. The gap between those two timescales — roughly two orders of magnitude — is the window in which every incident in the table above happens, unobserved.

OBSERVE, UNDERSTAND, DECIDE, ACT

3. How HyperTrace Functions — The Four-Phase Engine

The system is a pipeline of four phases. Each phase consumes the output of the one before it and adds exactly one kind of meaning: raw measurement becomes money, money becomes an assessment, and an assessment becomes an action. Keeping these four concerns strictly separated is what makes the system explainable, because every conclusion can be traced back through the phases that produced it.

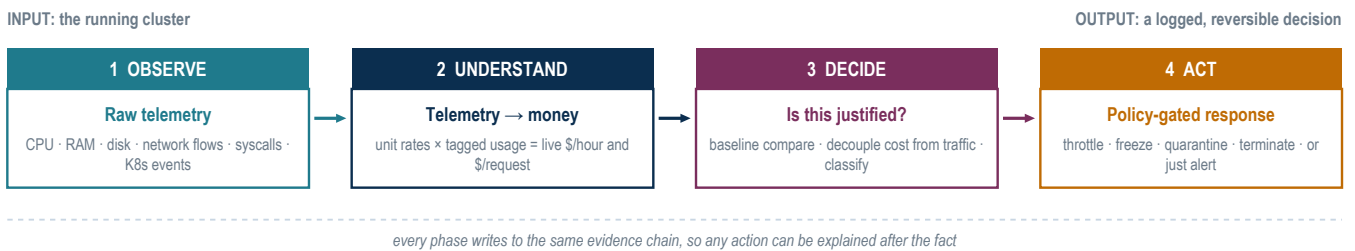


Figure 1 - The four-phase engine. Phases 2 and 3 are where HyperTrace differs from a monitoring tool: it converts telemetry into money, then asks whether that money is justified.

Phase 1 — Observe

A lightweight collection layer runs on every node, pod, or virtual machine and continuously reads: CPU utilisation per process and container; memory, including resident set size, working set, and growth rate; disk I/O in read and write operations and throughput; network flow data covering bytes in and out with source and destination addresses and geography; container lifecycle events such as start, stop, restart, crash and OOM-kill; and Kubernetes control-plane events including deployments, scaling events and pod evictions.

In practice this is implemented with two complementary mechanisms. A conventional metrics exporter stack — node exporter, cAdvisor and kube-state-metrics feeding Prometheus — supplies resource and lifecycle data cheaply and with well-understood semantics. Alongside it, an eBPF-based agent supplies process- and syscall-level visibility that no exporter can provide, and does so without requiring any change to application code. The distinction matters for the security phase later: an exporter can tell you a container is using 98% of a core, but only kernel-level instrumentation can tell you which process inside it is doing so, and to which remote address it is talking.

Two design commitments are made in this phase and never revisited. Collection is **read-only**: nothing in the observation path holds credentials that permit mutation. And collection is **push-to-bus**: collectors never write to a database directly, for reasons developed in chapter 4.

Phase 2 — Understand: converting technical data into money

This is the layer that gives raw metrics business meaning, and it is the layer most often missing from otherwise excellent monitoring stacks. Cloud providers publish per-resource unit pricing: per vCPU-hour, per GB-RAM-hour, per GB of egress, per GB-month of storage, per GPU-hour. The Cost Intelligence Engine multiplies live, tagged resource consumption by these rates to produce a continuously updating **\$/hour** figure per pod, per service, and per team-owned namespace. Using request or trace counts as a denominator, it also produces a **cost-per-unit-of-work** figure: cost per API call, per checkout, per search, per active user.

Conceptually this is the same computation real FinOps tools perform. The difference is where the input comes from and how fresh it is. Commercial cost platforms derive spend from the provider's billing export, which is authoritative but delayed. HyperTrace derives spend from runtime telemetry at second-level granularity, which is immediate but approximate, and then reconciles that estimate against the authoritative bill on a slower loop. This trade is made deliberately and openly: an estimate available in five seconds is operationally more useful than an exact figure available tomorrow, provided the system is honest about which of the two it is showing. Chapter 22 of Part II gives the arithmetic, the shared-cost allocation strategy, and the reconciliation loop in full.

Cost-per-unit-of-work deserves particular emphasis, because it is the quantity that makes the system's judgements defensible to a business audience. Total spend rising by 40% is ambiguous — it may simply mean the company is growing. Cost per checkout rising by 40% is never ambiguous. It means the same business outcome now costs more to produce, which is an engineering regression regardless of what the top-line number does.

Phase 3 — Decide: behavioural and anomaly analysis

The engine maintains a rolling behavioural baseline per service. The baseline is not a single number but a profile: every Monday between nine and eleven in the morning, traffic for this service roughly doubles, and that is normal. A state is flagged as abnormal only when the technical and financial signals *decouple* from the business signal — when cost and CPU rise sharply while request volume stays flat.

That decoupling test is the single idea the whole project rests on, and it is worth seeing rather than reading. The figure below contrasts the two cases the detector must separate. On the left, cost and traffic move together: expensive, but explained, and therefore not an incident. On the right, cost climbs while traffic is flat: the same cost curve, now unexplained, and therefore an incident. No single-metric threshold can tell these apart, because the cost curves are identical. Only the relationship between two curves distinguishes them.

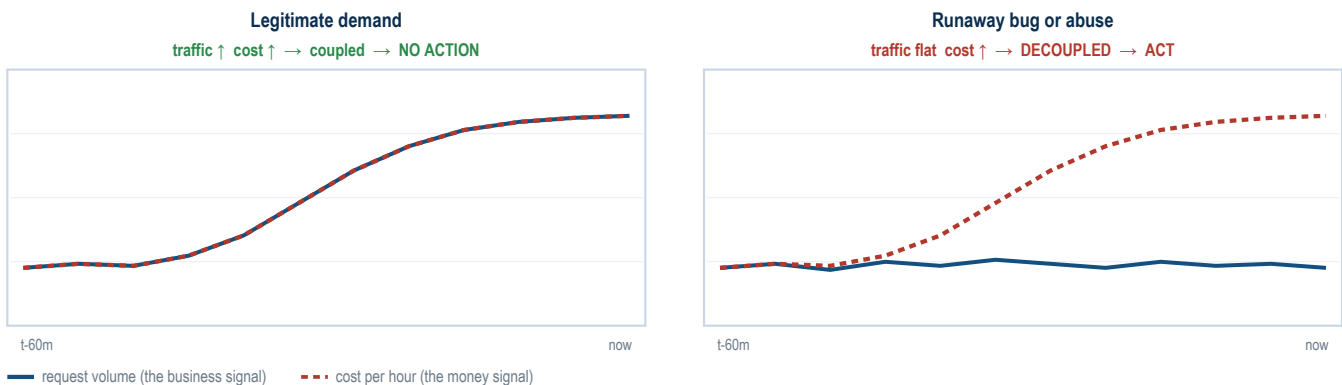


Figure 6 - The single idea the whole project rests on. Absolute cost is not the alarm; the divergence between spend and the work being done is.

Detection techniques are introduced in rough order of implementation difficulty, and the project deliberately stops early: static thresholds, then moving-average and EWMA deviation, then Z-score based statistical anomaly detection, and only as an optional stretch goal an Isolation Forest for multivariate anomaly scoring. The output of the phase is a normalized anomaly score together with a best-guess classification — legitimate traffic growth, software bug or inefficiency, misconfiguration, or suspected abuse. The cryptomining case is characterised specifically as near-100% CPU combined with flat or absent external request volume and a suspicious egress destination.

WHY THE SIMPLE DETECTOR IS THE RIGHT DETECTOR

The detector stops at Z-scores on purpose. When a panel asks “why did it flag this?”, an answer of the form “CPU was 4.2 standard deviations above the Monday-10am baseline while requests were within half a deviation of it” is a complete answer. An answer of the form “the model scored it 0.81” is not. Explainability is worth more than accuracy here, because an unexplainable flag cannot be allowed to trigger an automatic action.

Phase 4 — Act: policy-driven autonomous remediation

Every action is gated by an organization-defined policy rather than a hard-coded rule, and this distinction carries real weight in the defense. Representative policies read: never auto-terminate anything tagged **production**; throttle any workload exceeding a stated dollar-per-hour figure that is not tagged **critical**; auto-quarantine anything matching the cryptomining heuristic regardless of tag. Policy is data, held in a table, versioned and auditable — not a branch in the source code. The four available actions, from least to most aggressive:

Action	Mechanism	Effect
Throttle	Kubernetes <code>ResourceQuota</code> or <code>LimitRange</code> patch, or a cgroup CPU quota adjustment	Caps the CPU and requests the workload can consume without killing it
Freeze scaling	Patch the Horizontal Pod Autoscaler's <code>maxReplicas</code> , or pause the cloud Auto Scaling Group	Stops the runaway cost multiplication while keeping current capacity serving

Action	Mechanism	Effect
Quarantine	Apply a deny-all NetworkPolicy to the pod, or move it into an isolated security group	Cuts off suspicious outbound activity, such as contact with a mining pool
Terminate	Delete the pod, or stop the instance through the cloud provider API	Hard-stops confirmed-malicious or catastrophically runaway workloads

The ordering is not cosmetic. It is a ladder of increasing blast radius and decreasing reversibility, and the policy engine is required to prefer the lowest rung that plausibly contains the incident. Freezing an autoscaler is trivially reversible and stops cost multiplication immediately; terminating a pod is reversible only in the sense that Kubernetes will reschedule it, and destroys any in-flight work. Every automated action is logged with the full evidence chain — the metrics, the cost delta, the anomaly score and the policy rule matched — both for audit and so that engineers can root-cause the incident afterwards. Part II, chapter 26 specifies the safety interlocks that constrain this phase: the non-overridable policy floor, the rate limiter, the kill switch, and the rollback mechanism.

THE SHAPE OF THE SYSTEM

4. System Architecture Overview

The architecture is a layered pipeline with a deliberate seam in the middle. Everything below the seam collects and cannot

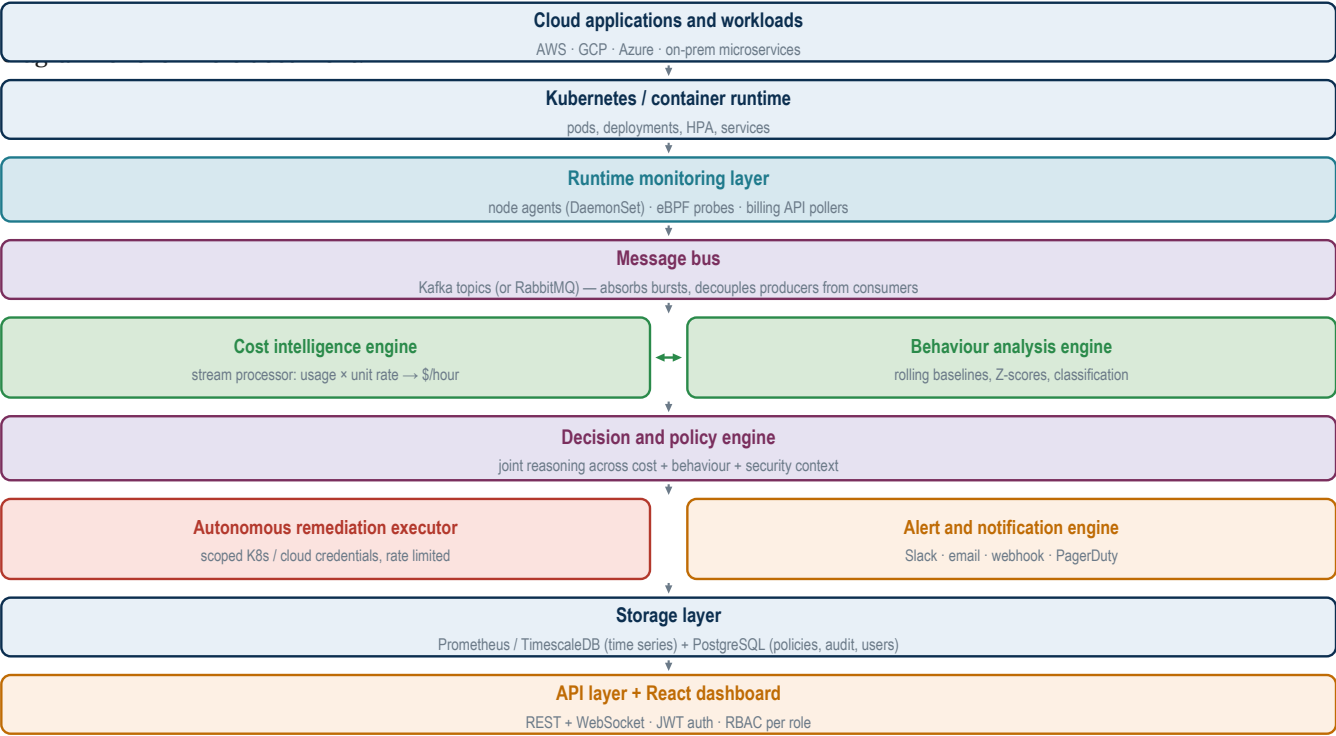


Figure 2 - End-to-end layered architecture. Read top to bottom as the path a single metric sample takes from the kernel to the dashboard.

Two data paths run in parallel through this stack. A **hot path** carries metrics into cost calculation, then anomaly scoring, then decision, with a target end-to-end latency of a few seconds; it drives real-time remediation and nothing else. A **cold or analytical path** handles long-term storage, historical trend queries, monthly rollups and investigative questions of the form “why did March cost spike?”; it powers dashboards and reports and has no low-latency requirement at all. Separating them means an expensive analytical query can never delay a remediation decision, and a burst of metrics during an incident can never stall a dashboard.

Every box in this diagram is expanded in Part II: the container-level view in chapter 18, the two paths with per-hop latency budgets in chapter 19, the bus topics in chapter 20, the storage schema in chapter 21, and the deployment topology in chapter 29.

4.1 Why an event bus instead of the collector writing straight to the database

Metric volume from thousands of pods spikes unpredictably, and it spikes hardest during exactly the incidents the system exists to catch. A direct-write architecture would therefore fall over precisely when it is needed most: the collector would block on database writes, back-pressure into the scrape loop, and the pipeline would go blind at the moment of maximum value. This failure mode is not a tail risk; it is the expected behaviour of a synchronous write path under the load profile this system is designed for.

A message bus absorbs bursts into durable storage at a cost per event far below that of an indexed insert, allows the consumers to be scaled independently of the producers, and lets the Cost Engine, the Behaviour Engine and the raw archival writer all read the same event stream without coupling to each other. The replay property matters as much as the buffering: because the bus retains the raw event log, a bug found later in the cost model can be fixed and the affected window recomputed from original inputs. Without a durable log, a wrong cost model produces permanently wrong history.

4.2 The single-writer principle

One architectural rule is worth stating at the top level because it constrains everything below: exactly one component in the system holds credentials that can change the state of the observed infrastructure, and that component is the Remediation Executor. Collectors, the cost engine, the behaviour engine, the policy engine, the notification service and the API all operate read-only. This makes the security review of the whole platform tractable, because the question “what can this system break?” reduces to the RBAC verbs granted to a single service account, enumerated in Part II, chapter 26.

DECOMPOSITION AND CONTRACTS

5. Backend Design — Services, Data Model, APIs

5.1 Service decomposition

Eight services, each independently deployable, each with a single responsibility and a distinct scaling characteristic. The scaling column is the useful one: it explains why these are separate services rather than modules in one process. A component that must run once per node cannot share a deployment with one that must be a singleton, and a stateful partitioned consumer cannot share a deployment with a stateless one.

Service	Responsibility	Scaling characteristic
Collector Service	Runs as a DaemonSet; scrapes node, container and eBPF metrics and pushes them to the bus	One instance per node; scales with cluster size
Billing Adapter Service	Polls AWS Cost and Usage Reports, GCP Billing Export or Azure Cost Management for authoritative pricing and reconciliation	Low-frequency scheduled jobs
Cost Intelligence Service	Stream processor; consumes raw metrics, applies the pricing model, emits \$/hour events per resource and service	Horizontally scalable stream consumer group
Behaviour Analysis Service	Maintains rolling baselines per service and computes anomaly scores	Stateful; partitioned by service identifier
Decision / Policy Service	Evaluates policy rules against anomaly, cost and security context; emits an action verdict or alert-only verdict	Low volume, low latency requirement
Remediation Executor	Holds scoped Kubernetes and cloud IAM credentials; executes throttle, freeze, quarantine and terminate	Strict RBAC; audit-logged; deliberately a singleton
Notification Service	Fan-out to Slack, email, PagerDuty and webhooks	Stateless
API / BFF Service	REST and GraphQL layer serving the dashboard; JWT authentication and RBAC per organization and team	Stateless, horizontally scalable

5.2 Data model

Seven entity groups, split across a time-series store and a relational store according to access pattern rather than data type. Anything written at high frequency and read as a range query belongs in the hypertable; anything read as a point lookup or joined against other entities belongs in ordinary PostgreSQL tables.

Entity	Key fields	Store
<code>raw_metrics</code>	timestamp, resource_id, cpu, mem, net_in, net_out, disk_io	Time-series database: Prometheus or TimescaleDB
<code>cost_events</code>	timestamp, service_id, resource_type, unit_rate, cost_per_hour	TimescaleDB hypertable
<code>baselines</code>	service_id, metric, rolling_mean, rolling_stddev, day_of_week_profile	PostgreSQL
<code>anomalies</code>	id, service_id, score, classification, evidence_json, status	PostgreSQL
<code>policies</code>	org_id, rule_dsl, scope by tag or namespace, action, priority	PostgreSQL
<code>actions_log</code>	id, anomaly_id, action_type, executed_at, result, rollback_ref	PostgreSQL, append-only and audit-grade
<code>services / teams / users</code>	ownership, criticality tag, RBAC role	PostgreSQL

Part II, chapter 21 gives the full DDL for each of these, including the hypertable definitions, the continuous aggregate over cost events, the trigger that enforces append-only semantics on the audit log, and the retention policies.

5.3 Key API endpoints

The API surface is deliberately small. Seven endpoints cover every screen in the product, which is a useful check on scope: an API that needs forty endpoints to serve six screens usually indicates the domain model is wrong.

API surface (illustrative)

```

GET  /api/v1/services/{id}/cost?range=1h    -> live + historical cost series
GET  /api/v1/services/{id}/anomalies        -> anomaly history + evidence
POST /api/v1/policies                       -> create / update a policy
GET  /api/v1/dashboard/summary              -> top spenders, active alerts
POST /api/v1/actions/{anomaly_id}/approve   -> human-in-the-loop approval
POST /api/v1/actions/{anomaly_id}/rollback  -> revert a remediation action
WS   /api/v1/stream/alerts                  -> websocket live alert feed

```

Two of these are worth noticing because they encode product decisions rather than data access. The `approve` endpoint exists because semi-autonomous operation is the default for high-blast-radius actions, not an afterthought. The `rollback` endpoint exists because an action the system cannot undo is an action the system should be reluctant to take.

5.4 Non-functional backend concerns

Four concerns are stated explicitly here because each is a question an evaluator is likely to ask, and because each shapes the design rather than merely decorating it.

- **Idempotency.** Remediation actions must be safe to retry. Freeze-scaling issued twice must not error, and must not leave the autoscaler in a different state than a single issue would have. This is achieved by making every action a declarative patch to a desired state rather than an imperative operation.
- **Least-privilege IAM.** The Remediation Executor's credentials are scoped per action: a role that can patch HPA and NetworkPolicy objects cannot delete arbitrary resources. This is both a security requirement and, reliably, something evaluators probe.
- **Backpressure handling.** The exact moment of an incident is when metric volume is highest. The pipeline must degrade by sampling and batching rather than by failing, and it must shed load in an order that preserves detection capability for as long as possible.
- **Multi-tenancy.** If the platform serves multiple client organizations, data must be partitioned per organization at both the database row level and the message-topic level, so that a bug in a query cannot leak across tenants.

TWO AUDIENCES, ONE DATASET

6. Frontend Design — Screens, Components, UX

The interface has an unusual constraint: it serves two audiences whose questions share a dataset but share almost nothing else. A finance stakeholder wants to know what the infrastructure costs and whether that is changing. An SRE wants to

know which pod is misbehaving and what the system did about it. Presenting one view to both produces something that is too technical for the first and too shallow for the second, so the screens are split by audience rather than by data source.

6.1 Primary screens

Screen	Purpose	Key components
Executive / FinOps dashboard	Business-level view for CTOs and finance	Live total <code>\$/hour</code> ticker, cost trend chart, top-ten expensive services, monthly forecast, savings-to-date counter
Service deep-dive	Per-service technical and financial correlation	CPU, RAM and network charts overlaid with <code>\$/hour</code> , cost-per-request metric, anomaly timeline
Incident / anomaly feed	SRE and DevOps triage view	Live feed of anomalies with severity, evidence graph, one-click approve and rollback for actions
Policy builder	Lets administrators define remediation rules without writing code	Rule builder mapping condition to action, simulation and dry-run mode, audit history
Root-cause / evidence graph	Visualizes the correlation chain from deployment through metric spike and cost spike to action	Timeline plus node-link graph component
Alerts and notification settings	Configure Slack, email and webhook routing	Form-driven settings

6.2 Frontend architecture

A single-page application in React with TypeScript. The component library is built on a design system using Tailwind CSS, with a charting library such as Recharts or D3 for time-series and the evidence graph. Server state is managed with React Query for caching and revalidation, with lightweight local state — Zustand or Context — for UI concerns only. A WebSocket connection carries the live alert feed so the dashboard never polls.

Role-based views follow from the two-audience split: an SRE sees the incident feed and the policy builder; a finance or FinOps user sees the executive dashboard and cost breakdowns without remediation controls. RBAC is enforced in the UI for usability and, non-negotiably, again in the backend API for security. A hidden button is a usability feature, never a security control.

TypeScript earns its place here for a specific reason rather than a general one. The dashboard consumes a cost model whose field semantics are easy to confuse — `cost_per_hour` and `cost_this_hour` differ by a factor that depends on elapsed time, and mixing them produces a plausible but wrong number on an executive screen. A compile-time check against the API contract catches that class of error before a demo does.

PRESENTATION TIP

Because this is a systems project rather than a consumer product, evaluators will weight backend correctness and architectural soundness far more heavily than frontend polish. But a clean, working, live-updating dashboard remains the single most convincing artifact available in a five-minute demo, so it is worth investing in one well-rehearsed demo scenario rather than six half-built screens.

THE REAL 2026 MARKET, NAMED

7. Competitive Landscape

This chapter replaces vague claims with the actual market. There is no single existing product that does exactly what HyperTrace proposes, but there are four adjacent categories, each of which solves part of the problem well — in several cases better than this project will. The defense needs to acknowledge that honestly and precisely, because every category below contains at least one product an experienced evaluator has used in production.

7.1 Category A — Cloud-native monitoring and observability

Examples: AWS CloudWatch, Google Cloud Monitoring and Azure Monitor, the third-party observability platforms Datadog and New Relic, and the open-source Prometheus and Grafana stack.

What they do well: extremely mature metric collection, dashboarding, alerting on infrastructure thresholds, log aggregation, and distributed tracing — Datadog and New Relic in particular have deep application performance monitoring and trace analysis that this project does not attempt to match.

Structural limitation relevant to this project: these tools report technical state — CPU is at 95% — and, in Datadog and New Relic's case, can layer basic cloud cost views on top. But none of them natively converts a specific anomaly into a live *\$/hour* figure correlated against a traffic baseline and then triggers a scoped infrastructure action. They alert; they do not decide and act. That distinction is structural rather than a missing feature: an observability platform that took automatic action on a customer's infrastructure would be assuming a liability its business model does not price.

7.2 Category B — Cloud billing and FinOps visibility platforms

Native tools: AWS Cost Explorer, AWS Budgets, Azure Cost Management and Billing, and GCP Billing reports.

Third-party platforms: CloudZero, Vantage, Apptio Cloudability, Finout, and Kubecost — the last being Kubernetes-specific, now owned by IBM, and built on the CNCF OpenCost standard.

What they do well, and this is a real and currently competitive market: CloudZero takes an engineering-led approach and can map cost down to a unit-economics level — cost per customer, per feature — with hourly granularity. Vantage is praised for ease of setup, strong multi-cloud and SaaS cost normalization including non-cloud spend such as Snowflake, Datadog and MongoDB, and its Autopilot feature for automating Reserved Instance and Savings Plan purchases. Kubecost is the deepest tool specifically for Kubernetes pod- and namespace-level cost allocation. Apptio Cloudability targets large-enterprise showback and chargeback governance.

Structural limitation: nearly every source describing this market in 2026 makes the same observation — these are *visibility-first* tools. They ingest billing data, often with a lag because the cloud billing exports themselves are not instant, and produce recommendations; a human still has to implement the fix. Vantage's own automation, Autopilot, is currently AWS-only and limited to commitment purchasing rather than runtime remediation. Kubecost is explicitly recommendation-based, and at least one published review notes that turning its suggestions into action can itself consume many engineer-hours. None of them correlate cost with live security telemetry.

7.3 Category C — Autonomous cost-optimization and cloud governance

Examples: Cast AI, Sedai, ScaleOps, nOps, Zesty and ProsperOps.

What they do well: this is the category conceptually closest to HyperTrace's Act phase, and it deserves respect rather than dismissal. These platforms genuinely close the loop between insight and remediation. Sedai autonomously rightsizes compute, memory and storage in real time and has documented substantial customer savings. Cast AI focuses on Kubernetes node autopilot — bin-packing and spot-instance orchestration. nOps and Zesty largely specialize in AWS-only commitment and resource automation with success-fee pricing models.

Structural limitation relevant to this project: this category's automation is almost entirely *efficiency-oriented* — rightsizing, bin-packing, spot allocation, predictive autoscaling — rather than *incident- or security-oriented*. None of the current autonomous platforms are built to reason jointly about whether a given change is a legitimate business spike, a software bug, or an intrusion, using security-relevant signals such as unusual egress destinations or unexpected process behaviour alongside cost. Stated plainly: they optimize known-good workloads; they do not triage suspected-abnormal ones. An optimizer's failure mode is leaving money on the table. A triage system's failure mode is acting on the wrong workload. These are different products with different risk profiles.

7.4 Category D — Runtime and cloud security tools

Cloud-native: AWS GuardDuty, Google Security Command Center, and Microsoft Defender for Cloud. **eBPF-based runtime security,** the technically closest neighbours to the Observe layer: Falco, which is CNCF-graduated, the de-facto default, and originally built by Sysdig; Tetragon, part of the Cilium and Isovalent ecosystem and notable for in-kernel enforcement, meaning it can actually block or kill processes rather than only alert; Tracee from Aqua Security; and the commercial platforms Sysdig Secure and Prisma Cloud.

What they do well: genuinely excellent kernel-level visibility into syscalls, process behaviour and network connections, at very low overhead, using CO-RE eBPF that works across kernel versions without per-distribution recompilation. This project's Observe layer aspires to a subset of what these tools already do well, and the honest framing is that HyperTrace should consume Falco events rather than reimplement them.

Structural limitation: these tools are tuned for classic security signatures — privilege escalation, reverse shells, suspicious syscalls — and are not cost-aware at all. A cryptominer that does not trip a behavioural security rule, because it never spawns a shell and never escalates privileges and simply burns CPU quietly, can slip through a pure security tool. It is far more visible as a *cost anomaly* than as a *security anomaly*. Rule-tuning overhead is also non-trivial in practice: one 2026 industry estimate puts ongoing Falco rule tuning and triage at 20 to 40 engineer-hours per month for a mid-size cluster of 50 to 200 nodes.

WHERE THE GAP ACTUALLY IS

Read the four categories together and the gap has a precise shape. Category A sees the metric but not the money. Category B sees the money but not in real time, and never the threat. Category C acts, but only to optimize what it already trusts. Category D sees the threat but has no concept of cost. The unfilled position is a system that computes money in real time and uses it as the correlating variable between the other three.

HYPERTRACE AGAINST NAMED PRODUCTS

8. Direct Feature Matrix

Seven capabilities against five competitor groupings and HyperTrace. The table is included in this form, cell by cell, because a vaguer comparison would be less useful to the reader and less defensible in front of a panel. Read the note underneath it before drawing any conclusion from the right-hand column.

Capability	AWS native: CloudWatch, Cost Explorer, GuardDuty	Datadog / New Relic	CloudZero / Vantage / Kubecost	Sedai / Cast AI / nOps	Falco / Tetragon	HyperTrace
Real-time infra metrics	Yes	Yes, best-in-class APM	Partial; cost-focused, not raw metrics	Yes, as optimization input	Yes, kernel-level	Yes
Live \$/hour per workload	Delayed by billing export lag	Add-on cost views, coarser	Yes — CloudZero hourly, Kubecost pod-level	Indirect: efficiency %, not live \$/hr	No	Yes, targeted sub-minute
Traffic-decoupled anomaly detection	No; threshold alerts only	Partial: APM anomaly, not cost-aware	Cost anomaly alerts, but not traffic-correlated	Pattern-based, not security-aware	No; not cost-aware	Yes — core function
Security signal correlated with cost	Separate product (GuardDuty)	Separate	No	No	Security only, no cost	Yes — joint reasoning
Autonomous remediation: throttle, quarantine, terminate	No; manual, or wire Lambda yourself	No	Mostly advisory; Vantage Autopilot = commitments only	Yes, but scoped to rightsizing, not to incidents	Tetragon can enforce, but on security rules only	Yes — policy-gated, incident-scoped
Cross-cloud unified decision layer	AWS only	Multi-cloud dashboards, not unified action	Vantage and CloudZero give multi-cloud views; automation often one cloud	Mostly one cloud per vendor; Sedai multi-cloud but efficiency-only	Per-cluster	Designed multi-cloud — ambitious, see ch. 11
Root-cause evidence chain, deploy to cost	No	Partial: deploy markers in APM	Emerging research area; not a mainstream product feature	No	No	Yes — proposed evidence graph

HOW TO READ THE RIGHT-HAND COLUMN

Read this table the right way for the defense. Almost every individual cell in the HyperTrace column already exists in *some* product. The honest, defensible claim is not “nothing like this exists” — it is “no single product joins all of these signals into one automatic, policy-governed decision.” That is a real and currently unfilled integration gap, but it is an integration and orchestration innovation, not the invention of a new primitive. Say this plainly to the panel; claiming otherwise is the fastest available way to lose credibility with anyone who knows this market.

There is a second, subtler way to read the table. Look along the rows rather than down the columns. Every row has at least one competitor with a “yes”, but no competitor has a “yes” in more than four rows, and the four they have are never the same four. That pattern is the market telling you something: these capabilities are not hard individually, they are hard to hold together, because each belongs to a different product category with a different buyer, a different data source and a different tolerance for taking automatic action on someone else's infrastructure.

THE HONEST POSITIONING

9. What Genuinely Makes HyperTrace Different (and What Doesn't)

9.1 The honest positioning

Existing cloud-provider stacks — CloudWatch, Cost Explorer, Budgets, GuardDuty, Auto Scaling and Trusted Advisor on AWS, and the equivalent Google Cloud services — already solve roughly 70 to 80 percent of the raw data-collection problem. A skilled architect could wire several of these together with Lambda and EventBridge glue code and get partway to this idea. An evaluator who knows AWS well may say exactly that, and they would not be wrong. Conceding this openly is not a weakness in the defense; it is what makes the remaining claim credible.

The task, then, is to defend the remaining 20 to 30 percent — the part that is genuinely hard to replicate by gluing services together. There are four such claims, and only four.

1. **Joint reasoning across domains.** Each AWS service answers one question in isolation. CloudWatch answers “is it high?”. Cost Explorer answers “did we spend more?”. GuardDuty answers “was there a threat?”. None of them asks the joint question the Decision Engine asks: given that CPU is high, cost is high, traffic is flat, and a deployment happened five minutes ago, what is the most likely cause, and does policy permit an automatic response? Gluing three tools together with Lambda produces three answers and leaves the correlation to a human at two in the morning.
2. **Policy-driven, organization-specific autonomous remediation gated by business criticality.** Rules of the form “never touch anything tagged production” and “throttle non-critical workloads first” are not a feature any of the reviewed competitors ships as a unified, cross-signal engine today. Individually each vendor has a policy layer; none of them evaluates policy against a combined cost, behaviour and security context.
3. **An explainable evidence chain** — deployment, then metric change, then cost change, then action taken — connecting causally linked events rather than emitting a set of independent alerts that a human must correlate manually. This is close to an active academic research area, specifically counterfactual deployment event graphs for cost attribution, which strengthens rather than weakens the case: it means the project builds toward something the literature itself identifies as missing from production tooling.
4. **Treating cost velocity as a security signal, and the reverse.** Using “high CPU with no matching traffic and no security signature” as a cryptomining heuristic is a genuinely underused detection angle. The eBPF security literature concentrates on syscall and process anomalies; it does not treat financial behaviour as an observable at all.

9.2 What should not be claimed

The following claims are false or unprovable, and each one is a trapdoor in a defense. Stating them as non-claims in the report, before anyone asks, converts a potential ambush into evidence of judgement.

- Do not claim HyperTrace is the first tool to do live cost estimation. CloudZero already achieves hourly granularity commercially, and Kubecost does pod-level allocation.

- Do not claim it is the first to do anomaly detection on infrastructure metrics. Prometheus plus any of a dozen detection layers already does this, and has for years.
- Do not claim it is the first to do autonomous remediation. Sedai and Cast AI already act automatically on production infrastructure, at a scale and reliability level this project will not reach.
- Do not claim production-grade multi-cloud parity as an achieved feature. Treat true multi-cloud normalization as future scope and build the working prototype against one cloud, or against Kubernetes only, which is inherently more portable.

THE SCEPTIC TEST

A useful test for any claim in the final presentation: could a well-informed sceptic in the room name a product that already does this? If yes, the claim needs narrowing until the answer is no. The four claims in section 9.1 survive that test. Broader versions of them do not.

WHAT THE PUBLISHED WORK DOES AND DOES NOT COVER

10. Literature Survey and Research Gap

Six representative works, chosen because together they cover the three research communities this project sits between: eBPF runtime security, Kubernetes anomaly detection, and cost attribution. The final column is the one that matters for positioning — it records, for each work, the specific dimension it does not address.

Author(s) and year	Focus	Key finding	Gap relative to HyperTrace
Yang et al. (2025), <i>IEEE Access</i>	Survey of eBPF-based runtime security tools: Falco, Tracee, KubeArmor, Tetragon	eBPF gives low-overhead kernel-level visibility across heterogeneous workloads	Security only; no cost intelligence integration of any kind
Li et al. (2025), <i>IEEE Access</i>	eBPF detection of semantic DDoS attacks inside Linux containers	Effective runtime attack detection at low overhead	No correlation with cloud spend or business impact
Li et al. (2025), <i>IEEE Trans. Services Computing</i>	EProbe — eBPF-based container status probing	Accurate, lightweight container health monitoring	No decision-making and no remediation layer
Saini et al. (2021), <i>IEEE</i>	Kubernetes anomaly detection via Prometheus metrics	Effective infrastructure anomaly detection in Kubernetes clusters	No financial analysis and no automated correction
Chen et al. (2025), <i>IEEE Trans. Cloud Computing</i>	BPFGuard — multi-granularity container access control	Strengthens runtime protection through eBPF policy enforcement	No cost or FinOps dimension
Prabhu et al. (2025)	Counterfactual deployment event graphs for cost attribution	Explains deployment-driven cost increases by constructing event graphs	Post-event analysis only; no real-time detection and no autonomous action

Research gap, stated for a report

Across the observability, FinOps and runtime-security literature, systems are designed to solve one dimension in isolation. No reviewed work combines real-time operational telemetry, live financial cost modeling, and security-behaviour analysis into a single autonomous decision-and-remediation loop governed by organizational policy. This is the gap HyperTrace targets.

The Prabhu work is worth singling out, because it is the closest neighbour and the most useful citation available. It establishes that deployment-driven cost increases can be explained causally through an event graph — that is, it validates the evidence-chain idea in claim three of chapter 9 as a research contribution rather than a product embellishment. What it does not do is operate in real time or act on its conclusions. HyperTrace's position is therefore defensible as an engineering extension of a published idea into the online, actuating regime, which is a stronger framing than an unsupported novelty claim.

NAMED WEAKNESSES, WITH MITIGATIONS

11. Drawbacks, Risks and Honest Limitations

A strict evaluation panel will respect a project considerably more if it names its own weaknesses before being asked. What follows is a candid list of seven. Each one is a real limitation of the design, not a rhetorical concession, and each mitigation is something the implementation actually does.

11.1 False positives leading to autonomous action on legitimate traffic

Risk. A real flash sale or viral event looks statistically identical, in the first minutes, to a runaway bug: high CPU, high cost, and traffic that is genuinely spiking but that the baseline model has never seen before. Auto-throttling during a real sale is a worse outcome than the problem being solved, and it is the failure that would end the product's credibility with any customer.

Mitigation. Default to *semi-autonomous* mode for high-severity, high-blast-radius actions — recommend, then require one-click human approval — and reserve full autonomy for low-risk, easily reversible actions such as freeze-scaling and alerting, and for high-confidence classifications such as a cryptomining pattern with corroborating network evidence. Make the confidence threshold for full autonomy configurable per policy, and always support instant rollback. The design principle is that the system should be permitted to act unsupervised only where being wrong is cheap.

11.2 Cold-start and baseline-learning problem

Risk. The Behaviour Analysis Engine needs historical data — ideally several weeks, to capture weekly seasonality — before its baseline is trustworthy. A newly onboarded service has no baseline at all, and will therefore be either silent, missing real incidents, or noisy, false-alarming on everything. There is no configuration that avoids both.

Mitigation. Ship with a conservative global default threshold that works from day one at low precision, and replace it gradually with the learned per-service baseline as data accumulates. Surface a “confidence: low / learning” indicator in the interface so that users do not over-trust an immature baseline, and refuse to grant full autonomy to any policy whose service is still in the learning window. Part II, chapter 23 specifies the minimum sample count per bucket before a baseline is considered trustworthy.

11.3 The remediation layer is itself a security and reliability risk

Risk. Giving any component the ability to terminate or quarantine production workloads means a bug in HyperTrace, or a compromise of it, becomes a route to taking down the customer's infrastructure. This is the objection an evaluator with real security experience will raise, and it is the correct objection to raise. The system's value proposition requires it to hold exactly the capability that makes it dangerous.

Mitigation. Least-privilege, narrowly scoped IAM roles per action type; a mandatory audit log for every action carrying a full who-and-what-triggered-it evidence trail; a hard-coded, non-overridable policy tier — never terminate anything tagged production or database — sitting *beneath* the user-configurable policy layer so that no customer configuration can remove it; and rate limiting on how many remediation actions may fire per time window, which bounds the blast radius of a bug in HyperTrace itself to a known number of workloads. Chapter 26 of Part II gives the exact RBAC verbs, the rate limits, and the kill switch.

11.4 Multi-cloud cost normalization is genuinely hard

Risk. AWS, GCP and Azure price similar resources differently, expose billing data through different APIs with different lag and granularity, and tag resources differently. This is precisely why well-funded commercial competitors such as Vantage and CloudZero still have automation coverage gaps per cloud. Claiming full real-time multi-cloud parity in a prototype is not credible and should not be attempted.

Mitigation. Build and demonstrate against one cloud, or against Kubernetes only, which is inherently more portable because the abstractions being observed and acted upon are the same everywhere. Describe multi-cloud as a designed-for extension point — a per-provider adapter interface with a defined contract — and place full multi-cloud automation explicitly in future scope. An interface with one implementation and a written contract is a credible extension point; an interface with three half-implementations is not.

11.5 Cryptomining and abuse detection can be evaded

Risk. A sophisticated attacker can throttle their own miner to stay beneath the anomaly threshold, or time activity to piggyback on legitimate traffic spikes when the baseline expects elevated resource use anyway. A statistical detector with a published threshold is, in principle, a detector an adversary can plan around.

Mitigation. Be explicit that this is a defense-in-depth layer rather than a replacement for endpoint and security tooling such as Falco or GuardDuty. Position HyperTrace as detecting the *financially expensive* subset of anomalies that those tools structurally under-detect, not as a general-purpose security product. Note also that evasion is not free for the attacker: a miner throttled to stay under a three-sigma threshold earns proportionally less, so the detector's real effect is to cap the economic value of a successful intrusion rather than to prevent it outright. That is a legitimate security outcome and should be described as one.

11.6 The monitoring layer has a cost and performance overhead of its own

Risk. eBPF probes and per-node collectors consume CPU and memory. At scale, the tool that saves money has a non-zero infrastructure cost, and a poorly implemented agent can add meaningful latency to the very services it observes. An observer that costs 5 percent of the cluster to detect 4 percent waste is a net loss.

Mitigation. Use sampling and CO-RE eBPF — compile once, run everywhere — for low overhead; benchmark and report agent overhead as part of the evaluation, with a target in the low single-digit percent of CPU per node; and make collection frequency configurable so that the overhead can be traded against detection latency deliberately rather than accidentally. Chapter 28 of Part II makes the platform report its own cost as a first-class metric, which turns this risk into something continuously measured rather than assumed.

11.7 Scope realism for a student or major project

Risk. The full production-grade version of this system, as scoped across chapters 4 to 8, represents a multi-team, multi-year commercial product's worth of engineering. Presenting it as fully built when only a subset functions invites the panel to probe exactly the parts that are not real, which is the worst possible use of question time.

Mitigation. Be explicit in the presentation about what is implemented end to end — one Kubernetes cluster, a synthetic workload, live cost calculation, Z-score anomaly detection, one or two remediation actions, and a working dashboard — versus what is architecture and design only, namely multi-cloud, machine-learning detection, and the full policy DSL. Evaluators consistently reward projects that are honest and precise about scope over projects that overclaim. Part III of this document builds exactly the honest subset, on a laptop, at zero cost.

SRS-STYLE

12. Complete Requirements Specification

12.1 Functional requirements

ID	Requirement
FR-1	The system shall collect CPU, memory, disk and network metrics from every monitored node and pod at a configurable interval, default 10 seconds or less.
FR-2	The system shall collect Kubernetes lifecycle events: pod create, delete, restart and OOM-kill, deployment events, and HPA scaling events.
FR-3	The system shall compute a live estimated cost in dollars per hour per service and namespace using configurable unit pricing.
FR-4	The system shall compute cost per unit of work — per request or per user — wherever request-count telemetry is available.
FR-5	The system shall maintain a rolling behavioural baseline per service and flag statistically significant deviations from it.
FR-6	The system shall classify flagged anomalies into categories — traffic growth, inefficiency or bug, misconfiguration, suspected abuse — with a confidence score.
FR-7	The system shall allow administrators to define remediation policies scoped by tag, namespace or service criticality.
FR-8	The system shall support at minimum four remediation actions: throttle, freeze-scaling, quarantine and terminate.
FR-9	The system shall log every remediation action with full evidence — metrics snapshot, policy matched, timestamp, initiator — in an append-only audit log.
FR-10	The system shall support a recommend-only mode requiring approval, configurable per policy, in addition to full autonomy.
FR-11	The system shall provide a rollback action for any remediation within a defined time window.
FR-12	The system shall expose a dashboard showing live total cost, the most expensive services, and active alerts.
FR-13	The system shall send alert notifications through at least one external channel: Slack, email or webhook.
FR-14	The system shall provide role-based access control distinguishing at minimum an admin or SRE role with full access from a read-only finance role.
FR-15	The system shall retain historical cost and anomaly data for trend and root-cause queries, a minimum of 90 days for the prototype.

12.2 Non-functional requirements

ID	Requirement
NFR-1	Latency. End-to-end time from metric ingestion to anomaly flag shall not exceed a few seconds under normal load. A concrete target must be defined and measured: 5 seconds at the 95th percentile.
NFR-2	Overhead. Collector agent CPU overhead per node shall remain under a defined budget, 3 percent or less, and this figure shall be measured and reported rather than asserted.
NFR-3	Availability. The monitoring and decision pipeline shall degrade gracefully — dropping to sampling, then to alert-only — rather than failing hard under ingestion overload.
NFR-4	Security. Remediation Executor credentials shall follow least-privilege scoping. No component shall hold broader cloud permissions than its specific action requires.
NFR-5	Auditability. Every automated action must be fully reconstructable after the fact from stored evidence alone.
NFR-6	Extensibility. The billing and metrics adapter layer shall be pluggable per cloud provider through an interface, not hard-coded to one cloud.
NFR-7	Usability. The executive dashboard shall be understandable to a non-technical finance stakeholder without training.

12.3 Scope boundaries

These boundaries belong in the report as a numbered subsection, not as a verbal caveat during the presentation. Written down, they read as engineering judgement; offered aloud under questioning, the same words read as retreat.

In scope for the prototype	A single Kubernetes cluster or one cloud account; synthetic and seeded workloads used to demonstrate incidents reproducibly; threshold and Z-score based anomaly detection; policy-gated remediation for two to four action types; a working React dashboard; and a complete audit log.
Out of scope, future work	True multi-cloud parity; machine-learning anomaly detection — Isolation Forest or deep learning — at production accuracy; natural-language querying; carbon-footprint estimation; and cross-cloud workload migration.

CHOICES AND THE ALTERNATIVES REJECTED

13. Technology Stack

Twelve layers, each with the recommended choice and the reasoning, including the alternatives considered and why they were not selected. A stack table without the third column is a list of fashions; with it, it is a design record.

Layer	Recommended	Why, and alternatives considered
Frontend	React with TypeScript, Tailwind CSS, Recharts or D3 for charts	Component reuse across many dashboard widgets; TypeScript catches data-shape bugs against the API contract. Alternative: Vue, equally valid; React has the larger ecosystem of chart and dashboard libraries.
Backend API	FastAPI (Python)	Fast to build, async-native which suits the streaming WebSocket alert feed, and strongly typed through Pydantic. It is the natural fit because the anomaly-detection code is also Python, sharing the numpy, pandas, statsmodels and scikit-learn ecosystem. Spring Boot was considered and would be the stronger choice for a Java-skilled team wanting a more rigid service layer — but pick one and defend one. Part III commits to FastAPI throughout.
Stream processing	The Kafka protocol — Apache Kafka in production, Redpanda locally	Kafka gives a durable, replayable log, which is what makes the hot-path and cold-path dual consumption of chapter 4 possible. Part III runs Redpanda, which speaks the same protocol from a single binary with no ZooKeeper and roughly a fifth of the memory — the client code is identical, so nothing in the design is tied to the choice.
Time-series storage	Prometheus for metrics, TimescaleDB for cost events and analytical queries	Prometheus is the de-facto standard for Kubernetes metric scraping and integrates natively with Grafana and kube-state-metrics. TimescaleDB, a PostgreSQL extension, provides SQL over the cost and anomaly data the dashboard queries analytically.
Relational storage	PostgreSQL	Policies, users, audit logs and service metadata need strong consistency and joins. It is well understood, and it doubles as the TimescaleDB base, which removes an entire operational component.
Runtime telemetry	eBPF via libbpf or BCC, with Cilium's Tetragon as a reference implementation	Kernel-level visibility into process and network behaviour without instrumenting application code. CO-RE eBPF avoids recompilation per kernel version, which is what makes agent distribution practical.
Visualization and ops dashboards	Grafana for the engineering raw-metric view; a custom React dashboard for the business-facing cost and decision view	Do not rebuild what Grafana already does well. Build custom interface only where the project adds value: cost correlation, the policy builder, and the evidence graph.
Containerization	Docker	Standard; every service ships as a container image, which is also what makes the local-only build in Part III possible.
Orchestration	Kubernetes	Needed twice over: as the deployment platform, and as the primary thing being observed and acted upon. HPA patches, NetworkPolicy and pod termination all assume Kubernetes primitives.
Platform for the demo	A local kind or Minikube cluster, with managed EKS or GKE as an optional step up	Demonstrating true multi-cloud is a scope trap. A local cluster costs nothing, can be destroyed and rebuilt in two minutes, and lets incidents be injected deliberately — which a shared cloud account does not. Part III uses kind exclusively.
Authentication	JWT-based auth with RBAC middleware	Stateless and standard; works cleanly with a horizontally scaled API service because no session state has to be shared.
Anomaly detection libraries	<code>numpy</code> and <code>statsmodels</code> for Z-score and moving average; <code>scikit-learn</code> Isolation Forest as an optional stretch	Keep the core detector simple and explainable. Explainability matters more than model sophistication when the question “why did it flag this?” has to be answered in front of a panel, and when the answer authorises an automatic action.

PSEUDOCODE WITH REASONING

14. Core Algorithms

Four algorithms carry the intelligence of the system. They are presented here in pseudocode, at the level of detail appropriate for a report; Part III implements all four as working Python.

14.1 Moving-average and EWMA baseline

14.1 exponentially weighted moving average baseline

```
baseline[service] = EWMA(historical_cost_series, alpha=0.2)
deviation = (current_cost - baseline[service]) / baseline[service]
if deviation > THRESHOLD:      # e.g. 3x baseline
    flag_anomaly(service, "cost_spike", deviation)
```

An exponentially weighted moving average is the cheapest useful baseline: it needs one number in memory per series rather than a window of history, and it adapts automatically as behaviour drifts. The `alpha` parameter sets how fast it forgets — at 0.2, roughly the last ten observations dominate. This is the right detector for a first implementation and the wrong one to stop at, because a ratio against a single mean has no notion of how variable the service normally is. A service whose cost routinely doubles and halves will trip a 3× threshold constantly; a service that is normally rock-steady will not trip it even when something is clearly wrong.

14.2 Z-score anomaly detection, the core explainable detector

14.2 Z-score against a seasonal baseline

```
mean, stddev = rolling_stats(service_metric_history,
                             window="14d", bucket="same_hour_of_week")
z = (current_value - mean) / stddev
if abs(z) > 3:
    flag_anomaly(service, metric, z_score=z)
```

The Z-score fixes exactly the deficiency just described: it measures deviation in units of the service's own variability, so one threshold is meaningful across services with completely different volatility. The important detail is not the arithmetic but the `bucket` parameter. Bucketing by same hour of week is what lets the model treat the Monday nine-to-eleven traffic doubling from chapter 3 as normal rather than anomalous; it is the practical implementation of the “learn normal behaviour” idea. Without it, every Monday morning generates a false positive, and a detector that cries wolf weekly is a detector that gets switched off. Part II, chapter 23 develops the 168-bucket scheme, the warm-up policy and the state-update rule in full.

14.3 Joint decision logic — the actual differentiator

14.3 joint decision logic across cost, traffic and security signals

```
if cpu_z > 3 and traffic_z < 1 and cost_z > 3:
    if egress_to_unusual_geo or process_signature_matches_miner:
        classification = "suspected_abuse"
    elif recent_deployment_within(minutes=15):
        classification = "likely_bug_from_deployment"
    else:
        classification = "misconfiguration_or_waste"
elif cpu_z > 3 and traffic_z > 3:
    classification = "legitimate_traffic_growth" # no action

decision = policy_engine.evaluate(classification,
                                  service.criticality_tag,
                                  org_policies)
```

This block is short, and it is the project. Read the first condition carefully: it requires CPU high, cost high, *and traffic not high*. The conjunction is the contribution. Any single clause on its own is a commodity threshold available in any monitoring tool; the conjunction expresses a hypothesis about causation, namely that resource consumption which is not explained by demand is consumption that needs explaining some other way. The nested branches then attempt that explanation, in order of how much corroborating evidence they demand: security evidence first, then deployment correlation, then waste as the residual classification when nothing else fits.

The final `elif` is as important as the first branch, and easier to overlook. High CPU with high traffic is explicitly classified as legitimate growth and explicitly generates no action. A detector needs a defined shape for normal just as much as it needs one for abnormal, and the most expensive mistake this system can make is acting during a real commercial success.

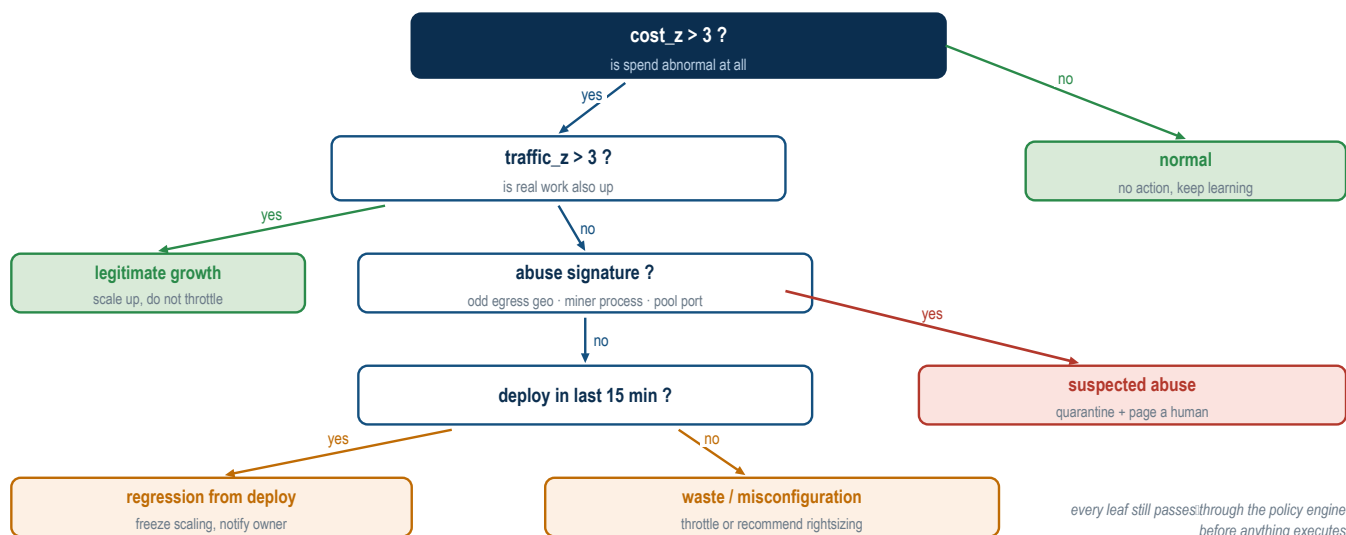


Figure 8 - Classification tree. The tree only produces a proposed class; the policy engine decides whether the organisation permits acting on it.

14.4 Policy evaluation

14.4 first-match policy evaluation, highest priority first

```

for rule in sorted(org_policies, key=priority, reverse=True):
    if rule.matches(service.tags, classification, cost_rate):
        return rule.action    # "throttle", "alert_only", "quarantine"

return default_action        # conservative fallback: alert_only
  
```

First-match-wins evaluation in descending priority order, with a conservative default. Two properties of this loop matter more than its brevity suggests. It is **total**: every possible input produces a decision, because the fallback cannot be reached and fail. And it is **conservative by construction**: the fallback is alert-only, so a policy set that fails to match anything degrades to notifying a human rather than to acting or to silence. Part II, chapter 25 turns this loop into a real policy language with scope matching, conflict resolution, dry-run simulation, and its own audit trail.

NOTE

Isolation Forest, optional and stretch. A multivariate unsupervised model trained on the vector of CPU, memory, network, cost and request count per service is useful for catching anomalies that do not show up on any single metric's Z-score but are unusual in combination. Mention it as future work if there is no time to validate it properly. An unvalidated machine-learning claim is worse than an honestly scoped statistical one, because the first invites questions that cannot be answered and the second does not.

HARD QUESTIONS, STRONG ANSWERS

15. Presentation Defense Kit

Ten questions, each with an answer that concedes what is true before asserting what is defensible. The order matters: the first two are the most likely to be asked and the most dangerous to answer badly.

Q1. AWS already has CloudWatch, Cost Explorer, Budgets, GuardDuty and Auto Scaling. Why build this?

Each of those tools answers one question in isolation — is it high, did we spend more, was there a threat, do we need more capacity. None of them jointly reasons across all four to decide whether a spike is legitimate and then act automatically within a policy boundary. HyperTrace's contribution is that decision-and-policy layer. It is not a replacement for the underlying data sources; it consumes them.

Q2. Isn't this just Kubecost plus Falco plus a Lambda function?

You could approximate parts of it that way, and that is a fair characterization of the engineering effort involved. But Kubecost does not see security telemetry, Falco does not see cost, and neither correlates the two into one classification before deciding whether to act. Wiring them together with glue code is exactly the unified decision layer this project proposes to build properly — with an explainable policy engine and an audit trail — rather than as an ad-hoc integration that no one can reason about six months later.

Q3. What happens when your system wrongly throttles a production service during a real traffic spike?

That is why high-blast-radius actions default to recommend-with-approval rather than full autonomy, why there is a hard-coded policy floor protecting tagged-critical services that no customer configuration can remove, and why every action is instantly reversible with a logged rollback. Full autonomy is reserved for low-risk actions and very high-confidence classifications. The design principle is that the system acts unsupervised only where being wrong is cheap.

Q4. How is this secure? You are giving a monitoring tool the power to kill production workloads.

The Remediation Executor uses least-privilege, action-scoped IAM roles — the exact Kubernetes verbs are enumerated in the design, and the role cannot delete arbitrary resources. Every action is audit-logged with full evidence in an append-only table, and there is a rate limit on total actions per time window so that a bug in the tool can affect a bounded number of workloads before a human notices. There is also a global kill switch that reverts the entire platform to alert-only in one operation.

Q5. Is this actually multi-cloud, as you claim?

The architecture is designed around a pluggable per-cloud adapter interface, but the working prototype is demonstrated on Kubernetes with a simulated pricing table. True multi-cloud billing normalization is a hard problem — even funded commercial competitors such as Vantage and CloudZero have coverage gaps per cloud — so I treat it as a defined extension point and future work, not a shipped feature. The interface exists and has one real implementation; that is an honest extension point.

Q6. What is your detection false-positive rate?

Give an actual measured number from synthetic-incident testing: across N injected incidents and M hours of normal synthetic traffic, precision was X percent and recall Y percent. This is far stronger than any qualitative claim, and evaluators specifically probe for whether anything was measured at all. Part III includes a measurement harness that produces exactly these numbers, so the figure quoted in the defense is reproducible rather than remembered.

Q7. Your cost figure is an estimate from telemetry. What happens when it diverges from the actual cloud bill?

It will diverge, and the system is built expecting that. The Billing Adapter polls the provider's authoritative billing export on a slow loop and computes a drift metric — estimated spend against billed spend for the same window. Drift within a few percent is treated as normal, since the estimate excludes items like cross-zone traffic that telemetry cannot see. Sustained drift beyond that is itself surfaced as an alert about the cost model, not about the workload. The estimate is used for detecting change, where relative accuracy is what matters; the bill remains the source of truth for absolute figures, and the interface labels which of the two a given number is.

Q8. Why not just set an AWS Budget alert at a dollar threshold?

A budget alert answers a question about a level, and the incidents this project targets are about a rate. A runaway retry loop that will consume \$9,000 over a weekend crosses no threshold in its first hour, which is the hour in which stopping it is cheap. Budgets are also evaluated against billing data, so they inherit its lag, and they cannot express the traffic-decoupling condition at all — a budget has no idea whether the spend was earned. Budgets are a useful backstop and the two are complementary, but a backstop that fires after the money is spent is not a detector.

Q9. How does the baseline handle a service that legitimately grows 20 percent week over week?

Two mechanisms. The baseline is exponentially weighted, so sustained growth is absorbed into the mean within roughly two weeks rather than flagged indefinitely. And because the detector requires cost and traffic to *decouple*, growth that is matched by request volume never satisfies the anomaly condition in the first place, no matter how large it is. The case that does produce a flag is a permanent step change that is not explained by traffic — a deployment that made the service structurally more expensive per request. Flagging that once is correct behaviour, and the operator can then accept it, which rebases the baseline immediately instead of waiting for it to adapt.

Q10. If you had to point at the single most novel line of code in the project, which is it?

The conjunction in section 14.3: `cpu_z > 3 and traffic_z < 1 and cost_z > 3`. Every clause in it is ordinary. The conjunction is not, because it encodes a hypothesis rather than a threshold: resource consumption that is not explained by demand requires a different explanation, and the system's job is to find it and act on it within policy. Everything else in the architecture exists to make that one line computable in real time, explainable after the fact, and safe to act on.

WHAT THE PROJECT ACTUALLY CLAIMS

16. Conclusion to Part I

HyperTrace, also proposed as CloudAegis, is a credible and well-scoped systems project precisely because its value proposition is an **integration and decision-making gap** in a real, currently competitive market — not a fictional gap where nothing like it exists. That framing survives contact with an informed panel, and the alternative does not.

The strongest version of this project for a strict evaluation does four things. It names the real competitors accurately, as chapters 7 and 8 do, including the ones that do parts of this better. It claims only the specific joint-reasoning and policy-driven-automation gap as its innovation, as chapter 9 does, and explicitly disclaims the broader versions of that claim. It builds a working, narrowly scoped, honestly labelled prototype rather than an overclaimed one, as chapters 11 and 12 scope and Part III implements. And it enters the defense with the objections of chapter 15 already answered before anyone asks them.

What follows is the engineering. Part II expands every component named in chapter 4 into a specified design: the topic layout, the schema, the cost arithmetic with a worked numeric example, the baseline state machine, the policy language, the safety interlocks around the one component permitted to change the world, and the failure modes with their designed degradation. Part III then builds the honest subset from nothing, on a laptop, at zero cost, with every command and every file given in full.

IF YOU REMEMBER ONE LINE FROM PART I

One sentence to carry into the room: the innovation is not a new measurement, it is a new conjunction of existing measurements, evaluated fast enough to act on and explained well enough to be trusted with the authority to act.

Detailed Architecture & Systems Design

The complete engineering blueprint across thirteen deep-dive chapters. Establishes the seven core architectural principles, C4 container views, hot/cold data paths, Kafka topic design, TimescaleDB schema, pricing arithmetic, rolling Z-score baselines, policy engine mechanics, and security controls.

Chapters 17–20	Principles, C4 views, hot/cold latency paths, event bus schemas
Chapters 21–24	Database DDL, cost model math, rolling baselines, classification
Chapters 25–27	Policy engine, autonomous remediation, and failure modes
Chapters 28–29	Platform security model and multi-cloud deployment topologies

THE RULES EVERYTHING ELSE OBEYS

17. Architecture Principles and Constraints

Part I described what the system does. Part II specifies how it is built, at the level of detail an implementer would need and an evaluator would probe. It begins with principles, because every subsequent decision in this part is derived from one of them, and because a principle is only useful if it rules something out.

17.1 Seven principles

Principle	What it rules out	Why
Read-only by default; exactly one privileged writer	Convenience credentials in collectors, the cost engine, or the API; any component patching Kubernetes objects “just for this one case”	It reduces the security review of the whole platform to the RBAC verbs of a single service account. Any other arrangement means the blast radius is the union of every component's permissions.
Every decision reconstructable from stored evidence	Decisions that depend on in-memory state not written anywhere; log lines as the only record of why something happened	An automatic action taken on production infrastructure that cannot be explained afterwards is indistinguishable from a bug, and will be treated as one by whoever is paged.
Degrade authority before accuracy	Continuing to act autonomously when the pipeline is behind, sampling, or missing a signal	A detector working from partial data may still be usefully accurate as an alert, but it has lost the standing to act unsupervised. Authority is the first thing to give up under stress, not the last.
Policy is data, not code	Thresholds and protected-namespaces lists compiled into the binary; customer-specific branches in the decision path	Policy changes weekly and must be audited, simulated and rolled back by people who do not deploy software. Anything in the source tree cannot offer those properties.
The cost figure is an estimate and must be reconcilable	Presenting a telemetry-derived number as a bill; any design where the estimate cannot be compared against the provider's authoritative total	The estimate exists because it is fast, not because it is right. A fast number that is never checked against a true one becomes wrong silently, and then becomes the reason nobody trusts the product.
Single-cloud prototype, adapter seams for more	Provider-specific field names or pricing assumptions leaking past the adapter boundary into the cost engine or the detector	Multi-cloud is future scope, but retrofitting a seam through a codebase that assumed one provider is a rewrite. One implementation behind a written interface is cheap; three leaky ones are not.
Prefer the least aggressive action that contains the incident	Terminating as a first response; treating all four actions as interchangeable responses to “an anomaly”	The four actions differ by orders of magnitude in reversibility. A system that does not encode that ordering will eventually use the wrong end of it on the wrong workload.

17.2 Hard constraints

Principles constrain design; numbers constrain implementation. The following figures are the ones an implementer must design against, and the ones the evaluation in chapter 28 measures against. Each is stated as a budget rather than an aspiration, meaning that exceeding it is a defect.

Constraint	Value	Consequence if exceeded
Metric collection interval	10 s default, 5 s minimum, 60 s maximum	Below 5 s the agent overhead budget cannot be met; above 60 s a runaway can double the cluster between two samples
End-to-end detection latency	5 s at p95, measured from sample timestamp to anomaly row committed	Remediation arrives after the autoscaler has already multiplied capacity, which removes most of the value
Agent CPU overhead	3% of one core per node, hard ceiling; 1% target	The platform becomes a material line item on the bill it is auditing
Agent memory overhead	128 MiB resident per node	Agent becomes an OOM-kill risk on small nodes, taking observability down during load
Raw metric retention	7 days at full resolution, 90 days at 5-minute rollup	Storage cost grows without bound; investigative queries slow to unusable
Metric cardinality ceiling	50,000 active series per cluster	TimescaleDB chunk indexes stop fitting in memory and ingest latency climbs nonlinearly

Constraint	Value	Consequence if exceeded
Actions per rolling window	10 per 5 minutes globally, 1 per service per 15 minutes	A bug in HyperTrace becomes an outage in the customer's cluster
Rollback window	60 minutes from action execution	Beyond this, the recorded prior state is no longer a safe thing to restore

THE ONE THAT BITES

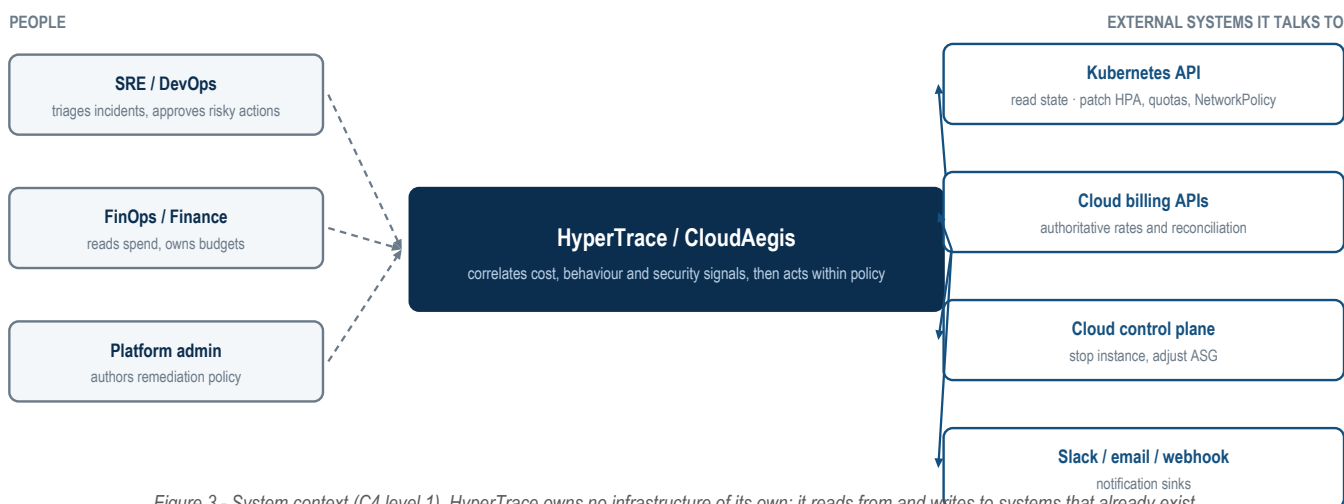
The cardinality ceiling is the constraint most often discovered too late. It is not a storage limit — it is an index-locality limit, and it is why the design in chapter 21 deliberately refuses to put pod names into metric labels. A cluster that recreates pods on every deploy will manufacture unbounded cardinality if allowed to, and the symptom is not an error but a slow, permanent degradation of the ingest path.

CONTEXT, CONTAINERS, COMPONENTS

18. The Four Views

The architecture is presented in the C4 style: successive zoom levels, each of which is complete at its own level of abstraction. The system-context view establishes what is inside the system and what merely touches it. The container view enumerates the deployable processes. The component view opens the two processes that contain real algorithms. The fourth view, deployment, is chapter 29.

18.1 System context



Three things are worth noticing in the context view, all of them about what is *outside* the boundary. The observed Kubernetes cluster is external: HyperTrace has no privileged position inside the workloads it watches, and the only channel back into them is the single arrow from the Remediation Executor to the Kubernetes API server. The cloud billing API is external and one-directional: the platform reads it and never writes to it, which is what makes the reconciliation loop safe to run against a production account. And the notification targets are external sinks with no return path, so a failure in Slack or PagerDuty can never propagate back into a decision.

The two human actors are separated deliberately, and the separation drives the RBAC model. An SRE has authority over policy and over individual actions. A finance stakeholder has read access to cost and no authority over infrastructure at all. There is no role that combines the finance view with remediation authority, because the person who most wants a workload throttled to save money is the person least equipped to judge whether throttling it will break a customer.

18.2 Container view

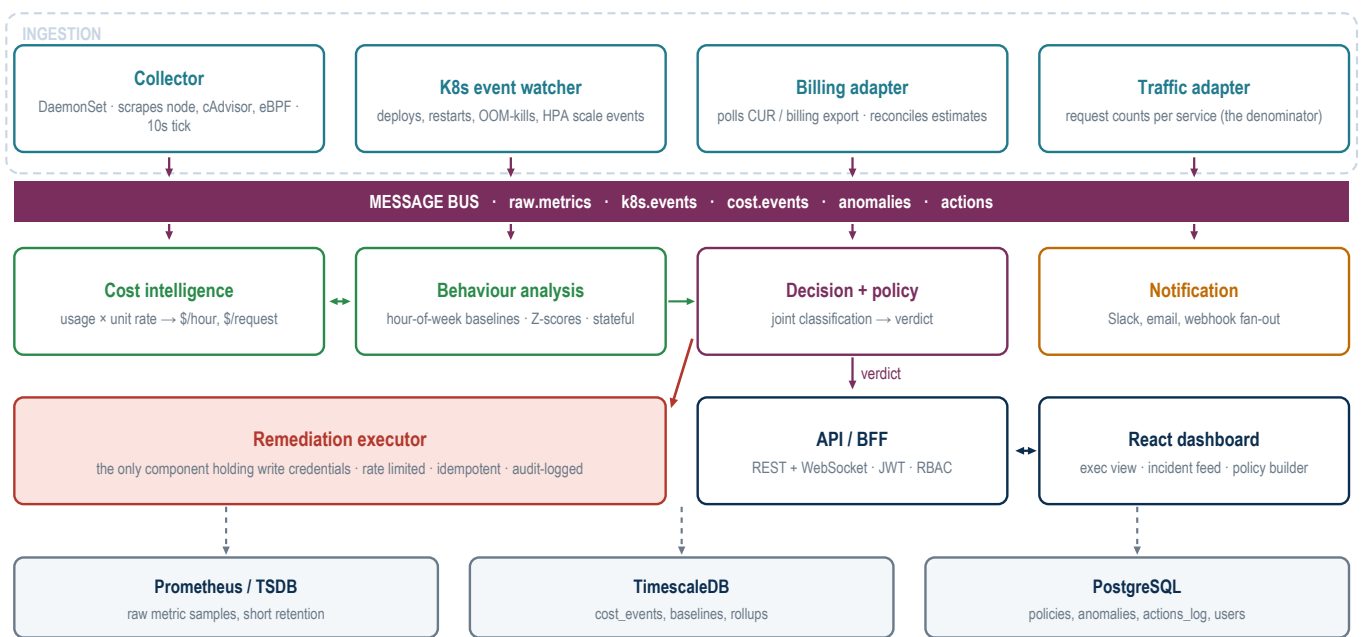


Figure 4 - Container view (C4 level 2). Note that exactly one service holds credentials able to change the cluster; every other service is read-only by construction.

Each of the eight services is specified below along the four dimensions that actually matter operationally: what it is responsible for, what it reads, what it writes, and what happens when it dies. The last column is the one usually omitted from architecture documents and the one most worth writing down, because it is the column that determines whether the system fails safe.

Service	Reads	Writes	If it dies
Collector (DaemonSet)	cAdvisor, node exporter, kube-state-metrics, eBPF ring buffer, local kubelet read-only port	metrics.raw and events.k8s on the bus	That node goes dark. Other nodes are unaffected. Detection for workloads on the node degrades to nothing, so the Behaviour Engine marks their baselines stale rather than treating absent load as low load.
Billing Adapter (CronJob)	Cloud billing export or, locally, a static price table	prices table; reconciliation rows	Pricing goes stale. The cost engine keeps using the last-known rates, so estimates drift slowly rather than stopping. Drift alerting stops, which is itself alerted on.
Cost Intelligence (Deployment, consumer group)	metrics.raw , prices	cost.events on the bus; cost_events hypertable	Consumer lag grows on a durable topic, so nothing is lost. On restart it resumes from its committed offset and catches up. Live cost figures freeze at their last value, and the interface must show a staleness indicator rather than a stale number presented as current.
Behaviour Analysis (StatefulSet, partitioned)	metrics.raw , cost.events , baselines	anomalies topic and table; baselines checkpoints	The affected partition stops detecting. In-memory baseline state is recovered from the last checkpoint and replayed forward from the bus, so the cost is minutes of detection blindness, not the loss of weeks of learning.
Decision / Policy (Deployment, stateless)	anomalies topic, policies , services	decisions topic; anomalies.status	Anomalies accumulate undecided on a durable topic. Nothing acts, which is the safe direction to fail. Alerts continue only if the Notification Service is configured to subscribe to anomalies directly as well as decisions.
Remediation Executor (Deployment, single replica)	decisions topic, policies floor	Kubernetes API (patch and delete); actions_log	No actions execute. The system reverts to advisory behaviour with no loss of detection or alerting. This is the intended failure mode and the reason this service is deliberately a single replica rather than a highly available set.
Notification (Deployment, stateless)	decisions and anomalies topics	External sinks; notifications_log	Humans stop being told. Actions still execute, which is the most dangerous single-service failure in the platform and the reason it is monitored as a critical dependency rather than a peripheral one.

Service	Reads	Writes	If it dies
API / BFF (Deployment, stateless, HPA)	All read models; PostgreSQL and TimescaleDB	Only policies and approval or rollback intents	The dashboard goes down. Detection and remediation continue entirely unaffected, because the API is a consumer of the pipeline and never a participant in it.

One pattern runs through the last column and is worth naming. Every failure in this system either loses observation or loses authority, and never gains it. There is no component whose death causes an action to be taken; there are several whose death causes actions to stop. That asymmetry is not accidental — it follows from putting the only writer at the end of the pipeline, so that every upstream failure starves it rather than confusing it.

18.3 Component view — Cost Intelligence Service

The cost engine is a stream processor with five internal modules. It is the only component in the system whose correctness is arithmetic rather than logical, which makes it the easiest to unit-test and the easiest to be subtly wrong in.

Module	Responsibility	Notable detail
Sample normalizer	Converts heterogeneous metric shapes into a canonical (resource, dimension, value, interval) tuple	Rate metrics arrive as counters and must be differenced; gauge metrics must not be. Getting this backwards produces cost figures wrong by orders of magnitude and is the single most common implementation bug.
Ownership resolver	Maps a resource identifier to a service, namespace, team and criticality tag	Cached with a short TTL against the services table. Resources that resolve to nothing are attributed to a synthetic unallocated service rather than dropped, so the total always reconciles.
Rate lookup	Resolves the unit price for a (resource_type, region, dimension) triple at the sample's timestamp	Prices are versioned with validity intervals, so recomputing history uses the rates that were in force then, not today's.
Direct cost calculator	Applies rates to normalized consumption to produce a per-resource \$/hour	Pure function with no I/O; the whole of chapter 22's arithmetic lives here and is unit-testable against hand-computed fixtures.
Shared cost amortizer	Distributes cluster-level cost that belongs to no single workload	Requires a cluster-wide denominator, so it runs on a short tumbling window rather than per sample. This is the only stateful part of the service.

18.4 Component view — Behaviour Analysis Service

The behaviour engine is the stateful heart of the system. Its five modules are arranged so that all mutable state lives in exactly one of them, which is what makes checkpointing tractable.

Module	Responsibility	Notable detail
Bucket router	Maps a sample timestamp to one of 168 hour-of-week buckets in the service's local timezone	Timezone matters: bucketing in UTC smears a business-hours pattern across two buckets for any service whose users are not on UTC, roughly halving the detector's sensitivity.
Baseline store	Holds the mean, variance and sample count per (service, metric, bucket) and is the only mutable state in the service	Roughly 168 buckets × 6 metrics × 4 numbers per service. Small enough to hold in memory for thousands of services, which is what permits the p95 latency budget.
Updater	Applies the exponentially weighted mean and variance recursion of chapter 23	Update is skipped, not zero-filled, for missing samples. Zero-filling a gap teaches the baseline that idle is normal, which suppresses detection for exactly the services that were recently down.
Scorer	Computes Z-scores for each metric against its bucket, and the derived confidence score	Refuses to score against a bucket that has not met the maturity threshold, falling back to the pooled service baseline and then to a global default.
Classifier	Applies the joint decision logic and attaches the evidence payload	Emits the evidence it used, not a summary of it, so the anomaly row is sufficient to reconstruct the decision without re-querying metrics that may since have been rolled up.

TWO SPEEDS, ONE EVENT STREAM

19. Data Flow: Hot Path and Cold Path

The system reads the same event stream twice, at two different speeds, for two different purposes. Everything that must happen before a decision is taken lives on the hot path and is budgeted in milliseconds. Everything that only has to be true eventually lives on the cold path and is budgeted in minutes. Conflating the two is the most common way a design like this fails: a single path that must serve both a five-second decision and a ninety-day analytical query will be tuned for neither.

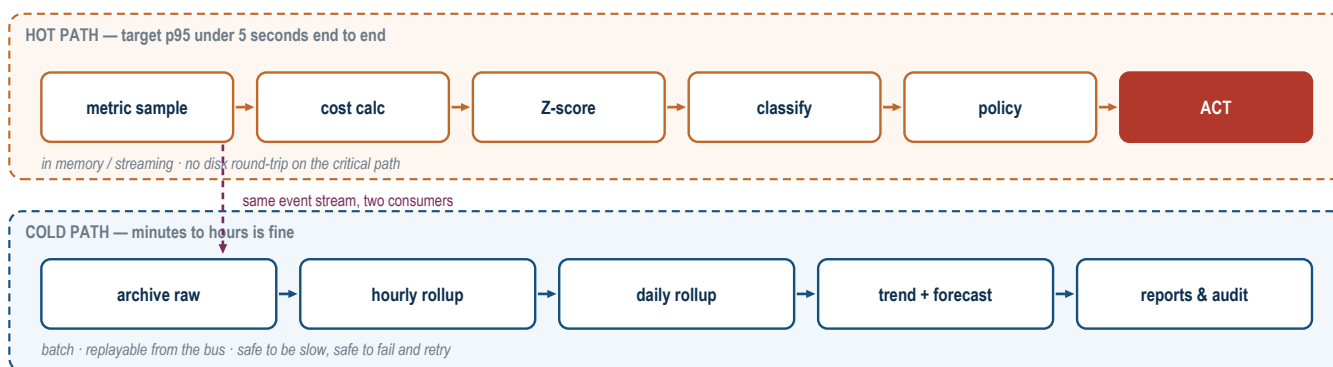


Figure 5 - Two consumers of one event stream. Splitting the paths means a slow analytical query can never delay an incident response.

19.1 Hot path latency budget

The NFR-1 target is 5 seconds at the 95th percentile from sample timestamp to anomaly row committed. That number is only meaningful if it is decomposed, because an undecomposed end-to-end target cannot be debugged when it is missed. The budget below sums to 4.4 seconds, leaving 600 ms of headroom.

Hop	Budget (p95)	What consumes it
Sample to collector scrape	500 ms	Scrape alignment jitter. A 10-second scrape interval means a sample can be up to one interval old, but the timestamp travels with the sample, so only alignment jitter counts against the budget.
Collector to bus acknowledged	150 ms	Batch fill time plus one network round trip plus the broker's fsync. Batching is capped at 100 ms of linger so that a partly full batch cannot sit waiting.
Bus to Cost Intelligence consumed	300 ms	Consumer poll interval and fetch minimum bytes. Set fetch.min.bytes low on this topic; throughput is the cold path's concern, not this one's.
Cost computation	80 ms	Rate lookup from a warm cache, arithmetic, ownership resolution. All in-memory; any database round trip here would blow the hop.
Cost event to Behaviour Engine	300 ms	Second bus hop, same tuning as the first.
Baseline update and scoring	120 ms	In-memory recursion over six metrics plus confidence computation. No I/O.
Classification and evidence assembly	150 ms	Assembling the evidence payload, including one cached lookup for recent deployment markers.
Anomaly persisted and published	400 ms	One PostgreSQL insert plus one bus publish. The insert is what makes the decision reconstructable, so it is inside the budget deliberately.
Policy evaluation	100 ms	In-memory rule set, sorted once on load. Policies are re-read on change notification, never per evaluation.
Decision to action attempted	2,300 ms	Dominated by the Kubernetes API call itself, including admission webhooks on the target object. This hop is outside the platform's control and is why the remaining budget is kept tight.

19.2 What is deliberately excluded from the hot path

Four things could plausibly be on the hot path and are not, each for a stated reason. Excluding work is a design decision and should be recorded as one.

- **Authoritative billing lookups.** The provider's billing API has latency measured in seconds and availability outside the platform's control. It is polled on the cold path and used only for reconciliation, never to make a decision.
- **Historical query for context.** It is tempting to ask "has this happened before?" before deciding. That is a range query over a hypertable and would add unbounded latency; instead, the small amount of history the classifier needs —

recent deployment markers, recent actions on the same service — is held in a bounded in-memory cache updated from the bus.

- **Notification delivery.** A Slack call can take seconds or fail. Notification consumes the decision topic asynchronously, so a slow webhook can never delay an action or an audit write.
- **Machine-learning inference.** Even a fast model adds a variable tens of milliseconds and, more importantly, adds an unexplainable term to a decision that must be explainable. If Isolation Forest is added at all, it scores on the cold path and contributes only to a follow-up advisory, never to an autonomous action.

19.3 The incident lifecycle, end to end

The figure below traces a single incident across all eight lanes, from the first elevated sample to the audit record and the operator's rollback decision. It is the same pipeline as the layer stack in chapter 4, drawn against time instead of against abstraction, and it is the diagram to have open when reasoning about latency or about what evidence exists at each moment.

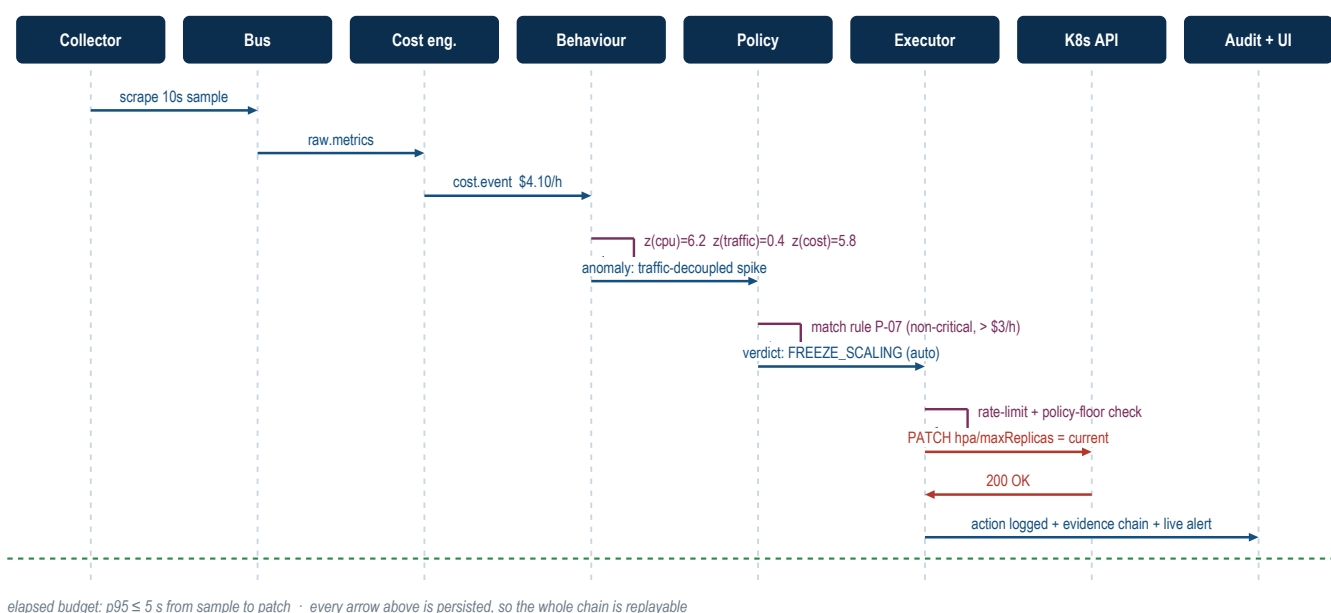


Figure 7 - Incident lifecycle, one runaway workload. The self-calls on the Behaviour and Policy lanes are where the reasoning happens; everything else is plumbing.

19.4 The cold path and replay

The cold path consumes the same topics with a different consumer group, a larger fetch size and no latency requirement. It writes the raw metric archive, maintains the continuous aggregates that dashboards query, runs the billing reconciliation job, and computes the daily rollups that make ninety-day queries fast.

Its most valuable property is replay, and replay is worth stating concretely because it is the concrete payoff of choosing a durable log over a queue. Suppose a bug is found in the shared-cost amortizer: for three weeks, egress-heavy pods have been over-charged and compute-heavy pods under-charged. Because `metrics.raw` is retained and `prices` is versioned by validity interval, the fix is a rerun rather than an apology.

```
replaying the cost model over a historical window

# Recompute a window of cost history after fixing the cost model.
# 1. Deploy the corrected Cost Intelligence Service.
# 2. Start a NEW consumer group so the live group is untouched.
# 3. Write to a shadow table, compare, then swap.

python -m hypertrace.costengine.replay \
  --from 2026-07-01T00:00:00Z \
  --to 2026-07-21T00:00:00Z \
  --consumer-group cost-replay-2026-07 \
  --output-table cost_events_shadow \
  --price-asof event # use rates valid at event time, not now
```

Two details in that command carry the whole idea. A separate consumer group means the replay cannot disturb live detection, because offsets are tracked per group. And `--price-asof event` means the recomputation uses the rates that were in force at event time rather than current rates, which is the difference between correcting history and rewriting it.

DO THIS

Replay is also the honest answer to a question the panel may ask about trustworthiness: what happens when you discover your cost model was wrong? A system without a durable raw log has to answer “the history stays wrong.” A system with one answers “we recompute the affected window from original inputs and show both versions.” That difference is worth the operational cost of running a broker.

THE CONTRACT BETWEEN SERVICES

20. Event and Topic Design

The bus is the only integration point between services, which makes the topic layout and the event schemas the real interface specification of the platform. Six topics, each with one producer and a defined ordering requirement.

20.1 Topic layout

Topic	Key	Partitions	Retention	Producer	Consumers	Ordering
<code>metrics.raw</code>	<code>service_id</code>	12	7 days	Collector	Cost Intelligence, Behaviour, archive writer	Per key
<code>events.k8s</code>	<code>namespace</code>	3	30 days	Collector	Behaviour, archive writer	Per key
<code>cost.events</code>	<code>service_id</code>	12	30 days	Cost Intelligence	Behaviour, aggregate writer, API cache	Not required
<code>anomalies</code>	<code>service_id</code>	6	90 days	Behaviour	Decision, Notification, API	Per key
<code>decisions</code>	<code>service_id</code>	3	90 days	Decision	Remediation Executor, Notification	Per key, strict
<code>actions</code>	<code>service_id</code>	3	365 days	Remediation Executor	Notification, API, audit writer	Per key

20.2 Why partition by service, and where ordering actually matters

Every topic is keyed by `service_id`, which guarantees that all events for one service land on one partition and are therefore consumed in order by one consumer. This single decision does two jobs at once. It gives the Behaviour Engine the ordering it needs, and it gives that engine a natural sharding key, since a consumer that owns a partition owns the complete baseline state for every service on it and never has to coordinate with its peers.

Ordering matters asymmetrically across the topics, and understanding where it does not matter is as useful as knowing where it does. Baseline updates are order-sensitive: the exponentially weighted recursion of chapter 23 is not commutative, so processing Tuesday's sample before Monday's produces a different mean than the reverse. Cost events are order-insensitive, because each one is an independent arithmetic result over a bounded interval and they are only ever summed. Decisions are the strictest case: a throttle followed by a rollback and a throttle followed by a rollback in the other order leave the workload in opposite states, so this topic is single-partition-per-service and its consumer is deliberately not

parallelised within a key.

20.3 Core event schemas

Four schemas cover every message on the bus. All four share an envelope carrying `schema_version`, `event_id`, `emitted_at` and `producer`, which is what makes replay and idempotency possible without per-topic special cases.

`metrics.raw` - a raw metric sample

```
{
  "schema_version": 1,
  "event_id": "01J8Z9K3M4N5P6Q7R8S9T0V1W2",
  "emitted_at": "2026-08-26T10:31:04.118Z",
  "producer": "collector/kind-worker2",
  "window_start": "2026-08-26T10:30:54.000Z",
  "window_end": "2026-08-26T10:31:04.000Z",
  "resource": {
    "kind": "pod",
    "uid": "3f1c8a52-91bd-4f0e-9c77-6a2b8e4d0a11",
    "namespace": "shop",
    "workload": "checkout-api",
    "node": "kind-worker2"
  },
  "samples": {
    "cpu_cores": 0.8471,
    "mem_working_gib": 1.502,
    "net_egress_bytes": 33333333,
    "net_ingress_bytes": 9182731,
    "disk_read_bytes": 0,
    "disk_write_bytes": 40960,
    "http_requests": 118
  }
}
```

Note that the sample carries a window rather than an instant. Counters are differenced by the collector, not by the consumer, because the collector is the only component that knows whether the counter reset in between — a pod restart resets counters to zero, and a consumer that differenced naively would record a large negative egress or, worse, treat the reset as a spike.

`cost.events` - a derived cost event

```
{
  "schema_version": 1,
  "event_id": "01J8Z9K3P7R2S4T6V8W0X2Y4Z6",
  "emitted_at": "2026-08-26T10:31:04.402Z",
  "producer": "costengine-7d9f",
  "derived_from": ["01J8Z9K3M4N5P6Q7R8S9T0V1W2"],
  "service_id": "svc_checkout",
  "window_start": "2026-08-26T10:30:54.000Z",
  "window_end": "2026-08-26T10:31:04.000Z",
  "price_set_version": "2026-08-01",
  "components_usd_per_hour": {
    "cpu": 0.024990,
    "memory": 0.005933,
    "egress": 1.080000,
    "storage": 0.002192,
    "shared": 0.077800
  },
  "total_usd_per_hour": 1.190915,
  "requests_in_window": 118,
  "usd_per_1k_requests": 0.028035,
  "estimate": true
}
```

Three fields in this schema exist purely to keep the platform honest. `derived_from` links the derived event back to the raw samples that produced it, so an evidence chain can be walked backwards. `price_set_version` records which rate table was used, so a recomputation can be compared meaningfully against the original. And `estimate` is a literal boolean that is always `true` on this topic, because the reconciliation loop of chapter 22 produces rows where it is false, and no consumer should ever have to infer which kind of number it is holding.

`anomalies` - a scored, classified anomaly with its evidence

```
{
  "schema_version": 1,
  "event_id": "01J8Z9K48A0B1C2D3E4F5G6H7J",
  "emitted_at": "2026-08-26T10:31:04.836Z",
  "producer": "behaviour-1",
  "anomaly_id": 90412,
  "service_id": "svc_checkout",
  "bucket": 82,
  "classification": "likely_bug_from_deployment",
  "confidence": 0.79,
  "scores": { "cpu_z": 6.41, "cost_z": 5.88, "traffic_z": 0.22 },
  "evidence": {
    "baseline": { "cpu_mean": 0.31, "cpu_sd": 0.084, "n": 744 },
    "observed": { "cpu_cores": 0.8471, "usd_per_hour": 1.1909 },
    "cost_delta_usd_per_hour": 0.981,
    "deployment_marker": {
      "revision": "checkout-api-6f4b9c",
      "at": "2026-08-26T10:23:41Z"
    },
  },
  "egress_geo_unusual": false,
  "miner_signature": false
},
"baseline_mature": true
}
```

`actions` - an executed action record

```
{
  "schema_version": 1,
  "event_id": "01J8Z9K5C3D4E5F6G7H8J9K0M1",
  "emitted_at": "2026-08-26T10:31:07.219Z",
  "producer": "executor-0",
  "idempotency_key": "90412:freeze_scaling:shop/checkout-api",
  "anomaly_id": 90412,
  "policy_id": 17,
  "action": "freeze_scaling",
  "mode": "autonomous",
  "target": { "kind": "HorizontalPodAutoscaler",
    "namespace": "shop", "name": "checkout-api" },
  "prior_state": { "maxReplicas": 12 },
  "applied_state": { "maxReplicas": 4 },
  "result": "succeeded",
  "rollback_deadline": "2026-08-26T11:31:07Z"
}
```

The `prior_state` field is what makes rollback possible, and it is captured by the executor immediately before the patch rather than reconstructed afterwards. This ordering is important: reading the object's state after patching it, or trusting the value the decision was based on, both produce a rollback that restores something other than what was there.

20.4 Idempotency and delivery semantics

The platform provides **at-least-once** delivery with idempotent consumers, not exactly-once. This is a deliberate choice and should be defended as one rather than apologised for. Exactly-once across a bus, a relational database and an external Kubernetes API would require a distributed transaction spanning a system the platform does not control; the Kubernetes API server has no notion of the platform's transaction and cannot be asked to join one.

Idempotency is therefore achieved per consumer, by a different mechanism in each case, chosen to suit what that consumer writes. The cost engine's writes are keyed on (`resource_uid`, `window_start`) with an upsert, so recomputing a window replaces rather than duplicates. The behaviour engine guards its baseline recursion with the last processed offset per partition, checkpointed alongside the state it protects, so a replayed sample is recognised and skipped. The executor carries the `idempotency_key` shown above, a unique index enforces it in `actions_log`, and every action is expressed as a declarative patch to a desired state — so even if the guard were bypassed, applying `maxReplicas: 4` twice leaves the autoscaler in the same place as applying it once.

IDEMPOTENT EFFECTS BEAT EXACTLY-ONCE DELIVERY

The general rule the design follows: make the effect idempotent, then the delivery guarantee stops mattering. Every action in the ladder of chapter 25 is expressed as a desired state rather than an operation — freeze to this ceiling, apply this network policy, set this quota — with exactly one exception. Terminate is not idempotent, and that is one more reason it sits at the top of the ladder and requires the most evidence to reach.

SCHEMA, INDEXES, RETENTION

21. The Data Model in Full

Nine tables, split between TimescaleDB hypertables for anything written at high frequency and read as a time range, and ordinary PostgreSQL tables for everything read as a point lookup or joined. Both live in the same database instance, which removes an operational component and lets a single query join a cost range against service ownership.

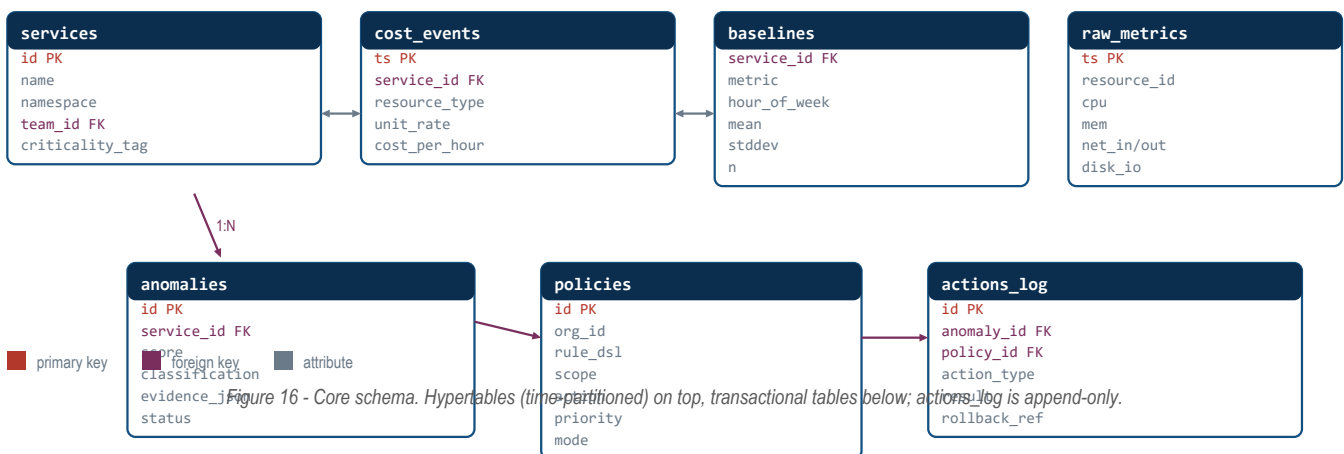


Figure 16 - Core schema. Hypertables (time-partitioned) on top, transactional tables below; `actions_log` is append-only.

21.1 Dimension tables

schema/01_dimensions.sql

```

CREATE TABLE teams (
  id          BIGSERIAL PRIMARY KEY,
  org_id      BIGINT      NOT NULL,
  name        TEXT        NOT NULL,
  slack_channel TEXT,
  UNIQUE (org_id, name)
);

CREATE TYPE criticality AS ENUM ('critical', 'production', 'staging', 'dev');

CREATE TABLE services (
  id          TEXT          PRIMARY KEY,          -- e.g. 'svc_checkout'
  org_id      BIGINT        NOT NULL,
  team_id     BIGINT        REFERENCES teams(id),
  namespace   TEXT          NOT NULL,
  workload    TEXT          NOT NULL,
  tag         criticality   NOT NULL DEFAULT 'dev',
  timezone    TEXT          NOT NULL DEFAULT 'UTC',
  requests_cpu NUMERIC(9,4),          -- for shared amortisation
  onboarded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (org_id, namespace, workload)
);

CREATE TABLE users (
  id          BIGSERIAL PRIMARY KEY,
  org_id      BIGINT NOT NULL,
  email       TEXT    NOT NULL UNIQUE,
  role        TEXT    NOT NULL CHECK (role IN ('admin','sre','finance')),
  pwd_hash    TEXT    NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Two columns on `services` are load-bearing beyond their appearance. `timezone` exists because bucketing by hour of week in UTC smears a business-hours pattern across adjacent buckets for any service whose users are not on UTC, which roughly halves detector sensitivity. And `requests_cpu` is the denominator of the shared-cost amortisation in chapter 22; it is stored on the dimension row rather than read from Kubernetes on the hot path so that the cost engine performs no API calls per sample.

21.2 Hypertables

```
schema/02_hypertables.sql
```



```

CREATE TABLE raw_metrics (
  ts          TIMESTAMPTZ NOT NULL,
  resource_uid UUID       NOT NULL,
  service_id  TEXT        NOT NULL REFERENCES services(id),
  cpu_cores   REAL        NOT NULL,
  mem_gib     REAL        NOT NULL,
  net_in_bps  REAL        NOT NULL,
  net_out_bps REAL        NOT NULL,
  disk_io_bps REAL        NOT NULL,
  http_requests INTEGER    NOT NULL DEFAULT 0
);
SELECT create_hypertable('raw_metrics', 'ts', chunk_time_interval => INTERVAL '6 hours');
CREATE INDEX ON raw_metrics (service_id, ts DESC);
ALTER TABLE raw_metrics SET (timescaledb.compress,
                             timescaledb.compress_segmentby = 'service_id');
SELECT add_compression_policy('raw_metrics', INTERVAL '24 hours');
SELECT add_retention_policy('raw_metrics', INTERVAL '7 days');

CREATE TABLE cost_events (
  ts          TIMESTAMPTZ NOT NULL,
  window_start TIMESTAMPTZ NOT NULL,
  resource_uid UUID       NOT NULL,
  service_id  TEXT        NOT NULL REFERENCES services(id),
  price_set   TEXT        NOT NULL,
  cpu_usd_hr  NUMERIC(12,6) NOT NULL,
  mem_usd_hr  NUMERIC(12,6) NOT NULL,
  egress_usd_hr NUMERIC(12,6) NOT NULL,
  storage_usd_hr NUMERIC(12,6) NOT NULL,
  shared_usd_hr NUMERIC(12,6) NOT NULL,
  total_usd_hr NUMERIC(12,6) NOT NULL,
  requests    INTEGER      NOT NULL DEFAULT 0,
  is_estimate  BOOLEAN      NOT NULL DEFAULT TRUE,
  PRIMARY KEY (resource_uid, window_start) -- idempotent upsert key
);
SELECT create_hypertable('cost_events', 'ts', chunk_time_interval => INTERVAL '1 day');
CREATE INDEX ON cost_events (service_id, ts DESC);
SELECT add_retention_policy('cost_events', INTERVAL '400 days');

```

The composite primary key on `cost_events` is what makes the cost engine idempotent under at-least-once delivery, as described in chapter 20: a reprocessed window collides on the key and upserts rather than double-counting. Choosing a one-day chunk interval for cost events against six hours for raw metrics reflects their different write rates; chunks should be sized so that the indexes of the most recently written chunk stay resident in memory.

21.3 Continuous aggregate for the dashboard

The executive dashboard asks for total spend per service per hour over ranges up to ninety days. Computing that from raw cost events on every page load would be wasteful and slow; a continuous aggregate maintains it incrementally instead.

```
schema/03_aggregates.sql
```

```
CREATE MATERIALIZED VIEW cost_hourly
WITH (timescaledb.continuous) AS
SELECT time_bucket(INTERVAL '1 hour', ts) AS hour,
       service_id,
       avg(total_usd_hr)           AS avg_usd_hr,
       max(total_usd_hr)           AS peak_usd_hr,
       sum(total_usd_hr) / 360.0    AS usd_spent,    -- 10s windows in an hour
       sum(requests)               AS requests
FROM cost_events
GROUP BY hour, service_id
WITH NO DATA;

SELECT add_continuous_aggregate_policy('cost_hourly',
    start_offset => INTERVAL '3 hours',
    end_offset   => INTERVAL '10 minutes',
    schedule_interval => INTERVAL '5 minutes');
```

The `end_offset` of ten minutes is deliberate. It keeps the aggregate from refreshing over a window that is still receiving late-arriving events, which would otherwise produce a total that decreases as more data arrives — a phenomenon that destroys trust in a financial dashboard faster than almost any other bug. Anything more recent than the offset is served live from the hypertable.

21.4 State, decision and audit tables

```
schema/04_state.sql
```

```

CREATE TABLE baselines (
  service_id TEXT NOT NULL REFERENCES services(id),
  metric TEXT NOT NULL,
  bucket SMALLINT NOT NULL CHECK (bucket BETWEEN 0 AND 167),
  mean DOUBLE PRECISION NOT NULL,
  variance DOUBLE PRECISION NOT NULL,
  n_samples BIGINT NOT NULL DEFAULT 0,
  weeks_seen SMALLINT NOT NULL DEFAULT 0,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (service_id, metric, bucket)
);

CREATE TABLE anomalies (
  id BIGSERIAL PRIMARY KEY,
  detected_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  service_id TEXT NOT NULL REFERENCES services(id),
  bucket SMALLINT NOT NULL,
  classification TEXT NOT NULL,
  confidence REAL NOT NULL CHECK (confidence BETWEEN 0 AND 1),
  cost_delta_usd_hr NUMERIC(12,6) NOT NULL,
  evidence JSONB NOT NULL,
  status TEXT NOT NULL DEFAULT 'open'
  CHECK (status IN ('open','acted','acknowledged',
                    'dismissed','expired')),
  baseline_mature BOOLEAN NOT NULL
);
CREATE INDEX ON anomalies (service_id, detected_at DESC);
CREATE INDEX ON anomalies (status) WHERE status = 'open';

CREATE TABLE policies (
  id BIGSERIAL PRIMARY KEY,
  org_id BIGINT NOT NULL,
  name TEXT NOT NULL,
  priority INTEGER NOT NULL,
  spec JSONB NOT NULL, -- the YAML policy, parsed
  enabled BOOLEAN NOT NULL DEFAULT TRUE,
  dry_run BOOLEAN NOT NULL DEFAULT TRUE,
  created_by BIGINT REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (org_id, name)
);

CREATE TABLE policy_revisions (
  id BIGSERIAL PRIMARY KEY,
  policy_id BIGINT NOT NULL REFERENCES policies(id),
  spec JSONB NOT NULL,
  changed_by BIGINT REFERENCES users(id),
  changed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

The audit table is the one place in the schema where correctness is enforced by the database rather than trusted to the application, because NFR-5 requires that every action be reconstructable and an application bug must not be able to violate that.

```
schema/05_audit.sql
```

```

CREATE TABLE actions_log (
  id          BIGSERIAL PRIMARY KEY,
  anomaly_id  BIGINT REFERENCES anomalies(id),
  policy_id   BIGINT REFERENCES policies(id),
  idempotency_key TEXT NOT NULL UNIQUE,
  action_type TEXT NOT NULL CHECK (action_type IN
    ('alert_only','throttle','freeze_scaling',
    'quarantine','terminate','rollback')),
  mode        TEXT NOT NULL CHECK (mode IN ('autonomous','approved','dry_run')),
  target      JSONB NOT NULL,
  prior_state JSONB,
  applied_state JSONB,
  evidence     JSONB NOT NULL,
  initiated_by BIGINT REFERENCES users(id), -- NULL when autonomous
  executed_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  result       TEXT NOT NULL,
  rollback_ref BIGINT REFERENCES actions_log(id),
  rollback_deadline TIMESTAMPTZ
);
CREATE INDEX ON actions_log (executed_at DESC);

-- Append-only, enforced by the database and not by convention.
CREATE OR REPLACE FUNCTION deny_mutation() RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'actions_log is append-only (attempted %)', TG_OP;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER actions_log_no_update BEFORE UPDATE ON actions_log
  FOR EACH ROW EXECUTE FUNCTION deny_mutation();
CREATE TRIGGER actions_log_no_delete BEFORE DELETE ON actions_log
  FOR EACH ROW EXECUTE FUNCTION deny_mutation();

REVOKE UPDATE, DELETE ON actions_log FROM hypertrace_app;

```

Note that a rollback is a new row referencing the original through `rollback_ref`, never an update to it. The audit log is therefore a ledger, and the current state of any action is derived by reading forward rather than stored in place. Both the trigger and the `REVOKE` are present because they fail differently: the grant stops the application's normal path, and the trigger stops anything that arrives with wider privileges, including a well-intentioned migration.

21.5 The cardinality problem and the chosen compromise

There is a strong temptation to label every metric with the pod name, so that the interface can show exactly which pod misbehaved. Resist it. In a cluster that deploys twice a day, pod names are regenerated on every rollout, so a label carrying pod name manufactures a new time series with every deploy. Series count then grows linearly with deployment frequency and never falls, and the failure is not an error message but a slow, permanent degradation: chunk indexes stop fitting in memory, ingest latency climbs, and the hot path starts missing its budget for reasons that look like nothing in particular.

The compromise this design adopts has three parts.

1. **Aggregate on the write path, keep detail on the read path.** The series that the detector consumes is keyed by `service_id` only — a stable identifier that survives rollouts. Pod-level detail is written to `raw_metrics` as an ordinary column, `resource_uid`, which is indexed but not part of any series key. Detail is therefore queryable without being cardinal.
2. **Bound the dimensions that are allowed to be labels.** Only `service_id`, `namespace` and `node` may appear as series-defining labels, and all three have a natural ceiling set by the size of the cluster rather than by how often it is deployed.
3. **Measure the ceiling and alert on approach.** Active series count is one of the platform's own health metrics in chapter 28, with a warning at 80 percent of the 50,000-series constraint from chapter 17. A limit that is not monitored is not a limit.

The cost of this compromise is honest and should be stated: a single misbehaving pod inside a service with twenty healthy replicas produces a diluted signal, because the detector sees the service average. The mitigation is that the evidence payload attached to any anomaly includes the top contributing `resource_uid` values from the raw table, so the interface can still point at the guilty pod even though the detector did not track it as a series. Detection is aggregate; attribution is detailed.

FROM CPU-SECONDS TO DOLLARS, WITH THE ARITHMETIC SHOWN

22. The Cost Model, Precisely

This is the chapter that turns telemetry into money, and it is the chapter most worth getting arithmetically right, because every downstream conclusion inherits its errors. It is presented as four steps — direct cost, shared cost, unit cost, reconciliation — followed by a fully worked numeric example that can be checked by hand.

22.1 Direct cost of a workload

For an interval of length τ hours, a workload's direct cost is the sum of four terms, each the product of a measured quantity and a published rate. Measured quantities are means over the interval for the rate-like dimensions and totals for the volume-like ones, which is the distinction that most often gets confused in implementation.

`cost/direct.py` - the four direct terms

```
direct_usd_per_hour =
    mean_cpu_cores          * rate_usd_per_vcpu_hour
+ mean_mem_gib             * rate_usd_per_gib_hour
+ egress_gb_per_hour       * rate_usd_per_gb_egress
+ provisioned_gib          * (rate_usd_per_gib_month / 730.0)

# CPU and memory are AVERAGES over the window: a pod that used one core for
# half the window and none for the other half costs the same as one that used
# half a core throughout, which is correct - billing is for core-seconds.
#
# Egress is a TOTAL over the window, converted to an hourly rate. Storage is
# PROVISIONED, not used: an empty 20 GiB volume costs exactly what a full one
# costs, which is a distinction FinOps reports get wrong surprisingly often.
```

22.2 Shared and unallocated cluster cost

A cluster costs more than the sum of its pods. The control plane has a fixed hourly charge on managed Kubernetes. Nodes are billed whole even when partly idle, so unused headroom is real money attributable to no workload. Load balancers, NAT gateways and the platform's own components add further cluster-level cost. If this pool is ignored, the sum of every service's reported cost falls materially short of the bill, and the first person to notice will stop trusting the dashboard.

Three allocation strategies are available. The design picks the third.

Strategy	Rule	Behaviour and why it was or was not chosen
Even split	Divide the shared pool equally across all workloads	Trivial to implement and indefensible in practice: a single-replica cron job carries the same node overhead as a twenty-replica API. Rejected.
Proportional to direct cost	Each workload's share equals its direct cost over the cluster's total direct cost	Better, but it lets an egress-heavy workload absorb node overhead it does not cause. Egress does not make the cluster buy nodes. Rejected, though it is the intuitive first choice.
Proportional to requested compute (chosen)	Each workload's share equals its requested CPU, plus requested memory converted at the node's own CPU-to-memory price ratio, over the cluster total of the same quantity	Node count is determined by the scheduler packing <i>requests</i> , not usage and not egress. Allocating shared cost by requests therefore attributes it to the thing that causes it, and it creates the right incentive: a team that over-requests pays for the headroom it reserves.

The chosen strategy has one property worth calling out to a panel, because it is the argument for it. It makes over-requesting visible as cost. A team that requests four cores and uses one will see a shared-cost component roughly four

times that of a right-sized neighbour with identical usage, which is exactly the signal a FinOps process wants to surface and exactly what a usage-proportional strategy would hide. Kubecost-style tools commonly offer both a usage basis and a request basis for this reason; this design picks the request basis and says so.

22.3 Cost per unit of work

Unit cost is the quantity that makes the platform's judgements legible to a business audience, for the reason given in chapter 3: a rising total is ambiguous and a rising cost-per-checkout is not. It is reported per thousand requests rather than per request, because per-request figures land in the fifth decimal place and are unreadable on a dashboard.

```
cost/unit.py - unit cost, with the zero-denominator case handled

requests_per_hour = requests_in_window * (3600.0 / window_seconds)

if requests_per_hour >= MIN_REQUESTS_FOR_UNIT_COST:      # default 100
    usd_per_1k_requests = total_usd_per_hour / (requests_per_hour / 1000.0)
else:
    # Denominator too small: the ratio is dominated by noise and would show
    # an idle service as infinitely expensive. Report the absolute cost and
    # an explicit null, never a large number.
    usd_per_1k_requests = None                          # UI renders 'n/a'
```

The zero-request case deserves the explicit branch it gets. An idle service has a real, non-zero cost and no work to divide it by, so the mathematically honest answer is undefined rather than large. Returning a large number here would make idle services top the “most expensive per request” list, which is both technically true and completely useless. The interface renders the null as “n/a, idle” and the detector never computes a Z-score against a null.

22.4 A fully worked example

The example uses the pod from the `metrics.raw` sample in chapter 20, so the arithmetic below reproduces the `cost.events` message shown there. Rates are the simulated us-east-1 on-demand figures used throughout Part III. Every line can be checked with a calculator.

Workload	namespace <code>shop</code> , deployment <code>checkout-api</code> , one pod on <code>kind-worker2</code>
Window	10 seconds, 2026-08-26 10:30:54Z to 10:31:04Z
Measured CPU	0.8471 cores, mean over the window
Measured memory	1.502 GiB working set, mean over the window
Measured egress	33,333,333 bytes in 10 s, which is 12.00 GB per hour
Provisioned storage	20 GiB gp3 volume, from the PVC inventory, not from metrics
Requests served	118 in the window, which is 42,480 per hour
Requested CPU	1.000 core; cluster total requested 5.400 cores
Shared pool	\$0.42/hour: control plane \$0.10, idle node headroom \$0.27, one load balancer \$0.05

Component	Quantity	Rate	Arithmetic	USD/hour
CPU	0.8471 vCPU	\$0.029500 per vCPU-hour	0.8471×0.0295	0.024990
Memory	1.502 GiB	\$0.003950 per GiB-hour	1.502×0.00395	0.005933
Egress	12.00 GB/hour	\$0.090000 per GB	12.00×0.09	1.080000
Storage	20 GiB provisioned	\$0.080000 per GiB-month	$20 \times (0.08 / 730)$	0.002192
Direct subtotal			sum of the four above	1.113115
Shared	1.000 of 5.400 requested cores	\$0.42/hour pool	$0.42 \times (1.000 / 5.400)$	0.077778
Total			$1.113115 + 0.077778$	1.190893
Unit cost	42,480 requests/hour		$1.190893 / 42.480$	0.028035 per 1k

WHAT THE WORKED EXAMPLE REVEALS

Read the egress line again: it is 91 percent of this pod's cost. That is not an artefact of contrived numbers, it is the normal shape of cloud spend for anything that serves data, and it is the reason the shared-cost strategy in section 22.2 must not be proportional to direct cost. Had it been, this one pod would have absorbed most of the cluster's node overhead on the strength of its network traffic, which no scheduler decision supports.

22.5 Reconciliation against the authoritative bill

The estimate is fast and approximate; the provider's billing export is slow and authoritative. Principle five of chapter 17 requires that the two be comparable, so the Billing Adapter runs a daily job that sums the estimate over a settled window and compares it with the billed figure for the same window.

```
billing/reconcile.py - drift metric and its thresholds

drift = (estimated_usd - billed_usd) / billed_usd

# |drift| <= 0.05 -> healthy. Expected: the estimate cannot see
#                  cross-AZ traffic, snapshot storage, or per-request
#                  charges on managed services.
# 0.05 < |drift| <= 0.15 -> warn. Surface a banner on the cost dashboard
#                  saying estimates are running high or low by N%.
# |drift| > 0.15 -> alert on the COST MODEL, not on the workload.
#                  Autonomous action is suspended for cost-derived
#                  classifications until drift returns under 15%.

# Reconcile over a settled window only. Billing data for the last 48 hours
# is still moving, so comparing against it manufactures phantom drift.
WINDOW_END   = today_utc() - timedelta(days=2)
WINDOW_START = WINDOW_END - timedelta(days=1)
```

The third threshold is the important one, and it is an application of principle three: degrade authority before accuracy. If the cost model is demonstrably wrong by more than fifteen percent, the platform continues to detect and to alert, but it stops taking autonomous action on classifications that depend on cost. It has lost the standing to act, not the ability to observe. Stating this rule explicitly is also the strongest available answer to a panel asking what happens when the estimate is wrong: the system notices, says so, and de-escalates itself.

WHAT NORMAL MEANS, AND HOW IT IS REMEMBERED

23. Baselines and State Management

The baseline is the system's model of normal, and it is the only substantial mutable state in the platform. This chapter specifies its shape, its update rule, when it is allowed to be trusted, what happens to it during gaps and step changes, and how it survives a restart.

23.1 The 168-bucket scheme

168 independent baselines, one per hour-of-week bucket

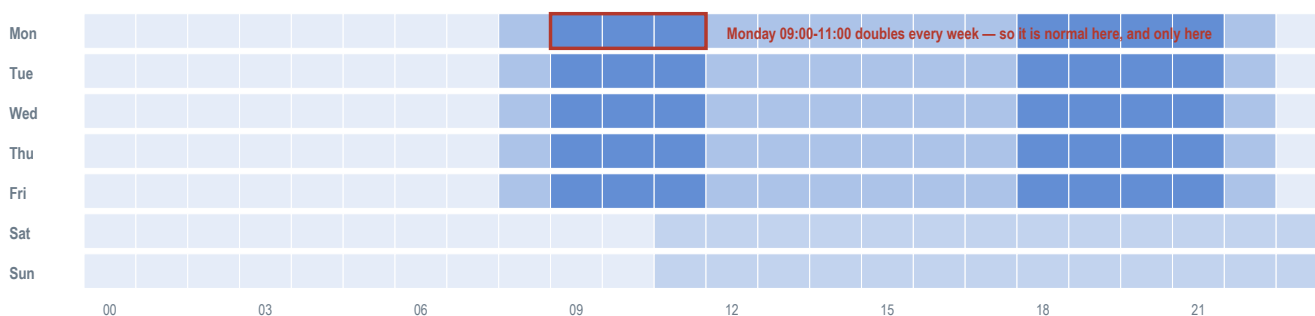


Figure 11 - Why bucketing matters. A global mean would flag every Monday morning; a per-hour-of-week mean learns that the spike belongs there.

Each service keeps one baseline per metric per hour-of-week bucket: 168 buckets, indexed zero for Monday 00:00 through 167 for Sunday 23:00, in the service's own timezone. A sample taken at 10:31 on a Tuesday updates bucket 34 and nothing else. Weekly rather than daily seasonality is the right granularity for this domain because business load is a weekly phenomenon: Monday morning does not resemble Saturday morning, and no daily profile can express that.

Sizing matters for the latency budget of chapter 19, so it is worth working out. At a ten-second collection interval, each bucket receives 360 samples per week. Six tracked metrics $\times$ 168 buckets $\times$ four numbers — mean, variance, count, weeks-seen — is 4,032 numbers per service, roughly 32 KiB. Ten thousand services therefore fit in a few hundred megabytes of process memory, which is what allows scoring to be a pure in-memory operation with no database round trip on the hot path.

23.2 The update rule

The rule is an exponentially weighted mean paired with an exponentially weighted variance, rather than Welford's online algorithm. Welford is the better choice when every observation should count equally forever; that is precisely the wrong property here, because a baseline must forget. A service refactored in March should not be judged against February's behaviour indefinitely. The exponentially weighted form gets forgetting for free from a single parameter and holds the same $O(1)$ state.

behaviour/baseline.py - the update rule

```
ALPHA_DEFAULT = 1.0 / 720.0    # effective memory ~= 720 samples == 2 weeks
MIN_SD_FLOOR  = 1e-6          # never divide by a variance of exactly zero

def update(b, x, alpha=ALPHA_DEFAULT):
    """Exponentially weighted mean and variance. Mutates one bucket.

    b.mean, b.variance, b.n_samples are the only state. The recursion is
    the standard EWMV form: update the mean, then decay the variance toward
    the squared deviation from the OLD mean.
    """
    if b.n_samples == 0:        # first ever sample for this bucket
        b.mean, b.variance, b.n_samples = x, 0.0, 1
        return

    delta = x - b.mean
    b.mean = b.mean + alpha * delta
    b.variance = (1.0 - alpha) * (b.variance + alpha * delta * delta)
    b.n_samples += 1

def sd(b):
    return max(b.variance ** 0.5, MIN_SD_FLOOR)
```

The identity worth remembering is that an alpha of $1/N$ gives an effective memory of roughly N samples. Choosing $1/720$ therefore encodes “remember about two weeks of this bucket” in one constant, which is the same window the pseudocode in chapter 14.2 specified and a far cheaper way to implement it than retaining a fourteen-day ring buffer per bucket. The variance floor exists to prevent division by zero for a metric that has been perfectly constant, which is common for idle services and would otherwise produce infinite Z-scores.

23.3 Warm-up and the maturity threshold

A bucket is not trusted until it has seen at least 120 samples spanning at least two distinct weeks. Both conditions are needed: 120 samples inside a single hour prove only that the service was busy once, while two weeks with a handful of samples prove only that it existed. Until a bucket matures, scoring falls back through a defined chain.

Level	Used when	Consequence for authority
Bucket baseline	The bucket has ≥ 120 samples across ≥ 2 weeks	Full detection. Autonomous action permitted where policy allows.

Level	Used when	Consequence for authority
Pooled service baseline	The bucket is immature but the service has $\geq 2,000$ samples pooled across all 168 buckets	Detection with seasonality blind. Anomaly is emitted with <code>baseline_mature: false</code> and autonomous action is refused; the policy engine downgrades to recommend-with-approval.
Global static threshold	The service is newly onboarded and has neither	Coarse threshold detection only, per chapter 11.2. Alert-only; no action of any kind, and the interface shows a “learning” badge.

23.4 Gaps, missing data and aging

Missing samples are skipped, never zero-filled. This is a one-line implementation detail with a large behavioural consequence: zero-filling a gap teaches the baseline that idle is normal, which lowers the mean and shrinks the variance, which in turn suppresses detection for exactly those services that were recently down or recently deployed. A service returning from a maintenance window would look anomalous simply for resuming work.

- **Short gaps**, under five minutes, are ignored entirely. Nothing is written and nothing is inferred.
- **Long gaps**, over one hour, mark the affected buckets stale by setting `updated_at` without touching mean or variance. A stale bucket still scores, but the anomaly it produces carries a staleness note in its evidence so that a human reading it later knows the baseline predates the gap.
- **Aging** happens automatically through the exponential weighting: after roughly two effective windows, old behaviour contributes negligibly. A hard time-to-live of ninety days additionally drops buckets that have not been touched at all, which keeps the state size bounded for services with genuinely intermittent load.

23.5 Legitimate step changes

The hardest case for any baseline is a permanent, legitimate change in level: a service that grows twenty percent week over week, or a deployment that makes a service structurally more expensive for a good reason. Three mechanisms handle it, in increasing order of directness.

1. **Absorption.** The exponentially weighted mean drifts toward the new level, and with an effective memory of two weeks a sustained step is largely absorbed within that period. Gradual growth of a few percent per week never produces a flag at all, because the mean tracks it.
2. **Hysteresis and level-shift detection.** A deviation that persists beyond a configured dwell time — thirty minutes by default — is reclassified from an anomaly into a single `level_shift` event, and a per-service cooldown suppresses repeats. This is what stops one permanent change from generating an anomaly every ten seconds for a fortnight, which is the failure mode that makes operators mute alerts.
3. **Operator rebase.** An SRE can accept an anomaly as the new normal from the incident feed. This sets the bucket's mean to the current observation and inflates its variance to the observed deviation, so the baseline moves immediately rather than over two weeks. The action is recorded in the audit log with the user who took it, because rebasing a baseline is a decision about what normal means and should be attributable.

23.6 Where the state lives and how it survives restart

Baseline state is authoritative in the Behaviour Analysis Service's process memory and durable in the `baselines` table. The service is a `StatefulSet` whose replicas map one-to-one onto bus partitions, so a given service's baseline is owned by exactly one process and never contended.

```
behaviour/checkpoint.py - state and offset in one transaction

# Checkpoint every 60s and on SIGTERM. Two rows per (service, metric, bucket)
# are never needed - the upsert makes the checkpoint idempotent.

async def checkpoint(store, pool, partition, offset):
    async with pool.acquire() as con:
        async with con.transaction():
            await con.executemany(
                """INSERT INTO baselines (service_id, metric, bucket, mean,
                    variance, n_samples, weeks_seen, updated_at)
                VALUES ($1,$2,$3,$4,$5,$6,$7, now())
                ON CONFLICT (service_id, metric, bucket) DO UPDATE SET
                    mean=$4, variance=$5, n_samples=$6, weeks_seen=$7,
                    updated_at=now()""",
                store.dirty_rows())
            # Offset is committed in the SAME transaction as the state it
            # describes. This is what makes recovery exact rather than close.
            await con.execute(
                """INSERT INTO consumer_offsets (partition, offset_val)
                VALUES ($1,$2) ON CONFLICT (partition)
                DO UPDATE SET offset_val=$2""", partition, offset)
            store.clear_dirty()
```

Recovery follows from that single transaction. On start, the service loads the checkpointed buckets, reads the offset committed with them, and resumes consuming from exactly there. Because the offset and the state were written atomically, replayed samples are the ones whose effects were never persisted, and the update rule is applied to them exactly once. The worst case is therefore up to sixty seconds of recomputation rather than the loss of weeks of learning — which is the failure mode the naive design, checkpointing state and offsets separately, produces the first time a pod is evicted mid-write.

THRESHOLDS, CONFIDENCE, AND THE TRADE BEING MADE

24. Detection and Classification

Chapter 14 gave the detector as pseudocode. This chapter specifies it: which metrics are scored, how the scores combine, why the thresholds are the numbers they are, how confidence is defined, and what evidence each classification requires before it may be assigned.

24.1 What is scored

Metric	Role in the decision	Scored?
CPU cores	Primary resource signal; the thing that moves first in almost every scenario in chapter 2	Yes
Memory working set	Secondary resource signal; distinguishes a leak from a compute loop	Yes
Egress bytes per second	Both a cost driver and a security signal; the cryptomining heuristic depends on it	Yes
Total \$/hour	The financial signal, and the one no competitor scores in real time	Yes
HTTP requests per second	The <i>business</i> signal, and the denominator of the decoupling test. It is scored so that its Z-score can be compared, never so that a high value alone triggers anything	Yes
Disk I/O	Retained as evidence and for the interface; too noisy in containerised workloads to threshold usefully	No

24.2 Thresholds, and why these numbers

The decision condition is the conjunction from chapter 14.3, with the constants made explicit and justified. A threshold without a justification is a magic number, and magic numbers are what a panel asks about.

Constant	Value	Justification
Resource Z threshold	3.0	For an approximately normal baseline, three standard deviations is a one-in-370 observation. At one sample per ten seconds and 360 samples per bucket per week, that is roughly one false crossing per bucket per week per metric from chance alone — tolerable only because the conjunction requires several conditions at once. Any single-metric detector at 3.0 would be unusably noisy.
Traffic Z ceiling	1.0	The decoupling test needs traffic to be <i>unremarkable</i> , not low. One standard deviation is the boundary of ordinary variation, so a service serving somewhat more than usual still qualifies as decoupled if its cost has moved by three.
Cost Z threshold	3.0	Held equal to the resource threshold deliberately. Cost is derived from resources, so a materially lower cost threshold would double-count the same underlying movement and make the conjunction weaker than it appears.
Traffic Z co-movement floor	3.0	The value above which traffic is treated as explaining the resource movement, producing the explicit <code>legitimate_traffic_growth</code> classification and no action.
Dwell time before acting	30 s, three consecutive qualifying samples	A single sample can be an artefact of a scrape landing mid-garbage-collection. Three consecutive qualifying samples cost 30 seconds of the latency budget and eliminate essentially all single-sample noise.
Level-shift dwell	30 minutes	Beyond this, a persistent deviation is treated as a new normal to be reported once rather than an incident to be re-reported, per chapter 23.5.

24.3 The confidence score

Confidence is what gates authority, so it must be a defined function rather than a feeling. It combines four terms: how far past the threshold the signal is, how cleanly it decoupled from traffic, how much corroborating evidence exists, and how mature the baseline it was judged against is.

`behaviour/confidence.py` - the confidence function and its gates

```
def confidence(scores, corroboration, n_samples):
    """Return 0.0-1.0. Every term is bounded, so no single term can
    carry the score on its own."""
    peak = max(scores.cpu_z, scores.cost_z)

    magnitude = min(1.0, (peak - 3.0) / 5.0)          # 3 sigma -> 0, 8 -> 1
    decoupling = min(1.0, (peak - scores.traffic_z) / 6.0)
    corrob = corroboration                            # 0.0, 0.5 or 1.0
    maturity = min(1.0, n_samples / 720.0)

    return round(0.40 * magnitude
                + 0.30 * decoupling
                + 0.20 * corrob
                + 0.10 * maturity, 3)

# Authority is a function of confidence, and only of confidence.
# >= 0.85 autonomous action permitted, if policy also allows it
# >= 0.60 recommend and require one-click human approval
# < 0.60 alert only; never proposed as an action
```

Two properties of this function are deliberate. Every term is capped at one, so no single enormous Z-score can push confidence to autonomous levels on its own — a CPU spike of forty standard deviations with no decoupling and no corroboration reaches 0.40 and stays advisory. And baseline maturity is a term rather than a gate, so an immature baseline lowers confidence smoothly instead of switching detection off, which keeps behaviour continuous as a service warms up.

24.4 Classification and the evidence each one requires

A classification is a claim about cause, and the design requires each claim to be supported by specific evidence rather than by the absence of an alternative. The residual classification is named as such.

Classification	Required evidence	Typical action tier
<code>legitimate_traffic_growth</code>	Resource or cost Z above 3.0 <i>and</i> traffic Z above 3.0. Nothing further is required, because this classification authorises nothing.	None. Recorded for the trend view only.
<code>suspected_abuse</code>	The decoupling condition, <i>plus</i> at least one of: egress to a destination outside the service's historical geography set; a process signature matching a known miner; or a Falco event on the same pod within the window. Cost evidence alone is never sufficient for this label.	Quarantine, and terminate at the highest confidence with policy permission.
<code>likely_bug_from_deployment</code>	The decoupling condition, <i>plus</i> a deployment marker for the same workload within fifteen minutes, taken from <code>events.k8s</code> rather than inferred from a pod restart.	Freeze scaling, then throttle. Highly reversible, which suits the most likely false-positive case.
<code>misconfiguration_or_waste</code>	The decoupling condition, with no security corroboration and no recent deployment. This is the residual class — the label used when the system knows something is unexplained but cannot say why.	Alert, or throttle for non-critical tags only.
<code>level_shift</code>	A deviation satisfying the condition continuously for the 30-minute dwell time. Reported once per cooldown period.	Alert only, with an offer to rebase the baseline.

24.5 The trade being made, stated plainly

Every detector chooses a point on a curve between missing real incidents and flagging harmless ones, and pretending otherwise is the fastest way to lose a technical argument. This design is deliberately biased toward false negatives, for a reason specific to the domain: the cost of a missed incident is money, bounded by the rate limits and the eventual arrival of the bill, whereas the cost of a false positive is an automatic action taken against a healthy production service. Those are not symmetric, so the detector should not be tuned as though they were.

Concretely, the bias appears in four places: the conjunction requires three conditions where any one would produce more detections; the dwell time discards single-sample events; the confidence function caps every term so that an extreme single signal cannot authorise action; and the maturity term withholds authority from young baselines. Each of these lowers recall in exchange for precision. The measurement harness in Part III reports both numbers so the trade can be evaluated rather than asserted.

EXPLAINABILITY IS A FUNCTIONAL REQUIREMENT, NOT A PREFERENCE

This is also the answer to “why not a neural network?”. A model that flags anomalies more accurately but cannot state which of these four conditions it relied on cannot be granted the authority to act, because the audit requirement of principle two would be unsatisfiable. An unexplainable detector can inform a human; only an explainable one can replace them.

HOW A DECISION BECOMES A CHANGE TO PRODUCTION

25. Policy, Actions and Reversibility

Detection produces a claim. Policy decides whether that claim earns a change to a running system, and if so which one. Keeping those two concerns in separate services is the design's most consequential structural choice, because it means the detector can be improved without renegotiating what the platform is allowed to do, and the rules of engagement can be tightened without redeploying the detector.

25.1 Policy as data

Principle four of chapter 17 requires that policy be data rather than code. Concretely: rules are YAML documents, stored in the `policies` table, versioned in `policy_revisions`, editable by an administrator through the interface, and evaluated by an interpreter that contains no service-specific logic. Nobody has to ship a release to stop the platform from touching the payments service, and every change to what the platform may do has an author and a timestamp.

```

policies/checkout-api.yaml

# policies/checkout-api.yaml
name: checkout-api-guardrails
priority: 100                # lower number wins; see 25.3
enabled: true
dry_run: false

match:
  namespace: shop
  workload: checkout-api
  tag: critical

conditions:
  classification_in: [suspected_abuse, likely_bug_from_deployment]
  min_confidence: 0.85
  min_cost_delta_usd_hr: 0.50
  business_hours:           # in the service's own timezone
    days: [mon, tue, wed, thu, fri]
    from: "09:00"
    to:   "18:00"

actions:
  # Inside business hours a human decides. Outside them, act and report.
  - when: business_hours
    do: recommend                # queue for one-click approval
    notify: [slack:#sre-oncall, email:sre@example.com]
  - when: outside_business_hours
    do: throttle
    then_if_persists: freeze_scaling
    persist_for: 300s
    notify: [slack:#sre-oncall, pagerduty:P1]

limits:
  max_actions_per_hour: 3
  min_replicas: 2             # never scale below quorum
  never:
    - terminate               # this workload is never terminated
  auto_rollback_after: 900s   # 15 minutes, unless renewed

```

Read the **actions** block carefully, because it encodes a judgement that is easy to state and easy to get backwards. Inside business hours, when an engineer is available, the platform asks. Outside them, when the alternative is eight unattended hours of accruing cost, it acts and reports. Autonomy is therefore a function of human availability rather than a fixed setting, which is how a competent on-call rotation would want it configured.

25.2 The action ladder

LEAST AGGRESSIVE

Alert only	notify, change nothing	none	always automatic
Throttle	cgroup / ResourceQuota cap	low, reversible instantly	auto if non-critical
Freeze scaling	pin HPA maxReplicas	low, stops the bleeding	auto if non-critical
Quarantine	deny-all NetworkPolicy	medium, breaks connectivity	auto only on high-confidence abuse
Terminate	delete pod / stop instance	high, irreversible	human approval unless confirmed abuse

MOST AGGRESSIVE

autonomy shrinks as blast radius grows

Figure 9 - The escalation ladder. Full autonomy is granted in inverse proportion to how hard the action is to undo.

Actions are ordered by how much they cost to be wrong about, and the policy engine always selects the least aggressive rung that satisfies the condition. The ladder is not a menu of equivalent options; it is a ranking, and the ranking is by reversibility rather than by effectiveness.

Rung	Mechanism	Reversal	Blast radius if the detection was wrong
1. Alert only	Write the anomaly, notify the configured channels, change nothing	Nothing to reverse	An engineer reads an unnecessary message. This is the only rung with no production risk at all.
2. Throttle	Patch a <code>ResourceQuota</code> or <code>LimitRange</code> , or tighten the workload's cgroup CPU quota	Restore the recorded quota; instant and exact	The workload runs slower against a lower ceiling. Latency rises and some requests queue; nothing fails outright, and the ceiling is a configured number rather than a surprise.
3. Freeze scaling	Set the HPA <code>maxReplicas</code> to the current replica count and record the prior value	Restore the recorded <code>maxReplicas</code> ; instant	A genuine traffic surge is served with the replicas already running rather than with new ones. Cost stops multiplying immediately. Slightly wider than throttle because it removes the cluster's ability to respond to real load.
4. Quarantine	Apply a deny-all-egress NetworkPolicy and remove the pod from the Service endpoints, leaving it running	Delete the NetworkPolicy and restore the label; seconds	One pod stops serving and stops talking outward. Its replicas absorb the load. The pod survives, so its state can still be inspected — which is exactly why quarantine is preferred to termination for anything suspected to be an intrusion.
5. Terminate	Delete the pod, or scale the workload to zero for a total shutdown	Not reversible. A replacement pod is a new pod, and in-flight requests and unflushed local state are lost	Requests in flight fail. Any forensic evidence held only in that pod's memory is destroyed. This rung requires the highest confidence, an explicit policy opt-in, and it is refused outright for anything tagged <code>critical</code> .

THE ONE PERMITTED SKIP IN THE LADDER

Escalation is always sequential and never skips rungs, with one exception: an anomaly classified `suspected_abuse` with security corroboration may jump directly to quarantine. The reasoning is that throttling a compromised workload slows an attacker down without stopping the exfiltration, so for that one classification the intermediate rung is not a smaller version of the right action, it is the wrong action performed gently.

25.3 Conflict resolution and precedence

Multiple policies will match the same anomaly, and a rule engine that resolves such collisions unpredictably is worse than no rule engine, because its behaviour cannot be reasoned about before an incident. Four resolution rules are applied in order.

1. **A prohibition always beats a permission.** If any matching policy lists an action under `never`, that action is unavailable regardless of what any other policy permits. This is a hard veto and it is evaluated first, which means a safety policy can be added without auditing every existing policy for conflicts.

2. **Lower priority number wins** among the policies that remain, matching the convention used by ingress controllers and firewall rules so that the semantics are already familiar.
3. **Ties break toward the less aggressive rung.** Two policies of equal priority proposing throttle and quarantine yield throttle.
4. **A dry-run policy neutralises the match.** If the winning policy has `dry_run: true`, the intended action is written to the audit log with mode `dry_run` and nothing is executed. This is how every policy begins life, and chapter 26 makes it the mandatory first stage of the trust ladder.

25.4 Safety rails on the executor

The Remediation Executor is the only component in the platform holding a write-capable Kubernetes credential, so its constraints are the constraints of the whole system. Seven rails are enforced inside the executor rather than in the policy interpreter, on the principle that the component holding the privilege must enforce the limits on that privilege even if it is asked to do otherwise.

Rail	Enforcement
Rate limit	At most three actions per workload per hour and ten per cluster per hour, counted from <code>actions_log</code> rather than from process memory, so the limit survives a restart of the executor.
Idempotency key	A deterministic key of anomaly id, action type and target. The unique constraint on <code>actions_log.idempotency_key</code> makes a duplicate delivery a no-op at the database level rather than a second action.
Prior-state capture	The target's relevant state is read and written to <code>prior_state</code> before the mutation is attempted. If the capture fails, the action is abandoned. An irreversible change is never made from an unknown starting point.
Minimum replicas	Scaling actions are clamped so that a workload never drops below its policy floor. A freeze that would leave one replica of a two-replica quorum is rejected and downgraded to an alert.
Timed auto-rollback	Every action carries a <code>rollback_deadline</code> . A reconciliation loop reverses any action whose deadline has passed and which no human has renewed. The default of fifteen minutes means the platform's changes expire on their own.
Kill switch	A single cluster-wide flag, readable in one query and settable from the interface header, that puts the executor into alert-only mode within one reconciliation interval. It is deliberately the simplest control in the system.
Least privilege	The RBAC role permits <code>patch</code> on <code>horizontalpodautoscalers</code> , <code>create</code> and <code>delete</code> on <code>networkpolicies</code> , and <code>delete</code> on <code>Pods</code> in the labelled namespaces. It has no verb on Secrets, no verb on ClusterRoles, and no access to unlabelled namespaces.

The timed auto-rollback deserves emphasis because it inverts the usual failure posture. Without it, a wrong action persists until somebody notices; with it, a wrong action persists for at most fifteen minutes and a *right* action requires a human to confirm that it should continue. The system's default behaviour under neglect is therefore to return production to the state the humans left it in.

EARNING THE RIGHT TO ACT

26. Autonomy, Trust and Human Authority

The claim that a platform acts autonomously is the claim that most deserves scepticism, and the honest response is not to soften the claim but to specify what has to be true before the platform is permitted to act at all. Autonomy in this design is earned per policy, measured, and revocable.

26.1 The trust ladder

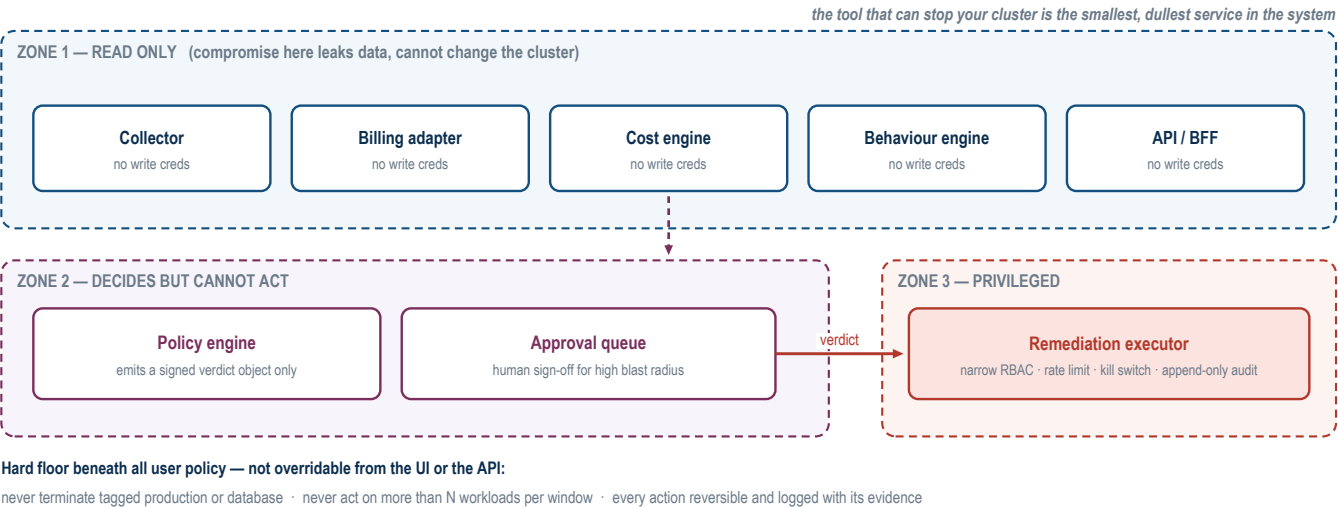


Figure 12 - Trust zones. The blast radius of a bug in HyperTrace is bounded by keeping write capability in one small, heavily constrained service.

Every policy begins in shadow mode and advances only on evidence. The four stages below are not a deployment convenience; they are the mechanism by which the false-positive rate of a specific rule against a specific workload becomes a known number rather than an assumption.

Stage	What the platform does	Promotion criterion
1. Shadow	Evaluates, logs the action it <i>would</i> have taken with mode <code>dry_run</code> , changes nothing	Fourteen days with a reviewed decision log and no decision an engineer judges would have been wrong
2. Recommend	Surfaces the proposed action in the interface with its evidence and a single approve button; a human executes	Thirty approvals with an approval rate above 90 percent. A rule humans routinely decline is not ready, and the decline rate is the measurement that says so
3. Autonomous, reversible only	Executes rungs 1 to 3 — alert, throttle, freeze scaling — without asking, notifies immediately, auto-rolls back on the deadline	Sixty days with no rollback initiated by a human overriding the platform
4. Autonomous, full ladder	May quarantine, and may terminate where the policy explicitly opts in	Requires named administrator sign-off recorded in the audit log. There is no automatic promotion to this stage

Demotion is automatic and asymmetric with promotion: two human-initiated rollbacks within seven days return a policy to the recommend stage without discussion. It is far easier to lose authority in this system than to gain it, which is the correct asymmetry for a component that can delete pods.

26.2 The anomaly lifecycle



state transitions are rows in `actions_log`, never in-place updates — that is what makes the audit trail defensible

Figure 15 - Anomaly lifecycle. Suppression and rollback are first-class states, not error paths.

An anomaly is a record with a lifecycle, not an event that fires and vanishes. Every transition below is written to the audit log with its actor, which is what allows the question “why did nothing happen to that anomaly?” to have an answer.

State	Meaning	Exits to
open	Detected and unresolved. Visible in the incident feed with its evidence and confidence.	acted, acknowledged, dismissed, expired
acted	A policy matched and an action was executed or queued for approval. Carries a reference to the <code>actions_log</code> row.	resolved when the metric returns to baseline, or rolled_back
acknowledged	A human has seen it and accepts it as known. Suppresses repeat notifications for the cooldown period but leaves the record open.	resolved, dismissed
dismissed	A human judged it a false positive and said so. This is the most valuable state in the system, because it is the only labelled negative the platform ever receives.	terminal
expired	The condition ceased before any action or acknowledgement. Kept, not deleted, because a pattern of self-resolving anomalies is itself a finding about the thresholds.	terminal

WHY THE DISMISS BUTTON MATTERS MORE THAN THE APPROVE BUTTON

Dismissals are the platform's only ground truth. A statistical detector cannot learn from unlabelled data, so the interface asks for one click and a reason on every dismissal, and the measurement harness in Part III computes the false-positive rate directly from those reasons. A dismissal that is only a dropdown selection with no reason is worth much less, which is why the field is required.

26.3 What remains a human decision, permanently

Four decisions are outside the platform's authority by design, and no confidence score or trust stage unlocks them. Naming them is not a limitation to be apologised for; it is the boundary that makes the rest of the autonomy defensible.

- **Changing a policy.** The platform never edits its own rules. It may report that a threshold looks poorly chosen; an administrator changes it. A system that tunes its own authority has no fixed authority.
- **Promoting a policy past stage three.** Termination authority is granted by a named person and recorded with their identity.
- **Declaring a security incident.** The platform quarantines and reports; classifying an event as a breach, with the disclosure obligations that follow, is a human and legal judgement.
- **Accepting a cost increase as the new normal.** Rebasement a baseline, per chapter 23.5, is a statement about what the business considers acceptable spend, and the platform has no standing to make it.

WHAT HAPPENS WHEN EACH PART BREAKS

27. Failure Modes and Graceful Degradation

A platform that takes action on production must be more careful about its own failures than the workloads it watches, because a confused controller is more dangerous than an absent one. This chapter enumerates the failure of every component and shows that each one loses observation or loses authority, and never silently gains either.

		TRIGGER
NORMAL	all services, 10s resolution, full autonomy	queue lag < 5s
SAMPLE	drop to 30s resolution on low-value services	lag 5-30s
SHED	archive path paused, hot path only	lag 30-120s
ALERT ONLY	detection continues, autonomous actions suspended	lag > 120s
FAIL SAFE	stop deciding, keep collecting, page a human	bus unreachable

Figure 13 - Degradation ladder. Incidents are exactly when load peaks, so the failure behaviour under load is a design feature, not an afterthought. the system never chooses "act on stale data" — when it cannot keep up, it gives up authority before it gives up accuracy

27.1 Component failure matrix

Component fails	Immediate effect	Degraded behaviour	Recovery
Metrics collector	No new samples enter the bus	Detection stops. Baselines are not updated, so no learning is corrupted by the gap. A staleness alert fires after 60 s and the interface marks data stale rather than showing the last value as current.	Restart; resume from the last committed offset. Prometheus retains the scrape window, so a short outage loses nothing.
Event bus	Producers cannot publish, consumers cannot read	Every downstream stage halts. Collectors buffer to local disk up to a bounded size and then drop oldest-first, which is the correct choice for a monitoring system: fresh data is worth more than complete data.	Restart the broker; consumers resume from committed offsets and replay the retained window.
Cost engine	No cost events produced	The behaviour engine still scores CPU, memory and traffic, so resource anomalies are still detected. Cost-dependent classifications and any policy with a <code>min_cost_delta_usd_hr</code> condition cannot fire, so those actions are withheld.	Restart and replay from the metrics topic; the composite key makes reprocessing idempotent.
Behaviour engine	No scores, no anomalies	Cost data still accrues and the dashboard remains accurate and current; the platform reverts to being a cost observability tool with static threshold alerts only.	Restart, load the checkpoint, replay from the offset committed with it. Up to 60 s of learning is recomputed, per chapter 23.6.
Policy engine	Anomalies are recorded but nothing is decided	The incident feed still fills, so humans can act manually. No autonomous action occurs. This is the failure mode the design most prefers.	Stateless; restart and resume. Open anomalies are re-evaluated on the next sample.
Remediation executor	Decisions are queued and not applied	Decisions accumulate with their rollback deadlines. On restart, any decision older than its deadline is discarded rather than executed — acting on a stale decision is worse than not acting.	Restart; the reconciliation loop reconciles actual cluster state against the audit log before accepting new work.
Database	No writes, no history, no policy reads	The executor fails closed: it cannot capture prior state or write the audit row, so per the rails in chapter 25.4 it takes no action at all. The system becomes read-only-blind and says so.	Restore or fail over. Buffered bus data is replayed once writes succeed.
API / dashboard	Humans cannot see or approve	Autonomous policies continue; recommend-stage policies stall, which correctly means no unattended action from rules that require attendance.	Stateless; restart behind the load balancer.

27.2 The three properties this table demonstrates

- No failure grants authority.** There is no component whose failure causes an action to be taken that would not otherwise have been taken. Every path degrades toward alert-only. This is a property that can be checked by reading the middle column, and it is the single most important claim in the chapter.
- No failure corrupts learning.** Gaps are skipped rather than zero-filled and state is checkpointed with its offset, so an outage costs the platform time but not accuracy. The alternative — a detector that emerges from an outage with a distorted model of normal — would produce a burst of false positives at exactly the moment operator patience is lowest.
- Staleness is always visible.** Every view that can show old data shows how old it is. A dashboard that displays a stale number as though it were current is the one failure mode with no technical alarm, because it looks exactly like

success.

27.3 Failure modes with no clean answer

Three cases are genuinely unresolved by this design, and stating them is more useful than pretending the matrix above is exhaustive.

- **A partitioned cluster.** If the executor can reach the Kubernetes API but the collector cannot reach the workloads, the platform holds authority over a cluster it cannot observe. The mitigation is a heartbeat that suspends the executor when collection is stale for more than 120 seconds, but a partition subtler than a total outage — where some services report and others do not — can still produce action on a partial picture.
- **A wrong price set.** If the price catalogue is loaded with incorrect rates, every cost figure is confidently wrong and the reconciliation loop of chapter 22.5 will take up to two days to notice, because billing data must settle first. A checksum against a published rate card at load time narrows this but does not close it.
- **The platform as the anomaly.** The collector scraping every pod every ten seconds is itself a workload with a cost, and a bug in it could plausibly trip its own detector. The platform's own namespace is therefore excluded from action but not from observation, which means a runaway collector is reported and never automatically remediated. That is the conservative choice and it is also an admission that the system cannot police itself.

PROTECTING THE THING THAT PROTECTS

28. Security, Tenancy and Self-Observability

A platform holding a credential that can delete pods across a cluster is a high-value target, and one holding cost data across teams is a confidentiality boundary. Both properties have to be designed for rather than assumed.

28.1 Authentication, authorisation and roles

Users authenticate with a short-lived JWT, fifteen minutes, paired with a refresh token rotated on use. Three roles cover the intended audiences and the differences between them are meaningful rather than cosmetic.

Role	May read	May write	May not
finance	Cost dashboards, unit costs, trends, and the anomaly feed without evidence payloads	Nothing	See process names, network destinations or any security evidence, because those are operational detail and not financial detail
sre	Everything, including full evidence chains	Approve or dismiss anomalies, execute recommended actions, trigger rollbacks, rebase baselines	Create or edit policies, promote a policy's trust stage, or change another user's role
admin	Everything	All of the above, plus policy authoring, trust promotion, kill-switch operation and user management	Modify or delete <code>actions_log</code> — the database refuses this for every role, including this one

28.2 Tenant isolation

Every tenant-scoped table carries `org_id`, and isolation is enforced by PostgreSQL row-level security rather than by a `WHERE` clause in application code. The distinction matters because a forgotten predicate in one of several hundred queries is a cross-tenant data leak, whereas a forgotten predicate under row-level security is simply an empty result set.

```

schema/06_rls.sql + the BFF side of the contract

ALTER TABLE cost_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE anomalies ENABLE ROW LEVEL SECURITY;
ALTER TABLE actions_log ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON cost_events
  USING (service_id IN (SELECT id FROM services
                        WHERE org_id = current_setting('app.org_id')::bigint));

-- The BFF sets this once per request, inside the transaction, from the JWT
-- claim. It is a SET LOCAL, so it cannot leak across pooled connections.
async with pool.acquire() as con:
    async with con.transaction():
        await con.execute("SELECT set_config('app.org_id', $1, true)",
                          str(claims['org_id']))
        rows = await con.fetch(SOME_QUERY)      # scoped automatically

```

The `true` third argument to `set_config` is what makes this safe under connection pooling: it scopes the setting to the transaction, so a connection returned to the pool carries no residue of the previous tenant. Getting that argument wrong is the classic way this pattern fails, and it fails silently.

28.3 Hardening the platform's own footprint

- **Credential separation.** Only the Remediation Executor holds a write-capable Kubernetes credential. Collectors use a distinct read-only ServiceAccount. The API holds no Kubernetes credential at all and reaches the executor over the bus, so compromising the internet-facing component yields no cluster write path.
- **Containers.** Every image runs as a non-root user with a read-only root filesystem, all capabilities dropped, `allowPrivilegeEscalation: false`, and a seccomp profile of `RuntimeDefault`. The collector is the one component needing host-level read access and it gets exactly that and nothing more.
- **Network.** A default-deny NetworkPolicy in the platform namespace, with explicit allows for bus, database and Kubernetes API traffic only. The platform cannot reach the internet, which means an exfiltration path through it does not exist even if a component is compromised.
- **Secrets.** No credential in an image or a manifest. Cloud credentials arrive through the provider's workload identity mechanism, so nothing long-lived is stored at all; the database password is the one static secret and it is mounted as a file rather than an environment variable, keeping it out of process listings and crash dumps.
- **Supply chain.** Images are pinned by digest rather than tag, dependencies are locked, and a vulnerability scan runs in the pipeline. Pinning by digest is what makes a deployment reproducible; a tag is a mutable pointer and treating it as a version is a common and consequential error.

28.4 Observing the observer

The platform exports its own metrics, and the eight below are the ones that actually predict its failures. They exist because the useful question is not whether the process is running but whether it is keeping up and whether anyone still believes it.

Metric	Why it is the one that matters	Alert
<code>ht_bus_consumer_lag_seconds</code>	The single best leading indicator. Growing lag means every latency number in chapter 19 is now fiction, and it grows before anything else visibly breaks.	> 30 s for 2 min
<code>ht_active_series_total</code>	The cardinality ceiling of chapter 21.5. This one degrades slowly and irreversibly, so it must be watched rather than discovered.	> 40,000
<code>ht_decision_latency_seconds</code>	The end-to-end p95 against the 5 s NFR-1 budget, measured rather than assumed.	p95 > 5 s
<code>ht_cost_drift_ratio</code>	The reconciliation metric of chapter 22.5. Rising drift silently invalidates every cost-based decision.	drift > 0.15
<code>ht_baseline_maturity_ratio</code>	The fraction of buckets past the maturity gate. A falling value means detection coverage is shrinking even though nothing has errored.	< 0.7
<code>ht_actions_total</code> by result	Executor health and the rate-limit picture in one series.	any failure

Metric	Why it is the one that matters	Alert
<code>ht_dismissal_rate</code>	Human dismissals over total anomalies — the platform's own false-positive estimate, and the number that decides whether anyone keeps using it.	> 0.3 over 7 d
<code>ht_stale_services_total</code>	Services with no sample inside two intervals. Distinguishes “nothing is wrong” from “nothing is being watched”.	> 0 for 5 min

TWO METRICS THAT MEASURE WHETHER THE PLATFORM IS TRUSTED

The last two metrics are the ones a project of this kind usually omits, and they are the ones that decide whether it survives contact with users. A platform with a dismissal rate above 0.3 is being ignored, and a platform with stale services is reporting an all-clear about workloads it cannot see. Neither shows up as an error in a log.

WHAT THE NUMBERS HAVE TO BE, AND WHAT THEY COST

29. Performance, Scale and Capacity

The non-functional requirements of chapter 12 assert targets. This chapter works out what those targets imply for data volume, memory and compute, so that the prototype's sizing is a calculation rather than a guess and so that the point at which the architecture would have to change can be named.

29.1 Data volume arithmetic

Take the target scale of NFR-2: ten thousand monitored resources at a ten-second collection interval. That is one thousand samples per second entering the bus. Each sample serialises to roughly 220 bytes of JSON, giving about 220 KB per second, which is 19 GB per day before compression.

Samples per second	10,000 resources / 10 s = 1,000 per second
Raw bus throughput	$1,000 \times 220 \text{ B} \approx \textbf{220 KB/s}$, or 19 GB/day
Bus retention at 24 h	19 GB, roughly 6 GB with compression enabled
Raw metrics in the database	7-day retention, columnar-compressed at about 8:1 $\approx \textbf{17 GB}$
Cost events	One per resource per window, similar rate; 400-day retention with compression $\approx \textbf{60 GB}$
Baseline state	10,000 services $\times$ 6 metrics $\times$ 168 buckets $\times$ 4 values $\approx \textbf{320 MB}$ resident
Continuous aggregate	Hourly rows: $10,000 \times 24 \times 400 \approx 96 \text{ M rows}$, under 10 GB

The conclusion is worth stating because it is counter-intuitive and it shapes every hardware decision that follows: the whole system at its target scale fits comfortably in about 100 GB of storage and a few gigabytes of memory. This is not a big-data problem. It is a latency problem with a modest data volume attached, and treating it as the former — reaching for a distributed data warehouse — would add operational cost with no benefit.

29.2 Where the compute goes

Stage	Cost per sample	At 1,000 samples/s
Deserialise and validate	$\sim 15 \mu\text{s}$	1.5 percent of one core
Cost calculation, five multiplications and a lookup	$\sim 5 \mu\text{s}$	0.5 percent of one core
Baseline update and scoring, six metrics	$\sim 25 \mu\text{s}$, entirely in memory with no I/O	2.5 percent of one core
Policy evaluation	Only on anomaly, not per sample; $\sim 200 \mu\text{s}$ each	Negligible — anomalies are rare by construction
Database write, batched at 500 rows	$\sim 2 \text{ ms}$ per batch, amortised $\sim 4 \mu\text{s}$	0.4 percent of one core

Under five percent of a single core for the entire hot path at target scale. The reason is that the design deliberately excluded every expensive operation from the hot path in chapter 19: no model inference, no joins, no per-sample cloud API calls, no cross-service correlation. The engineering that produces this number was done in the architecture, not in the implementation, and it is the direct payoff of choosing an EWMA Z-score over a learned model.

29.3 Sizing the local demonstration

The Part III build runs everything on one laptop against a small cluster, which is a different regime: the constraint is total memory, not throughput. Concrete figures follow so that the environment can be prepared before anything is installed.

Component	CPU request	Memory	Note
kind control plane + 2 workers	1.0	2.0 GiB	The floor. A single-node cluster works but cannot demonstrate a node-level cost story.
Prometheus	0.2	1.0 GiB	6-hour retention is ample; the database holds history.
Redpanda (single broker)	0.5	1.0 GiB	Kafka-compatible and markedly lighter than Kafka with ZooKeeper on a laptop.
TimescaleDB	0.5	1.0 GiB	Tune <code>shared_buffers</code> down to 256 MB for a laptop.
Five platform services	0.5	0.75 GiB	150 MB each; they are small.
Frontend dev server	0.2	0.5 GiB	Vite in development mode.
Victim microservice, 3 replicas	0.3	0.3 GiB	The thing being watched.
Total	3.2	6.55 GiB	Workable on 8 GiB; comfortable on 16 GiB

29.4 The first bottleneck, and the honest ceiling

Predicting where a system breaks first is more useful than claiming it does not. In this design the first constraint reached is single-writer throughput in the Behaviour Analysis Service: because baseline state for a given service must be owned by exactly one process, per chapter 23.6, throughput per partition is bounded by one core's worth of scoring. At roughly 25 μ s per sample that is about 40,000 samples per second per partition, so a twelve-partition deployment reaches a theoretical ceiling near 480,000 samples per second — well beyond the 1,000 per second target, but the ceiling is real and it is architectural rather than a matter of tuning.

Beyond that point the design would have to change in a specific way rather than simply be given more hardware: partitions would need to be split by service with consistent hashing and rebalancing, which introduces state migration between processes — the one genuinely hard distributed-systems problem this architecture currently avoids by construction. It is out of scope for a prototype, and it is the right answer to a panel asking what happens at ten times the target: not “it scales”, but “it scales to here, and past here the partition ownership model has to be replaced, for these reasons.”

FROM SPECIFICATION TO CONSTRUCTION

Part II has specified the system: its principles, structure, data, arithmetic, detection, policy, autonomy, failure behaviour, security and limits. Part III now builds it, from an empty laptop to a working demonstration, in fourteen milestones.

Hands-On Build Guide: Kind & Local Stack

A step-by-step implementation guide to build and run HyperTrace entirely on a local development machine using Docker, Kind, Prometheus, Redpanda, and TimescaleDB — with zero cloud costs.

Chapter 30	Prerequisites, tools installation, and local architecture
Milestones 1–3	Kind cluster setup, platform helm charts, and simulated shop workload
Milestones 4–6	Metrics collector, cost intelligence engine, and behaviour analysis
Milestones 7–9	Policy engine, remediation executor, and FastAPI backend
Milestones 10–14	React dashboard, live incident simulation, chaos testing, and verification

WHAT TO INSTALL, AND THE MENTAL MODEL TO HOLD

30. Before You Start

Part III builds the system specified in Part II on a single laptop, at zero cloud cost, in fourteen milestones. Everything runs in containers on your own machine. There is no cloud account, no credit card, and nothing to shut down afterwards except one Docker cluster.

30.1 What you need on the machine

Tool	Version	What it is, in one sentence	Check
Docker Desktop	4.30 or newer	Runs containers on your laptop. Everything else in this build runs inside it.	<code>docker --version</code>
kind	0.23 or newer	Creates a real Kubernetes cluster whose nodes are Docker containers — “Kubernetes IN Docker”.	<code>kind --version</code>
kubectl	1.30 or newer	The command-line client you use to talk to Kubernetes.	<code>kubectl version --client</code>
Helm	3.15 or newer	A package manager for Kubernetes; installs Prometheus and similar components in one command instead of twenty manifests.	<code>helm version</code>
Python	3.11 or newer	The language of all five backend services.	<code>python3 --version</code>
Node.js	20 LTS or newer	Needed only for the React dashboard in milestone 12.	<code>node --version</code>
A code editor	any	VS Code with the Python and Kubernetes extensions is the path of least resistance.	—

Hardware matters more than usual here, because you are running a small data centre on one machine. From the sizing in chapter 29.3: 8 GiB of RAM is the workable minimum, 16 GiB is comfortable, and you want roughly 20 GB of free disk. On 8 GiB, run the frontend dev server on the host rather than in the cluster and keep the browser to a few tabs. Apple Silicon, Intel and AMD all work; on Windows, use WSL 2 as the Docker backend and run every command from inside the WSL shell rather than from PowerShell, because line endings and path translation will otherwise cost you an afternoon.

30.2 The mental model

If Kubernetes is new to you, five ideas carry almost all of this build. They are worth reading slowly once, because every later milestone assumes them.

Container	A packaged process with its own filesystem. Think of it as a program plus everything it needs to run, in one file.
Pod	One or more containers scheduled together as a unit. Kubernetes never runs a container directly; it runs pods. A pod is disposable and gets a new name and IP every time it is replaced — which is exactly why chapter 21.5 refuses to make pod names part of a metric's identity.
Deployment	A declaration that says “keep three copies of this pod running”. You change the declaration; Kubernetes makes reality match it. This is the whole idea of the system.
Service	A stable name and address in front of a changing set of pods. Pods come and go; the Service name does not.
HPA	The Horizontal Pod Autoscaler. It watches a metric and changes the replica count automatically. It is the component that turns a bug into a bill, and therefore the component this platform most often reaches for when it needs to stop something.

Two further ideas belong to the platform rather than to Kubernetes. A **time-series database** stores numbers stamped with a time and is optimised for the question “what happened between 10:00 and 10:05” rather than “find me row 47”; you will use TimescaleDB, which is PostgreSQL with that capability added, so ordinary SQL works throughout. And an **event bus** is a durable queue that lets one service publish a message and several others read it independently, each at its own pace, with the messages retained afterwards so a new consumer can replay history — the property chapter 19.4 depends on.

30.3 What you are going to build

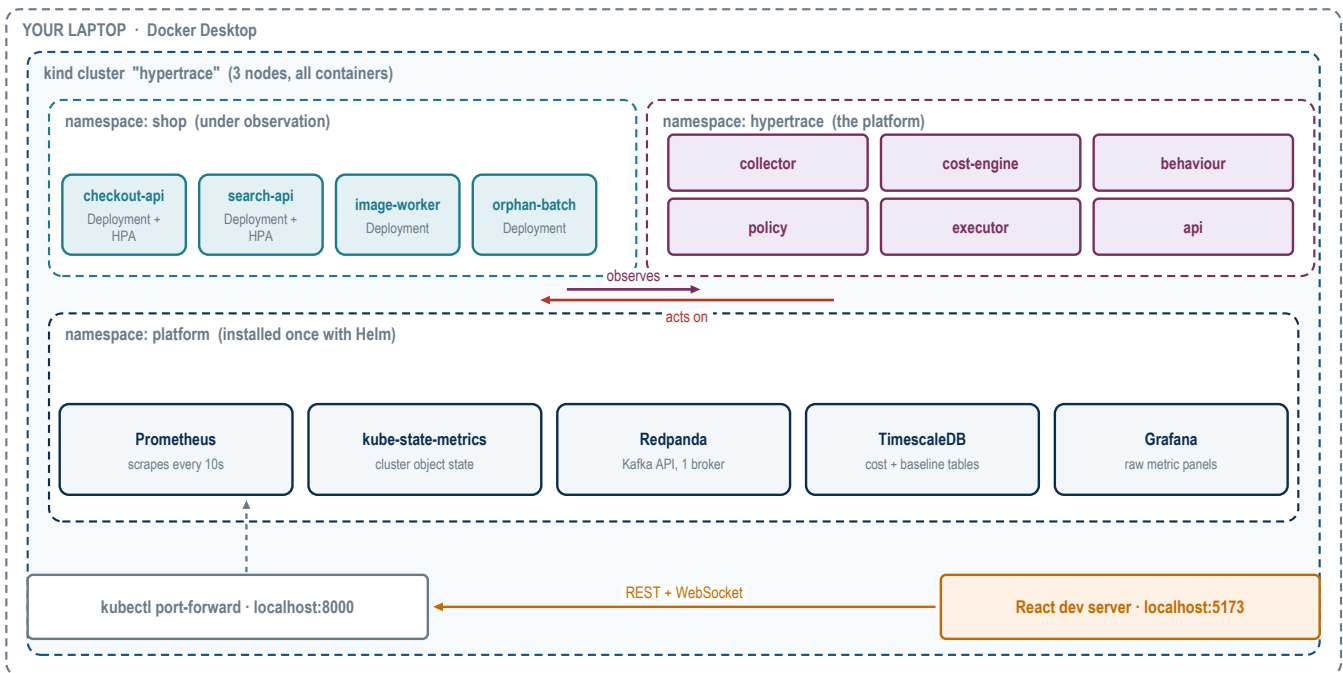


Figure 10 - What you actually run on your own machine in Part III. No cloud account, no bill, no credit card: the whole system including its victim workloads lives in one kind cluster.

Three namespaces, which are simply named partitions inside the cluster. **shop** holds the system under observation: four workloads with deliberately injectable failure modes, so you can cause an incident on demand rather than waiting for one. **hypertrace** holds the six platform components you will write. **platform** holds the dependencies you install once with Helm and then largely forget.

30.4 The route through the milestones

You build it in this order, and after every milestone something visibly works.

each row depends only on the row above

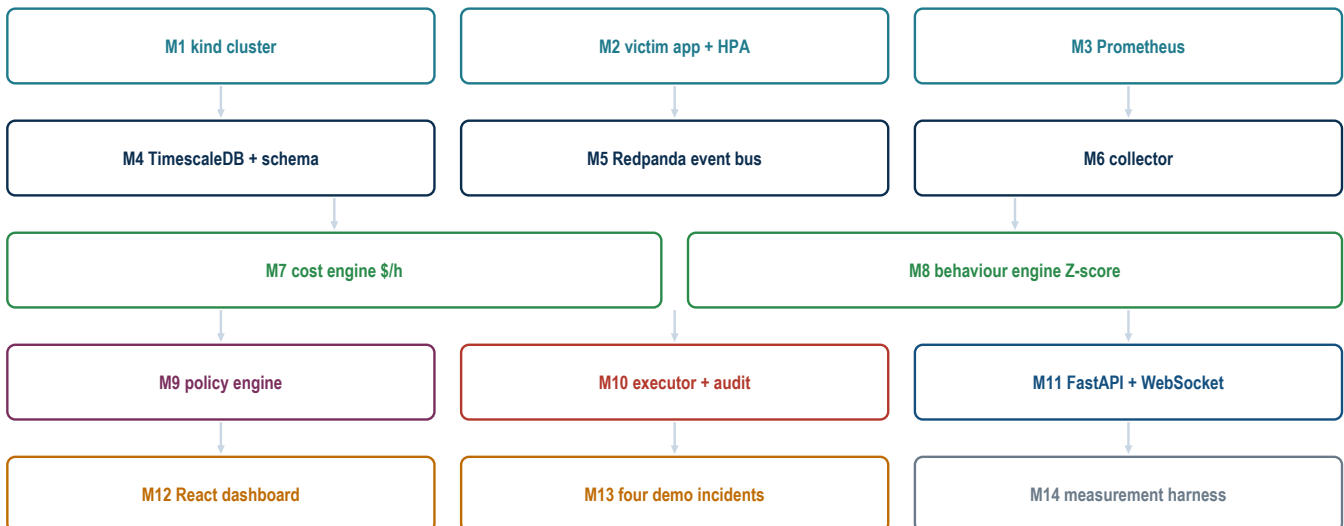


Figure 17 - Build order for Part III. Nothing in a later row is needed to demonstrate an earlier row, so you always have something working to show.

Work in order. The dependency chain is real: the cost engine cannot be tested without metrics flowing, and the behaviour engine cannot be tested without cost events. Each milestone ends with a checkpoint listing what must be observably true before you continue, and every one of those checks is a command with an expected output rather than an instruction to look and see whether it seems fine.

HOW TO WORK THROUGH PART III

Two habits will save you more time than any amount of care with the code. First, after every milestone, run its checkpoint commands and do not proceed on a failure — in a pipeline, a broken stage three shows up as confusing nonsense in stage seven. Second, commit to git after each milestone with the milestone number in the message. You will want to go back, and you will not remember which change broke it.

30.5 Repository layout

Create this structure first. Having the empty shape in front of you makes every later instruction unambiguous about where a file goes, and the layout itself encodes the service boundaries of chapter 18.

the shape to create before milestone 1

```
hypertrace/
  Makefile                # every command in this guide, named
  docker-compose.yaml     # database + bus for local dev without Kubernetes
  .env.example            # copy to .env; never commit .env
  schema/                 # 01_dimensions.sql .. 06_rls.sql from chapter 21
  services/
    common/               # shared: config, bus client, logging, models
    collector/            # milestone 6
    costengine/           # milestone 7
    behaviour/            # milestone 8
    policy/               # milestone 9
    executor/             # milestone 10
    api/                  # milestone 11
  frontend/               # milestone 12, React + Vite
  victim/                 # milestone 2, the app being watched
  deploy/
    kind-cluster.yaml     # milestone 1
    namespaces.yaml
    platform/             # Helm values for Prometheus, Redpanda, Timescale
    hypertrace/           # Deployments, Services, RBAC for the six services
    shop/                 # the victim workloads and their HPAs
  policies/               # YAML policy documents from chapter 25.1
  harness/                # milestone 14, the measurement scripts
  tests/
```

terminal

```
mkdir -p hypertrace/{schema,policies,harness,tests}
cd hypertrace
mkdir -p services/{common,collector,costengine,behaviour,policy,executor,api}
mkdir -p deploy/{platform,hypertrace,shop} frontend victim
git init && git add -A && git commit -m 'M0: repository skeleton'
```

Milestone 1 - A Kubernetes cluster on your laptop

MILESTONE 1

TIME

A Kubernetes cluster on your laptop

45-60 minutes

You will have, at the end: a three-node cluster running in Docker, with a namespace layout and enough cluster-level metrics to prove it is alive.

kind runs each Kubernetes node as a Docker container. That sounds like a toy and is not: it is a genuine, conformant Kubernetes API server with real scheduling, real RBAC and real networking. What it lacks is a cloud provider underneath, which is why the price data in milestone 7 is a simulated catalogue rather than a live API.

1.1 Define the cluster

```

deploy/kind-cluster.yaml

# deploy/kind-cluster.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: hypertrace
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 30080      # the dashboard, reachable at localhost:8080
    hostPort: 8080
    protocol: TCP
  - containerPort: 30090      # Prometheus, at localhost:9090
    hostPort: 9090
    protocol: TCP
- role: worker
  labels:
    ht.io/nodegroup: general    # pretend this is an m5.large
- role: worker
  labels:
    ht.io/nodegroup: general

```

The `extraPortMappings` block is what makes `localhost:8080` reach a service inside the cluster; without it you would need a port-forward running in a spare terminal for the rest of the project. The node labels are pretending to be an instance type: milestone 7 looks up a node's hourly price by this label, which is how a laptop cluster produces plausible dollar figures.

1.2 Create it

```

terminal

kind create cluster --config deploy/kind-cluster.yaml

# Expected, after 60-90 seconds:
#   Creating cluster "hypertrace" ...
#   - Ensuring node image (kindest/node:v1.30.0)
#   - Preparing nodes
#   - Writing configuration
#   - Starting control-plane
#   - Installing CNI
#   - Joining worker nodes
#   Set kubectl context to "kind-hypertrace"

kubectl get nodes -o wide
#  NAME                STATUS    ROLES          VERSION
#  hypertrace-control-plane  Ready    control-plane  v1.30.0
#  hypertrace-worker        Ready    <none>         v1.30.0
#  hypertrace-worker2       Ready    <none>         v1.30.0

```

If a node sits in `NotReady` for more than two minutes, the cause is almost always Docker memory. Raise Docker Desktop's memory allocation to at least 6 GiB in its resources settings, then `kind delete cluster --name hypertrace` and create it again. Deleting and recreating a kind cluster costs ninety seconds and is nearly always faster than diagnosing one that came up wrong.

1.3 Namespaces and resource discipline

```

deploy/namespaces.yaml

# deploy/namespaces.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    ht.io/monitored: "true"      # the collector watches labelled namespaces
    ht.io/team: checkout
---
apiVersion: v1
kind: Namespace
metadata:
  name: hypertrace
  labels:
    ht.io/monitored: "true"
    ht.io/no-action: "true"     # observed, never acted on - chapter 27.3
---
apiVersion: v1
kind: Namespace
metadata:
  name: platform
---
# Stop one runaway pod from taking the whole laptop down with it.
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: shop
spec:
  limits:
    - type: Container
      default:
        cpu: 300m
        memory: 256Mi
      defaultRequest:
        cpu: 100m
        memory: 128Mi
      max:
        cpu: "1"
        memory: 512Mi

```

DO NOT SKIP THE LIMITRANGE

That `LimitRange` is not bureaucracy, it is self-defence. In milestone 13 you will deliberately run a CPU-burning loop inside this namespace. Without a ceiling it consumes every core your Docker VM has, the Kubernetes API server starves, and your laptop stops responding to `kubectl` — including the command you would use to stop it.

1.4 The metrics server

kind ships without a metrics server, so `kubectl top` and, more importantly, any HPA will not work until you install one. The `--kubelet-insecure-tls` patch is required because kind's kubelets use self-signed certificates; it is acceptable here and would not be in production.

terminal

```
kubectl apply -f \
  https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

kubectl patch -n kube-system deployment metrics-server --type=json -p \
  '[{"op": "add", "path": "/spec/template/spec/containers/0/args/-",
    "value": "--kubelet-insecure-tls"}]'

kubectl -n kube-system rollout status deployment metrics-server --timeout=120s
kubectl top nodes
```

#	NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
#	hypertrace-control-plane	214m	5%	712Mi	9%

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ `kubectl get nodes` shows three nodes, all **Ready**.
- ▮ `kubectl get ns` lists **shop**, **hypertrace** and **platform**.
- ▮ `kubectl top nodes` returns numbers rather than an error — this is the check that the metrics server actually works.
- ▮ `docker ps` shows three **kindest/node** containers.
- ▮ You have committed **deploy/kind-cluster.yaml** and **deploy/namespaces.yaml** to git.

Milestone 2 - The workload you are going to watch

MILESTONE 2

TIME

The workload you are going to watch**1-1.5 hours**

You will have, at the end: four running workloads in the shop namespace, one of which can be told to misbehave in four different ways through an HTTP call.

You cannot test an anomaly detector without anomalies, and waiting for a real one is not a plan. This milestone builds a small FastAPI service with four injectable failure modes, each corresponding to one row of the scenario table in chapter 2. Being able to cause a specific incident on demand, reproducibly, is what makes the measurement harness in milestone 14 possible at all.

2.1 The victim service

victim/app.py - part 1: the service and the injection switch

```

# victim/app.py
import asyncio, hashlib, os, random, socket, time
from fastapi import FastAPI
from fastapi.responses import PlainTextResponse
from prometheus_client import Counter, Gauge, generate_latest

app = FastAPI(title="checkout-api (victim)")
POD = os.getenv("HOSTNAME", socket.gethostname())

REQS = Counter("app_requests_total", "Requests served", ["route"])
MODE = Gauge("app_failure_mode", "1 when a failure mode is active", ["mode"])

# Every mode starts off. Nothing misbehaves until you ask it to.
state = {"cpu_burn": False, "mem_leak": False,
         "egress_flood": False, "retry_storm": False}
_leak: list[bytes] = []
for m in state:
    MODE.labels(mode=m).set(0)

@app.get("/checkout")
async def checkout():
    """The normal path. Deliberately does a little real work so that CPU
    usage correlates with request rate under healthy conditions."""
    REQS.labels(route="checkout").inc()
    h = hashlib.sha256(os.urandom(2048))
    for _ in range(220):
        h = hashlib.sha256(h.digest())
    await asyncio.sleep(random.uniform(0.004, 0.020))
    return {"ok": True, "pod": POD, "digest": h.hexdigest()[:12]}

@app.get("/healthz")
async def healthz():
    return {"status": "ok"}

@app.get("/metrics")
async def metrics():
    return PlainTextResponse(generate_latest())

@app.post("/inject/{mode}")
async def inject(mode: str, on: bool = True):
    """Turn a failure mode on or off. This endpoint is the reason the
    whole project is demonstrable; in a real service it would not exist."""
    if mode not in state:
        return {"error": "unknown mode", "known": list(state)}
    state[mode] = on
    MODE.labels(mode=mode).set(1 if on else 0)
    if mode == "mem_leak" and not on:
        _leak.clear()
    return {"mode": mode, "active": on, "pod": POD}

```

victim/app.py - part 2: the four failure modes

```
# victim/app.py (continued)

async def cpu_burner():
    """Scenario 1, chapter 2: a runaway loop. CPU rises, traffic does not.
    Yields every 40ms so the event loop still answers /healthz - a pod that
    fails its liveness probe gets restarted, which would mask the anomaly."""
    while True:
        if state["cpu_burn"]:
            end = time.perf_counter() + 0.040
            x = 0.0
            while time.perf_counter() < end:
                x += hashlib.sha256(str(x).encode()).digest()[0]
            await asyncio.sleep(0.004)

async def mem_leaker():
    """Scenario 2: memory climbs steadily and is never released. 4 MiB/s."""
    while True:
        if state["mem_leak"]:
            _leak.append(os.urandom(4 * 1024 * 1024))
            await asyncio.sleep(1.0)

async def egress_flooder():
    """Scenario 3: sustained outbound traffic to a fixed destination, the
    shape a cryptominer or an exfiltration produces. Sends to a sink inside
    the cluster so nothing leaves your laptop."""
    sink = os.getenv("EGRESS_SINK", "egress-sink.shop.svc.cluster.local")
    payload = os.urandom(64 * 1024)
    while True:
        if state["egress_flood"]:
            try:
                r, w = await asyncio.open_connection(sink, 9000)
                for _ in range(16):
                    w.write(payload)
                    await w.drain()
                w.close()
            except OSError:
                pass
            await asyncio.sleep(0.05)

async def retry_stormer():
    """Scenario 4: an internal retry loop. Request count to a DEPENDENCY
    explodes while ingress request count stays flat - the case a
    traffic-based detector cannot see at all."""
    while True:
        if state["retry_storm"]:
            await asyncio.gather(*[checkout() for _ in range(24)])
            await asyncio.sleep(0.10)

@app.on_event("startup")
async def start_background():
    for coro in (cpu_burner, mem_leaker, egress_flooder, retry_stormer):
        asyncio.create_task(coro())
```

Each mode is chosen to exercise a different part of the detector, and the fourth is the interesting one. A retry storm raises CPU and cost while ingress request count stays completely flat, which is precisely the decoupling that chapter 24 tests for and precisely what a request-rate dashboard shows as normal. When you demonstrate this project, demonstrate that mode.

2.2 Container image and manifests

victim/Dockerfile

```
# victim/Dockerfile
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir fastapi==0.115.0 uvicorn==0.30.6 \
    prometheus-client==0.21.0
COPY app.py .
RUN useradd -u 10001 -m appuser
USER 10001
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

deploy/shop/checkout-api.yaml


```
# deploy/shop/checkout-api.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: checkout-api
  namespace: shop
  labels: {app: checkout-api, ht.io/service-id: svc_checkout}
spec:
  replicas: 3
  selector:
    matchLabels: {app: checkout-api}
  template:
    metadata:
      labels: {app: checkout-api, ht.io/service-id: svc_checkout}
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8000"
    spec:
      containers:
        - name: app
          image: hypertrace/victim:dev
          imagePullPolicy: IfNotPresent
          ports: [{containerPort: 8000}]
          resources:
            requests: {cpu: 100m, memory: 128Mi}
            limits: {cpu: 500m, memory: 384Mi}
          livenessProbe:
            httpGet: {path: /healthz, port: 8000}
            periodSeconds: 10
            failureThreshold: 6          # tolerant: a CPU burn must not
                                         # restart the pod mid-experiment
---
apiVersion: v1
kind: Service
metadata:
  name: checkout-api
  namespace: shop
spec:
  selector: {app: checkout-api}
  ports: [{port: 80, targetPort: 8000}]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: checkout-api
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: checkout-api
  minReplicas: 2
  maxReplicas: 12                      # the runaway-cost mechanism
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 60}
```

The HPA is the villain of the story and deserves a look. It scales on CPU utilisation, so a CPU-burning bug looks identical to popularity: utilisation rises, replicas are added, each new replica burns CPU too, utilisation stays high, more replicas are added. Twelve replicas of a bug cost twelve times as much as one and serve no more customers. This autoscaler is behaving exactly as configured, which is the point of chapter 2 — nothing here is broken, and the bill still triples.

2.3 Build, load and deploy

`kind` cannot pull from your local Docker daemon, so images must be loaded into the cluster explicitly. Forgetting `kind load` after a rebuild is the single most common source of “my change did nothing” in this whole build; the pod keeps running the old image and looks perfectly healthy.

```
terminal

docker build -t hypertrace/victim:dev victim/
kind load docker-image hypertrace/victim:dev --name hypertrace
kubectl apply -f deploy/shop/
kubectl -n shop rollout status deployment checkout-api

# Prove it serves traffic:
kubectl -n shop port-forward svc/checkout-api 8000:80 &
curl -s localhost:8000/checkout | head -c 120
# {"ok":true,"pod":"checkout-api-7d9f8c6b4-x2klm","digest":"9f2a..."}

# Prove injection works, then turn it straight back off:
curl -sX POST 'localhost:8000/inject/cpu_burn?on=true'
sleep 45 && kubectl top pods -n shop
# NAME CPU(cores) MEMORY(bytes)
# checkout-api-7d9f8c6b4-x2klm 487m 96Mi <- burning
# checkout-api-7d9f8c6b4-p8trn 3m 88Mi
curl -sX POST 'localhost:8000/inject/cpu_burn?on=false'
```

YOU HAVE BUILT THE PROBLEM

Watch `kubectl get hpa -n shop -w` while the burn is active. Within about a minute the replica count starts climbing toward 12. You have just reproduced, on a laptop and for free, the exact failure that motivates this entire project. Turn the mode off and confirm the count comes back down before moving on.

CHECKPOINT - VERIFY BEFORE MOVING ON

- `kubectl get pods -n shop` shows three `checkout-api` pods `Running`.
- `curl localhost:8000/checkout` returns JSON containing a pod name.
- `curl localhost:8000/metrics` returns Prometheus text including `app_requests_total`.
- Each of the four inject modes visibly changes `kubectl top pods` output, and switches off cleanly.
- The HPA scales up under `cpu_burn` and back down afterwards.

Milestone 3 - Metrics collection with Prometheus

MILESTONE 3

TIME

Metrics collection with Prometheus

1 hour

You will have, at the end: Prometheus scraping every pod in the monitored namespaces every ten seconds, with the exact queries the collector will use in milestone 6 verified by hand.

Prometheus is the observation layer of chapter 4 and it is deliberately not reinvented here. It scrapes, it stores a short window, and it answers queries in PromQL. The platform reads from it rather than instrumenting workloads itself, which is what allows a service to be monitored without being modified — the property that makes the whole thing adoptable.

3.1 Install the stack

```
deploy/platform/prometheus-values.yaml
```

```
# deploy/platform/prometheus-values.yaml
prometheus:
  prometheusSpec:
    scrapeInterval: 10s          # NFR-1 depends on this; see chapter 19
    evaluationInterval: 10s
    retention: 6h                # history lives in TimescaleDB, not here
    resources:
      requests: {cpu: 100m, memory: 512Mi}
      limits:   {cpu: 500m, memory: 1Gi}
    # Scrape any pod carrying prometheus.io/scrape=true, in any namespace.
    podMonitorSelectorNilUsesHelmValues: false
    serviceMonitorSelectorNilUsesHelmValues: false
  service:
    type: NodePort
    nodePort: 30090              # mapped to localhost:9090 in milestone 1

grafana:
  enabled: true
  adminPassword: admin          # local only; never do this elsewhere
  resources:
    requests: {cpu: 50m, memory: 128Mi}

kubeStateMetrics:
  enabled: true                  # HPA state, replica counts, requests
nodeExporter:
  enabled: true                  # node CPU and network counters

alertmanager:
  enabled: false                 # HyperTrace is the alerting path
```

terminal

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update

helm install kps prometheus-community/kube-prometheus-stack \
  --namespace platform \
  -f deploy/platform/prometheus-values.yaml \
  --wait --timeout 10m

kubectl -n platform get pods
#   kps-prometheus-node-exporter-*      Running
#   kps-kube-state-metrics-*            Running
#   prometheus-kps-prometheus-0        Running   (2/2)
#   kps-grafana-*                       Running
```

Give this ten minutes on first run; it pulls several hundred megabytes of images. If the Helm install times out, it is usually still progressing — check `kubectl -n platform get pods` before assuming failure, and re-run `helm upgrade --install` rather than deleting anything.

3.2 Tell Prometheus to scrape the shop namespace

```

deploy/platform/podmonitor-shop.yaml

# deploy/platform/podmonitor-shop.yaml
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: shop-workloads
  namespace: platform
spec:
  namespaceSelector:
    matchNames: [shop, hypertrace]
  selector:
    matchExpressions:
      - {key: ht.io/service-id, operator: Exists}
  podMetricsEndpoints:
    - port: "8000"
      path: /metrics
      interval: 10s
      relabelings:
        # Promote the service-id label to a first-class metric label. This is
        # the stable key of chapter 21.5 - it survives every rollout.
        - sourceLabels: [__meta_kubernetes_pod_label_ht_io_service_id]
          targetLabel: service_id
          action: replace
        - sourceLabels: [__meta_kubernetes_namespace]
          targetLabel: k8s_namespace
          action: replace

```

3.3 The six queries the collector will run

Verify each of these in the Prometheus expression browser at `localhost:9090` before writing any collector code. If a query returns nothing here, it will return nothing from Python too, and you will spend an hour debugging the wrong file. Every query aggregates by `service_id` for the cardinality reasons of chapter 21.5.

```

the PromQL contract between Prometheus and the collector

# 1. CPU cores actually used, per service, averaged over 2 minutes.
#   rate() on a counter gives per-second increase; for cpu_seconds that
#   is exactly "cores in use". This is the single most useful query here.
sum by (service_id) (
  rate(container_cpu_usage_seconds_total{namespace="shop"}[2m])
)

# 2. Memory working set in GiB. A gauge, so no rate() - just convert.
sum by (service_id) (
  container_memory_working_set_bytes{namespace="shop"}
) / 1024 / 1024 / 1024

# 3. Egress bytes per second, per service.
sum by (service_id) (
  rate(container_network_transmit_bytes_total{namespace="shop"}[2m])
)

# 4. Requests per second - the business signal and the decoupling denominator.
sum by (service_id) (rate(app_requests_total{namespace="shop"}[2m]))

# 5. Requested (not used) CPU - the shared-cost denominator of chapter 22.2.
sum by (service_id) (
  kube_pod_container_resource_requests{namespace="shop", resource="cpu"}
)

# 6. Current replica count, for the freeze-scaling action of chapter 25.2.
sum by (service_id) (kube_deployment_status_replicas{namespace="shop"})

```

COUNTERS VERSUS GAUGES

The distinction between `rate()` on a counter and reading a gauge directly is the one thing to be sure of before continuing. A counter only ever increases and is meaningless read raw — `container_cpu_usage_seconds_total` for a pod up two days is a large number with no relation to current activity. Wrapped in `rate(...[2m])` it becomes cores in use. A gauge such as memory working set is already a current value and `rate()` on it produces nonsense.

Two practical notes on the two-minute window. It is longer than the ten-second scrape interval on purpose: `rate()` needs at least two samples, and a window barely wider than the interval produces gaps whenever a scrape is late. And the window costs you latency — a step change in CPU takes roughly half the window to be fully reflected, which is one of the largest single contributors to the end-to-end budget in chapter 19.2. Two minutes is the compromise; below one minute the values get noisy enough to trip the detector on their own.

CHECKPOINT - VERIFY BEFORE MOVING ON

- `localhost:9090` loads the Prometheus interface in your browser.
- Status □ Targets shows the `shop-workloads` PodMonitor with three endpoints `UP`.
- All six queries above return a result with a `service_id` label of `svc_checkout`.
- Query 1 rises within about 30 seconds of enabling `cpu_burn` and falls afterwards.
- Query 4 stays flat while `retry_storm` is active — confirming the decoupling the detector depends on is really present in the data.

Milestone 4 - The database and the schema

MILESTONE 4

TIME

The database and the schema

1 hour

You will have, at the end: TimescaleDB running in the cluster with all nine tables, the continuous aggregate, the append-only trigger, and a seeded price catalogue.

The schema was fully specified in chapter 21, so this milestone is mostly execution. What it adds is the price catalogue — the table that lets a laptop produce credible dollar figures — and the verification that the append-only trigger actually refuses a write, because a safety mechanism you have not seen refuse something is only a hypothesis.

4.1 Deploy TimescaleDB

```
deploy/platform/timescale.yaml
```

```
# deploy/platform/timescale.yaml
apiVersion: v1
kind: Secret
metadata: {name: timescale-auth, namespace: platform}
stringData:
  POSTGRES_PASSWORD: devpassword      # local only
---
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: timescale, namespace: platform}
spec:
  serviceName: timescale
  replicas: 1
  selector: {matchLabels: {app: timescale}}
  template:
    metadata: {labels: {app: timescale}}
    spec:
      containers:
        - name: db
          image: timescale/timescaledb:2.15.2-pg16
          env:
            - name: POSTGRES_DB
              value: hypertrace
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef: {name: timescale-auth, key: POSTGRES_PASSWORD}
            # 256MB is right for a laptop; the default assumes a real server.
            - name: POSTGRES_SHARED_BUFFERS
              value: 256MB
          ports: [{containerPort: 5432}]
          resources:
            requests: {cpu: 200m, memory: 512Mi}
            limits: {cpu: "1", memory: 1Gi}
          volumeMounts:
            - {name: data, mountPath: /var/lib/postgresql/data}
      volumeClaimTemplates:
        - metadata: {name: data}
          spec:
            accessModes: [ReadWriteOnce]
            resources: {requests: {storage: 8Gi}}
---
apiVersion: v1
kind: Service
metadata: {name: timescale, namespace: platform}
spec:
  selector: {app: timescale}
  ports: [{port: 5432}]
```

4.2 Apply the schema

Save the five SQL files from chapter 21 into `schema/` in the order shown there, then add the price catalogue below. Order matters: the hypertables reference `services`, so dimensions must exist first.

```
schema/07_prices.sql - the catalogue and the seed data
```

```
-- schema/07_prices.sql
-- The simulated us-east-1 on-demand rates used throughout Part III.
-- price_set lets you swap catalogues without rewriting history, which is
-- what makes the --price-asof replay of chapter 19.4 possible.
CREATE TABLE price_catalogue (
  price_set TEXT NOT NULL,
  dimension TEXT NOT NULL,      -- vcpu_hour, gib_hour, gb_egress, ...
  unit TEXT NOT NULL,
  usd NUMERIC(12,6) NOT NULL,
  valid_from TIMESTAMPTZ NOT NULL DEFAULT '2026-01-01',
  PRIMARY KEY (price_set, dimension, valid_from)
);

INSERT INTO price_catalogue (price_set, dimension, unit, usd) VALUES
('sim-us-east-1', 'vcpu_hour', 'vCPU-hour', 0.029500),
('sim-us-east-1', 'gib_hour', 'GiB-hour', 0.003950),
('sim-us-east-1', 'gb_egress', 'GB', 0.090000),
('sim-us-east-1', 'gib_month_gp3', 'GiB-month', 0.080000),
('sim-us-east-1', 'control_plane', 'hour', 0.100000),
('sim-us-east-1', 'load_balancer', 'hour', 0.050000);

-- Two services and one team, so there is something to attribute cost to.
INSERT INTO teams (id, org_id, name) VALUES (1, 1, 'checkout');
INSERT INTO services
(id, org_id, team_id, namespace, workload, tag, timezone, requests_cpu)
VALUES
('svc_checkout', 1, 1, 'shop', 'checkout-api', 'critical',
 'Asia/Kolkata', 1.000),
('svc_search', 1, 1, 'shop', 'search-api', 'production',
 'Asia/Kolkata', 0.600),
('svc_images', 1, 1, 'shop', 'image-worker', 'dev',
 'Asia/Kolkata', 0.400);

-- One user to log in with. Hash generated by scripts/hashpw.py, not by hand.
INSERT INTO users (org_id, email, role, pwd_hash) VALUES
(1, 'admin@example.com', 'admin', '$2b$12$REPLACE_WITH_YOUR_OWN_HASH');
```

```
terminal

kubectl apply -f deploy/platform/timescale.yaml
kubectl -n platform rollout status statefulset timescale --timeout=180s

# Load every schema file in order. Note the sort: 01, 02, ... 07.
for f in $(ls schema/*.sql | sort); do
  echo "--- applying $f"
  kubectl -n platform exec -i statefulset/timescale -- \
    psql -U postgres -d hypertrace -v ON_ERROR_STOP=1 < "$f"
done

# Open a shell for the checks below, and keep this terminal.
kubectl -n platform exec -it statefulset/timescale -- \
  psql -U postgres -d hypertrace
```

4.3 Prove the schema does what chapter 21 claimed

the four checks that matter

```
-- 1. Are the hypertables really hypertables, and are the policies attached?
SELECT hypertable_name, num_chunks FROM timescaledb_information.hypertables;
--   raw_metrics   | 0
--   cost_events   | 0

SELECT j.proc_name, h.hypertable_name, j.schedule_interval
FROM timescaledb_information.jobs j
LEFT JOIN timescaledb_information.hypertables h
ON h.hypertable_name = j.hypertable_name;
--   policy_retention | raw_metrics | 1 day
--   policy_compression | raw_metrics | 12:00:00
--   policy_refresh_continuous_aggregate | | 00:05:00

-- 2. Does the append-only trigger actually refuse? Both must ERROR.
INSERT INTO actions_log
(idempotency_key, action_type, mode, target, evidence, result)
VALUES ('test-1', 'alert_only', 'dry_run', '{}', '{}', 'ok');

UPDATE actions_log SET result = 'tampered' WHERE idempotency_key = 'test-1';
-- ERROR: actions_log is append-only (attempted UPDATE)
DELETE FROM actions_log WHERE idempotency_key = 'test-1';
-- ERROR: actions_log is append-only (attempted DELETE)

-- 3. Does the idempotency key really stop a duplicate?
INSERT INTO actions_log
(idempotency_key, action_type, mode, target, evidence, result)
VALUES ('test-1', 'alert_only', 'dry_run', '{}', '{}', 'ok');
-- ERROR: duplicate key value violates unique constraint

-- 4. Is the price catalogue loaded?
SELECT dimension, usd FROM price_catalogue WHERE price_set = 'sim-us-east-1';
```

ERRORS YOU WANT TO SEE

Those two **ERROR** lines are a passing test, not a problem. Seeing the database refuse to let you tamper with the audit log is the moment NFR-5 stops being a sentence in a requirements document and becomes a property of the system. If either statement succeeds, your triggers did not load — re-apply `schema/05_audit.sql` and read the output for a syntax error.

CHECKPOINT - VERIFY BEFORE MOVING ON

- `\dt` in psql lists all nine tables plus `price_catalogue`.
- Both the **UPDATE** and the **DELETE** on `actions_log` fail with the append-only message.
- The duplicate `idempotency_key` insert fails with a unique-constraint violation.
- `SELECT * FROM services` returns three rows.
- `SELECT * FROM timescaledb_information.continuous_aggregates` shows `cost_hourly`.

Milestone 5 - The event bus

MILESTONE 5

TIME

The event bus

40 minutes

You will have, at the end: a single-broker Redpanda speaking the Kafka protocol, with the six topics of chapter 20 created and a message proved to survive a consumer restart.

Redpanda is used rather than Apache Kafka for one practical reason: it is a single binary with no ZooKeeper and roughly a fifth of the memory footprint, and it speaks the identical Kafka protocol. Every line of client code in this build would work unchanged against a real Kafka cluster, so nothing is being learned that does not transfer.


```

deploy/platform/redpanda.yaml

# deploy/platform/redpanda.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: redpanda, namespace: platform}
spec:
  serviceName: redpanda
  replicas: 1
  selector: {matchLabels: {app: redpanda}}
  template:
    metadata: {labels: {app: redpanda}}
    spec:
      containers:
        - name: redpanda
          image: redpandadata/redpanda:v24.1.11
          args:
            - redpanda
            - start
            - --smp=1 # one core; do not let it grab more
            - --memory=900M
            - --reserve-memory=0M
            - --overprovisioned # expect to share the CPU
            - --node-id=0
            - --check=false
            - --kafka-addr=PLAINTEXT://0.0.0.0:9092
            - --advertise-kafka-addr=PLAINTEXT://redpanda.platform.svc.cluster.local:9092
          ports:
            - {containerPort: 9092, name: kafka}
            - {containerPort: 9644, name: admin}
          resources:
            requests: {cpu: 300m, memory: 768Mi}
            limits: {cpu: "1", memory: 1200Mi}
          volumeMounts:
            - {name: data, mountPath: /var/lib/redpanda/data}
      volumeClaimTemplates:
        - metadata: {name: data}
          spec:
            accessModes: [ReadWriteOnce]
            resources: {requests: {storage: 4Gi}}
---
apiVersion: v1
kind: Service
metadata: {name: redpanda, namespace: platform}
spec:
  selector: {app: redpanda}
  ports:
    - {port: 9092, name: kafka}
    - {port: 9644, name: admin}

```

The `--overprovisioned` flag is not optional on a laptop. Without it Redpanda assumes it owns the machine and busy-polls a core, which on a three-node kind cluster means your fans run continuously and everything else gets slower. With it, the broker yields when idle.

5.2 Create the topics

```
terminal

RPK="kubectl -n platform exec -i statefulset/redpanda -- rpk"

# Partition counts follow chapter 20: partition by service_id so that all
# samples for one service land in order on one partition.
$RPK topic create metrics.raw -p 6 -r 1 -c retention.ms=86400000
$RPK topic create events.k8s -p 3 -r 1 -c retention.ms=604800000
$RPK topic create cost.events -p 6 -r 1 -c retention.ms=604800000
$RPK topic create anomalies -p 3 -r 1 -c retention.ms=2592000000
$RPK topic create decisions -p 3 -r 1 -c retention.ms=2592000000
$RPK topic create actions -p 1 -r 1 -c retention.ms=7776000000

$RPK topic list
# NAME PARTITIONS REPLICAS
# actions 1 1
# anomalies 3 1
# cost.events 6 1
# decisions 3 1
# events.k8s 3 1
# metrics.raw 6 1
```

One partition for `actions` is deliberate and worth understanding. Actions are the only stream where global ordering matters — a throttle followed by its rollback must never be observed in the reverse order — and a single partition is the simplest way to guarantee that. It caps throughput at one partition's worth, which is irrelevant when actions are rare by construction. This is the asymmetry chapter 20 described: ordering is bought where it matters and paid for nowhere else.

5.3 The shared bus client

```
services/common/bus.py
```

```
# services/common/bus.py
import json
import os
from typing import AsyncIterator

from aiokafka import AIOKafkaConsumer, AIOKafkaProducer

BROKER = os.getenv("BUS_BROKERS", "redpanda.platform.svc.cluster.local:9092")

async def producer() -> AIOKafkaProducer:
    p = AIOKafkaProducer(
        bootstrap_servers=BROKER,
        value_serializer=lambda v: json.dumps(v).encode(),
        key_serializer=lambda k: k.encode() if k else None,
        acks="all",           # durability over latency; see chapter 20
        linger_ms=20,         # batch for 20ms - a large throughput win
        compression_type="lz4",
    )
    await p.start()
    return p

async def consume(topic: str, group: str) -> AsyncIterator[dict]:
    """Yield decoded messages. Offsets are committed MANUALLY by the
    caller, because at-least-once plus idempotent effects (chapter 20)
    requires that the offset move only after the effect is durable."""
    c = AIOKafkaConsumer(
        topic,
        bootstrap_servers=BROKER,
        group_id=group,
        value_deserializer=lambda v: json.loads(v.decode()),
        enable_auto_commit=False,
        auto_offset_reset="latest",
        max_poll_records=500,
    )
    await c.start()
    try:
        async for msg in c:
            yield {"value": msg.value, "key": msg.key,
                  "partition": msg.partition, "offset": msg.offset,
                  "commit": c.commit}
    finally:
        await c.stop()
```

WHY AUTO-COMMIT IS OFF

`enable_auto_commit=False` is the single most consequential line in this file. With auto-commit, the client moves the offset on a timer, so a crash between the commit and your database write loses the message permanently — and the loss is silent. With manual commit, a crash means the message is redelivered, and because chapter 20 made every effect idempotent, redelivery is harmless. Choose redelivery over loss every time in this system.

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ `rpkg topic list` shows all six topics with the partition counts above.
- ▮ `rpkg topic produce metrics.raw` accepts a line of text and `rpkg topic consume metrics.raw -o start -n 1` reads it back.
- ▮ Deleting the Redpanda pod and letting it restart does not lose the message — this is the durability the replay story depends on.
- ▮ `pip install aiokafka==0.10.0` succeeds in your virtualenv.

Milestone 6 - The collector

MILESTONE 6

The collector

TIME

1.5 hours

You will have, at the end: a service that queries Prometheus every ten seconds and publishes normalised samples to metrics.raw, keyed by service_id.

This is the first component you write from scratch, and it is deliberately the dullest: it queries, it normalises, it publishes, and it holds no state. Chapter 18 called the collector a read-only component with no authority, and keeping it boring is what makes the rest of the system reasonable about failure — when the collector dies, nothing but observation is lost.

6.1 Configuration and the shared model

services/common/config.py

```
# services/common/config.py
import os
from dataclasses import dataclass

@dataclass(frozen=True)
class Settings:
    prom_url: str = os.getenv(
        "PROM_URL", "http://kps-prometheus.platform.svc.cluster.local:9090")
    pg_dsn: str = os.getenv(
        "PG_DSN",
        "postgresql://postgres:devpassword@timescale.platform:5432/hypertrace")
    brokers: str = os.getenv("BUS_BROKERS", "redpanda.platform:9092")
    interval_s: int = int(os.getenv("COLLECT_INTERVAL_S", "10"))
    rate_window: str = os.getenv("RATE_WINDOW", "2m")
    price_set: str = os.getenv("PRICE_SET", "sim-us-east-1")
    namespaces: tuple = tuple(os.getenv("NAMESPACES", "shop").split(","))

S = Settings()
```

services/collector/main.py - part 1: the queries

```

# services/collector/main.py
import asyncio
import time
from datetime import datetime, timezone

import httpx

from ..common.bus import producer
from ..common.config import S

NS = "|".join(S.namespaces)
W = S.rate_window

# One PromQL query per metric. Keys become field names in the message.
QUERIES = {
    "cpu_cores":
        f'sum by (service_id) (rate('
        f'container_cpu_usage_seconds_total{{namespace=~"{NS}"}}[{{W}}])',
    "mem_gib":
        f'sum by (service_id) ('
        f'container_memory_working_set_bytes{{namespace=~"{NS}"}}'
        f' / 1073741824',
    "net_out_bps":
        f'sum by (service_id) (rate('
        f'container_network_transmit_bytes_total{{namespace=~"{NS}"}}[{{W}}])',
    "net_in_bps":
        f'sum by (service_id) (rate('
        f'container_network_receive_bytes_total{{namespace=~"{NS}"}}[{{W}}])',
    "http_rps":
        f'sum by (service_id) (rate('
        f'app_requests_total{{namespace=~"{NS}"}}[{{W}}])',
    "requests_cpu":
        f'sum by (service_id) (kube_pod_container_resource_requests'
        f'{{namespace=~"{NS}"}, resource="cpu"})',
    "replicas":
        f'sum by (service_id) ('
        f'kube_deployment_status_replicas{{namespace=~"{NS}"}})',
}

async def query_one(client: httpx.AsyncClient, expr: str) -> dict[str, float]:
    """Run one instant query. Returns {service_id: value}.
```

services/collector/main.py - part 1: the queries (continued)

A failed query returns {} rather than raising: one broken metric must not stop the other six from being collected.

```

"""
try:
    r = await client.get(f"{S.prom_url}/api/v1/query",
                        params={"query": expr, "timeout": 5.0})
    r.raise_for_status()
    body = r.json()
    if body.get("status") != "success":
        return {}
    out = {}
    for series in body["data"]["result"]:
        sid = series["metric"].get("service_id")
        if sid:
            out[sid] = float(series["value"][1])
    return out
except (httpx.HTTPError, KeyError, ValueError):
    return {}
```

```

services/collector/main.py - part 2: the cycle

# services/collector/main.py (continued)

async def collect_once(client: httpx.AsyncClient, prod) -> int:
    """One collection cycle. Returns the number of samples published."""
    # All seven queries in parallel: 7 sequential 60ms calls would be 420ms,
    # which is 4% of the whole latency budget spent on waiting.
    names = list(QUERIES)
    results = await asyncio.gather(
        *[query_one(client, QUERIES[n]) for n in names])
    by_metric = dict(zip(names, results))

    services = set()
    for vals in by_metric.values():
        services.update(vals)

    ts = datetime.now(timezone.utc).isoformat()
    sent = 0
    for sid in sorted(services):
        # A service missing from a query is simply absent, never zero.
        # Zero-filling would teach the baseline that idle is normal - 23.4.
        msg = {"schema": "metrics.raw/v1", "ts": ts, "service_id": sid,
              "window_s": S.interval_s}
        for metric, vals in by_metric.items():
            if sid in vals:
                msg[metric] = round(vals[sid], 6)
        if "cpu_cores" not in msg:
            continue          # no CPU reading means no usable sample
        # Key by service_id: guarantees per-service ordering on one partition.
        await prod.send("metrics.raw", value=msg, key=sid)
        sent += 1
    return sent

async def main() -> None:
    prod = await producer()
    async with httpx.AsyncClient() as client:
        while True:
            started = time.perf_counter()
            try:
                n = await collect_once(client, prod)
                print(f"published {n} samples", flush=True)
            except Exception as exc:
                # never die on one cycle
                print(f"cycle failed: {exc!r}", flush=True)
            # Sleep the REMAINDER of the interval, so a slow cycle does not
            # push every later cycle later. Drift accumulates otherwise.
            elapsed = time.perf_counter() - started
            await asyncio.sleep(max(0.0, S.interval_s - elapsed))

if __name__ == "__main__":
    asyncio.run(main())

```

Three details in that loop are worth naming because each is a bug you would otherwise write. Sleeping the remainder rather than the full interval stops the collection cadence from drifting later and later under load. Catching every exception around the cycle rather than around the whole loop means one bad cycle costs one cycle, not the service. And skipping a service with no CPU reading rather than defaulting it to zero is the concrete implementation of the no-zero-fill rule from chapter 23.4.

6.3 Run it, in two modes

terminal

```
# Local first: port-forward Prometheus and the broker, run on the host.
kubectl -n platform port-forward svc/kps-prometheus 9090:9090 &
kubectl -n platform port-forward svc/redpanda 9092:9092 &

export PROM_URL=http://localhost:9090
export BUS_BROKERS=localhost:9092
python -m services.collector.main
#   published 3 samples
#   published 3 samples

# Watch the actual messages in another terminal:
kubectl -n platform exec -i statefulset/redpanda -- \
  rpk topic consume metrics.raw -n 2 --format json | jq '.value | fromjson'
#   {
#     "schema": "metrics.raw/v1",
#     "ts": "2026-08-26T10:31:04.118Z",
#     "service_id": "svc_checkout",
#     "window_s": 10,
#     "cpu_cores": 0.847100,
#     "mem_gib": 1.502000,
#     "net_out_bps": 3333333.3,
#     "http_rps": 11.8,
#     "requests_cpu": 1.0,
#     "replicas": 3
#   }
```

Developing against port-forwards and only then deploying into the cluster is worth the extra step for every service in Part III. The edit-run cycle is seconds instead of the minute a Docker build, `kind load` and rollout costs, and the code is identical either way because every endpoint comes from an environment variable.

CHECKPOINT - VERIFY BEFORE MOVING ON

- The collector prints a non-zero sample count every ten seconds.
- `rpk topic consume metrics.raw` shows messages whose `service_id` matches the seeded services.
- `cpu_cores` in the published message rises when you enable `cpu_burn` on the victim.
- Stopping Prometheus — `kubectl -n platform scale sts prometheus-kps-prometheus --replicas=0` — makes the collector log failed cycles and keep running rather than crash. Scale it back to 1 afterwards.

Milestone 7 - The cost engine

MILESTONE 7

TIME

The cost engine

2 hours

You will have, at the end: dollar-per-hour figures on the bus and in the database, matching the worked example of chapter 22.4 to six decimal places.

This milestone implements chapter 22 exactly. When it is done you will have the project's central claim working end to end: a resource measurement taken ten seconds ago expressed as money, without waiting for a billing export.

7.1 The price cache

Rates are read once at startup and refreshed every fifteen minutes. They are not read per sample: a database round trip per sample would put a query on the hot path that chapter 19.3 explicitly excluded, and rates change perhaps twice a year.

```
services/costengine/prices.py

# services/costengine/prices.py
import asyncpg

from ..common.config import S

class Prices:
    """In-memory rate card. Refreshed on a timer, never on the hot path."""

    def __init__(self) -> None:
        self._r: dict[str, float] = {}
        self.price_set = S.price_set

    async def refresh(self, pool: asyncpg.Pool) -> None:
        rows = await pool.fetch(
            """SELECT dimension, usd FROM price_catalogue
               WHERE price_set = $1 AND valid_from <= now()""",
            self.price_set)
        fresh = {r["dimension"]: float(r["usd"]) for r in rows}
        missing = {"vcpu_hour", "gib_hour", "gb_egress"} - set(fresh)
        if missing:
            # Refuse to run on a partial catalogue: silently missing rates
            # produce confidently wrong money, the worst failure in 27.3.
            raise RuntimeError(f"price catalogue incomplete: {missing}")
        self._r = fresh

    def rate(self, dimension: str) -> float:
        return self._r[dimension]
```

7.2 The calculation

```
services/costengine/calc.py
```



```
# services/costengine/calc.py
from dataclasses import dataclass, asdict

HOURS_PER_MONTH = 730.0
MIN_RPH_FOR_UNIT_COST = 100.0

@dataclass
class CostBreakdown:
    cpu: float
    memory: float
    egress: float
    storage: float
    shared: float
    total: float
    usd_per_1k_requests: float | None

    def as_message(self) -> dict:
        return {k: (round(v, 6) if isinstance(v, float) else v)
                for k, v in asdict(self).items()}

def compute(sample: dict, prices, ctx: dict) -> CostBreakdown:
    """Chapter 22, verbatim.

    sample: one metrics.raw message.
    ctx:    {'provisioned_gib', 'shared_pool_usd_hr', 'cluster_requested_cpu',
            'requests_cpu'} - slow-moving context, refreshed per minute.
    """
    cpu = sample["cpu_cores"] * prices.rate("vcpu_hour")
    memory = sample.get("mem_gib", 0.0) * prices.rate("gib_hour")

    # bytes/second -> GB/hour. 3600 seconds, 1e9 bytes per billing GB.
    # Providers bill egress in decimal GB, not GiB. Using 2**30 here is a
    # 7% error and it is a very common one.
    egress_gb_hr = sample.get("net_out_bps", 0.0) * 3600.0 / 1e9
    egress = egress_gb_hr * prices.rate("gb_egress")

    storage = (ctx.get("provisioned_gib", 0.0)
               * prices.rate("gib_month_gp3") / HOURS_PER_MONTH)

    # Shared cost by REQUESTED compute - chapter 22.2, chosen strategy.
    denom = ctx.get("cluster_requested_cpu", 0.0)
    share = (ctx.get("requests_cpu", 0.0) / denom) if denom > 0 else 0.0
    shared = ctx.get("shared_pool_usd_hr", 0.0) * share

    total = cpu + memory + egress + storage + shared

    rph = sample.get("http_rps", 0.0) * 3600.0
    unit = (total / (rph / 1000.0)) if rph >= MIN_RPH_FOR_UNIT_COST else None

    return CostBreakdown(cpu, memory, egress, storage, shared, total, unit)
```

GB AND GIB ARE DIFFERENT, AND IT MATTERS HERE

The `1e9` on the egress line is not a rounding preference, it is correctness. Cloud providers bill network transfer in decimal gigabytes while container runtimes report bytes, and dividing by `2**30` because “a gigabyte is 1024 megabytes” understates every egress charge by about seven percent. On the worked example that is four cents an hour on one pod, which compounds into reconciliation drift you will spend a day chasing.

7.3 The test that proves it

Write this test before running the service. It encodes the worked example from chapter 22.4, so a passing test means your arithmetic agrees with the specification to six decimal places — and if you later change the calculation, it tells you immediately.

```
tests/test_cost.py
```

```
# tests/test_cost.py
from services.costengine.calc import compute

class FakePrices:
    R = {"vcpu_hour": 0.0295, "gib_hour": 0.00395,
         "gb_egress": 0.09, "gib_month_gp3": 0.08}

    def rate(self, d):
        return self.R[d]

def test_worked_example_from_chapter_22():
    sample = {"cpu_cores": 0.8471, "mem_gib": 1.502,
              "net_out_bps": 3333333.3, "http_rps": 11.8}
    ctx = {"provisioned_gib": 20.0, "shared_pool_usd_hr": 0.42,
           "cluster_requested_cpu": 5.4, "requests_cpu": 1.0}

    b = compute(sample, FakePrices(), ctx)

    assert round(b.cpu, 6) == 0.024990
    assert round(b.memory, 6) == 0.005933
    assert round(b.egress, 6) == 1.080000
    assert round(b.storage, 6) == 0.002192
    assert round(b.shared, 6) == 0.077778
    assert round(b.total, 6) == 1.190893
    assert round(b.usd_per_1k_requests, 6) == 0.028035

def test_idle_service_reports_null_unit_cost_not_a_huge_number():
    b = compute({"cpu_cores": 0.01, "http_rps": 0.001},
                FakePrices(), {})
    assert b.total > 0 # idle still costs money
    assert b.usd_per_1k_requests is None # but unit cost is undefined
```

```
terminal
```

```
pytest tests/test_cost.py -v
# test_worked_example_from_chapter_22 PASSED
# test_idle_service_reports_null_unit_cost_not_a_huge_number PASSED
# 2 passed in 0.04s
```

7.4 The service loop

```
services/costengine/main.py
```

```

# services/costengine/main.py
import asyncio
from datetime import datetime, timezone

import asyncpg

from ..common.bus import consume, producer
from ..common.config import S
from .calc import compute
from .context import ContextCache      # provisioned storage, shared pool
from .prices import Prices

UPSERT = """
INSERT INTO cost_events (ts, window_start, resource_uid, service_id,
                        price_set, cpu_usd_hr, mem_usd_hr, egress_usd_hr, storage_usd_hr,
                        shared_usd_hr, total_usd_hr, requests, is_estimate)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,TRUE)
ON CONFLICT (resource_uid, window_start) DO UPDATE SET
    cpu_usd_hr=$6, mem_usd_hr=$7, egress_usd_hr=$8, storage_usd_hr=$9,
    shared_usd_hr=$10, total_usd_hr=$11, requests=$12
"""

async def main() -> None:
    pool = await asyncpg.create_pool(S.pg_dsn, min_size=2, max_size=8)
    prices, ctx = Prices(), ContextCache()
    await prices.refresh(pool)
    await ctx.refresh(pool)

    async def refresher():
        while True:
            await asyncio.sleep(60)
            try:
                await ctx.refresh(pool)
            except Exception as exc:
                print(f"context refresh failed: {exc!r}", flush=True)

    asyncio.create_task(refresher())
    prod = await producer()

    async for m in consume("metrics.raw", group="cost-engine"):
        sample = m["value"]
        sid = sample["service_id"]
        b = compute(sample, prices, ctx.for_service(sid))

        ts = datetime.fromisoformat(sample["ts"])
        out = {"schema": "cost.events/v1", "ts": sample["ts"],
              "service_id": sid, "price_set": prices.price_set,
              "window_s": sample["window_s"],
              "requests": int(sample.get("http_rps", 0.0)
                              * sample["window_s"]),
              "breakdown": b.as_message()}


```

services/costengine/main.py (continued)

```

# Write first, publish second, commit the offset last. On a crash
# the message is redelivered and the upsert absorbs it - 20.5.
await pool.execute(
    UPSERT, ts, ts, ctx.uid(sid), sid, prices.price_set,
    b.cpu, b.memory, b.egress, b.storage, b.shared, b.total,
    out["requests"])
await prod.send("cost.events", value=out, key=sid)
await m["commit"]()

if __name__ == "__main__":
    asyncio.run(main())

```

The ordering of those last three statements is the whole of the delivery-guarantee design in three lines: durable write, then publish, then commit. Reverse any two and you get either lost cost events or double-published ones. Because the write is an upsert on (`resource_uid`, `window_start`), a redelivered message overwrites its own row with the same values and changes nothing — which is what chapter 20 meant by making the effect idempotent so the delivery guarantee stops mattering.

```
terminal

export PG_DSN='postgresql://postgres:devpassword@localhost:5432/hypertrace'
kubectl -n platform port-forward svc/timescale 5432:5432 &
python -m services.costengine.main &

# After a minute, ask the database what things cost:
psql "$PG_DSN" -c "
    SELECT service_id,
           round(avg(total_usd_hr), 4) AS avg_usd_hr,
           round(avg(egress_usd_hr), 4) AS egress_usd_hr,
           count(*) AS windows
    FROM cost_events
    WHERE ts > now() - interval '5 minutes'
    GROUP BY service_id ORDER BY 2 DESC;"
#  service_id | avg_usd_hr | egress_usd_hr | windows
#  -----+-----+-----+-----
#  svc_checkout |    0.0361 |         0.0004 |        30
#  svc_search  |    0.0208 |         0.0002 |        18
```

YOU NOW HAVE REAL-TIME COST

Those are real dollar figures for workloads running on your own laptop, computed seconds after the resources were used. They are small because a kind cluster is small; the arithmetic and the pipeline are identical to what a production deployment would run. Turn on `cpu_burn` and re-run the query — watching `avg_usd_hr` climb within one interval is the demonstration that a billing dashboard cannot give you until tomorrow.

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ `pytest tests/test_cost.py` passes both tests.
- ▮ `cost.events` on the bus carries a `breakdown` object with six components and a total.
- ▮ `SELECT count(*) FROM cost_events` grows by roughly three rows every ten seconds.
- ▮ Enabling `cpu_burn` raises `cpu_usd_hr` for `svc_checkout` within two intervals.
- ▮ Killing the cost engine mid-run and restarting it produces no duplicate rows — the upsert absorbs the replay.
- ▮ `SELECT * FROM cost_hourly` returns rows after the aggregate's first five-minute refresh.

Milestone 8 - The behaviour engine

MILESTONE 8

TIME

The behaviour engine

3 hours

You will have, at the end: rolling baselines in memory, Z-scores per hour-of-week bucket, and an anomaly published only when cost and CPU move together while traffic does not.

This is the component that decides whether a number is unusual, and it is where the project earns or loses its credibility. Everything in chapter 23 is implemented here: the EWMA baseline, the 168 hour-of-week buckets, the joint decoupling condition, and the confidence score that determines how much authority the decision downstream is allowed to exercise.

8.1 The baseline store

One `Baseline` object per service, per metric, per hour-of-week bucket. For three services, four metrics and 168 buckets that is 2,016 objects of about 80 bytes — well under a megabyte, which is why chapter 29 could claim the whole engine fits in a 512 MiB container.

```

services/behaviour/baseline.py

# services/behaviour/baseline.py
import math
from dataclasses import dataclass, field
from datetime import datetime
from zoneinfo import ZoneInfo

ALPHA_DEFAULT = 1.0 / 720.0      # ~2 weeks of memory at 10s samples
MIN_SD_FLOOR = 1e-6             # never divide by zero
WARMUP_SAMPLES = 200            # below this the bucket is not trusted

@dataclass
class Baseline:
    mean: float = 0.0
    variance: float = 0.0
    n: int = 0
    dirty: bool = False

    @property
    def sd(self) -> float:
        return max(math.sqrt(max(self.variance, 0.0)), MIN_SD_FLOOR)

    @property
    def mature(self) -> bool:
        return self.n >= WARMUP_SAMPLES

    def z(self, x: float) -> float:
        return (x - self.mean) / self.sd

    def update(self, x: float, alpha: float = ALPHA_DEFAULT) -> None:
        """Welford-style EWMA of mean AND variance in one pass.

        Note the order: variance uses the OLD mean via delta, so the two
        lines must not be swapped. Getting this backwards produces a
        variance that lags the mean and quietly inflates every Z-score.
        """
        if self.n == 0:
            self.mean, self.variance, self.n = x, 0.0, 1
            self.dirty = True
            return
        delta = x - self.mean
        self.mean += alpha * delta
        self.variance = (1.0 - alpha) * (self.variance + alpha * delta * delta)
        self.n += 1
        self.dirty = True

    def bucket_of(ts: datetime, tz: str) -> int:
        """Hour-of-week 0..167 in the SERVICE's timezone, not the server's.

        A service serving Indian shoppers has its Monday-morning peak at a
        different UTC hour than one serving Berlin. Bucketing in UTC smears
        the two together and destroys the whole point of the buckets.
        """
        local = ts.astimezone(ZoneInfo(tz))
        return local.weekday() * 24 + local.hour

```

THE ONE LINE BEGINNERS GET WRONG

The two update lines look interchangeable and are not. **variance** must be computed from **delta**, which was measured against the mean *before* it moved. Update the mean first and then recompute the delta and your variance systematically underestimates spread, every Z-score inflates, and the detector fires on ordinary traffic. Write the unit test in 8.4 before you trust it.

8.2 The detector

```
services/behaviour/detect.py

# services/behaviour/detect.py
from dataclasses import dataclass

Z_STRONG = 3.0      # cost / CPU must be this far out
Z_FLAT = 1.0        # traffic must be BELOW this to count as decoupled
MIN_ABS_USD_HR = 0.02 # ignore statistically odd but financially trivial

@dataclass
class Signal:
    cost_z: float
    cpu_z: float
    traffic_z: float
    egress_z: float
    total_usd_hr: float
    mature: bool
    corroborated: bool    # 2+ metrics agree

def classify(sig: Signal) -> str | None:
    """Return an anomaly kind, or None. Order matters: the most specific
    pattern is tested first so a runaway loop is not reported as a
    generic cost spike."""
    if sig.total_usd_hr < MIN_ABS_USD_HR:
        return None

    decoupled = sig.traffic_z < Z_FLAT

    # The signature the whole project is built around: burning more
    # compute and more money while serving no more requests.
    if sig.cpu_z > Z_STRONG and sig.cost_z > Z_STRONG and decoupled:
        return "runaway_compute"

    # Egress is the expensive one; a smaller threshold is justified
    # because $0.09/GB makes the financial signal strong at lower Z.
    if sig.egress_z > 2.5 and decoupled:
        return "egress_anomaly"

    # Cost moved but compute did not: usually storage or a price change.
    if sig.cost_z > Z_STRONG and sig.cpu_z < Z_FLAT:
        return "cost_only_drift"

    # Everything moved together, including traffic. This is SUCCESS,
    # not an incident, and reporting it is how tools lose trust.
    return None

def confidence(sig: Signal) -> float:
    """Chapter 24.3. Bounded 0..1; each term is independently capped so
    one enormous Z-score cannot buy authority on its own."""
    magnitude = min(max(sig.cost_z, sig.cpu_z) / 6.0, 1.0)
    decoupling = min(max(0.0, Z_FLAT - sig.traffic_z) / 2.0, 1.0)
    corroboration = 1.0 if sig.corroborated else 0.0
    maturity = 1.0 if sig.mature else 0.0
    return round(0.40 * magnitude + 0.30 * decoupling
        + 0.20 * corroboration + 0.10 * maturity, 4)
```

Read the last branch again, because it is the most important line in the file and it does nothing. When cost, CPU and traffic all rise together the service is simply busy, and a tool that pages someone for a successful marketing campaign will be muted within a week. Chapter 23.2 argued that the value of the decoupling test is mostly in what it refuses to report, and this is where that refusal lives.

8.3 The engine loop

```

services/behaviour/main.py

# services/behaviour/main.py
import asyncio
import uuid
from collections import defaultdict
from datetime import datetime

import asyncpg

from ..common.bus import consume, producer
from ..common.config import S
from .baseline import Baseline, bucket_of
from .detect import Signal, classify, confidence

METRICS = ("total_usd_hr", "cpu_cores", "http_rps", "egress_usd_hr")

# state[(service_id, metric, bucket)] -> Baseline
state: dict[tuple[str, str, int], Baseline] = defaultdict(Baseline)
tz_of: dict[str, str] = {}

def observe(sid: str, metric: str, bkt: int, x: float) -> tuple[float, bool]:
    """Score the sample against its bucket, THEN fold it in.

    Scoring before updating matters: fold first and a large value has
    already pulled its own baseline towards itself, shrinking the very
    Z-score you are about to measure.
    """
    b = state[(sid, metric, bkt)]
    z = b.z(x) if b.n > 0 else 0.0
    mature = b.mature
    b.update(x)
    return z, mature

async def main() -> None:
    pool = await asyncpg.create_pool(S.pg_dsn, min_size=2, max_size=6)
    for r in await pool.fetch("SELECT id, timezone FROM services"):
        tz_of[r["id"]] = r["timezone"]
    await load_baselines(pool, state)      # survive restarts - 23.5
    asyncio.create_task(checkpointer(pool, state))
    prod = await producer()

    async for m in consume("cost.events", group="behaviour"):
        ev = m["value"]
        sid = ev["service_id"]
        ts = datetime.fromisoformat(ev["ts"])
        bkt = bucket_of(ts, tz_of.get(sid, "UTC"))
        bd = ev["breakdown"]

        vals = {"total_usd_hr": bd["total"],
                "egress_usd_hr": bd["egress"],
                "cpu_cores": ev.get("cpu_cores", 0.0),
                "http_rps": ev["requests"] / max(ev["window_s"], 1)}

```

```

services/behaviour/main.py (continued)

```

```

zs, mature = {}, True
for metric in METRICS:
    z, mat = observe(sid, metric, bkt, vals[metric])
    zs[metric] = z
    mature = mature and mat

sig = Signal(
    cost_z=zs["total_usd_hr"], cpu_z=zs["cpu_cores"],
    traffic_z=zs["http_rps"], egress_z=zs["egress_usd_hr"],
    total_usd_hr=bd["total"], mature=mature,
    corroborated=sum(1 for v in zs.values() if v > 2.0) >= 2)

kind = classify(sig)
if kind is None:
    await m["commit"]()
    continue

conf = confidence(sig)
out = {"schema": "anomalies/v1",
      "anomaly_id": str(uuid.uuid4()),
      "ts": ev["ts"], "service_id": sid, "kind": kind,
      "confidence": conf, "bucket": bkt,
      "z": {k: round(v, 3) for k, v in zs.items()},
      "observed": {k: round(v, 6) for k, v in vals.items()},
      "baseline": {k: round(state[(sid, k, bkt)].mean, 6)
                   for k in METRICS},
      "evidence_url": f"/anomaly/{sid}/{ev['ts']}"}
await prod.send("anomalies", value=out, key=sid)
print(f"{kind} {sid} conf={conf}", flush=True)
await m["commit"]()

```

The anomaly message carries the observed values, the baseline it was compared against and every Z-score, not just a verdict. Chapter 26.4 required that a human be able to audit a decision without querying the database, and an explanation assembled at detection time is the only kind that is guaranteed still to be true later — the baseline it was measured against has already moved on by the time anyone looks.

8.4 The tests you must not skip

```
tests/test_behaviour.py
```



```

# tests/test_behaviour.py
from services.behaviour.baseline import Baseline
from services.behaviour.detect import Signal, classify, confidence

def test_baseline_converges_on_a_constant_stream():
    b = Baseline()
    for _ in range(5000):
        b.update(1.0)
    assert abs(b.mean - 1.0) < 0.01
    assert b.sd < 0.01          # no spread in a constant stream

def test_baseline_variance_tracks_a_known_spread():
    import random
    random.seed(7)
    b = Baseline()
    for _ in range(20000):
        b.update(random.gauss(10.0, 2.0))
    assert 9.5 < b.mean < 10.5
    assert 1.5 < b.sd < 2.5      # recovers sigma=2, not 0 and not 8

def _sig(**kw):
    base = dict(cost_z=0, cpu_z=0, traffic_z=0, egress_z=0,
                total_usd_hr=1.0, mature=True, corroborated=True)
    return Signal(**(base | kw))

def test_runaway_is_detected():
    assert classify(_sig(cpu_z=5, cost_z=5, traffic_z=0.2)) \
        == "runaway_compute"

def test_a_real_traffic_spike_is_NOT_an_anomaly():
    # This is the false positive that would destroy trust: everything up,
    # including traffic. The system must stay silent.
    assert classify(_sig(cpu_z=6, cost_z=6, traffic_z=5.5)) is None

def test_trivial_amounts_are_ignored():
    assert classify(_sig(cpu_z=9, cost_z=9, total_usd_hr=0.001)) is None

def test_immature_baseline_cannot_reach_autonomous_confidence():
    hot = _sig(cpu_z=9, cost_z=9, traffic_z=-1, mature=False)
    assert confidence(hot) < 0.85    # so it can only ever recommend

def test_mature_and_extreme_does_reach_autonomous():
    hot = _sig(cpu_z=9, cost_z=9, traffic_z=-1, mature=True)
    assert confidence(hot) >= 0.85

```

A TEST THAT GUARDS A SAFETY PROPERTY

The last two tests encode the safety property from chapter 26.2 as executable code: an immature baseline can never produce autonomous confidence, no matter how extreme the reading. If someone later “improves” the confidence function and drops the maturity term, this test fails and the review catches it. That is the difference between a documented safeguard and an enforced one.

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ `pytest tests/test_behaviour.py -v` — all seven pass.
- ▮ With the victim idle, the engine publishes nothing to **anomalies** for several minutes.
- ▮ Enabling **cpu_burn** produces a **runaway_compute** anomaly within about two minutes of baseline warm-up.
- ▮ Ramping real load with `hey -z 60s` produces no anomaly — the decoupling test correctly stays silent.
- ▮ Restarting the engine does not reset the baselines: `SELECT count(*) FROM baselines` is non-zero and **n** keeps climbing.

Milestone 9 - The policy engine

MILESTONE 9

TIME

The policy engine**2 hours**

You will have, at the end: a decision on the bus for every anomaly, carrying the rule that produced it, the authority granted, and a plain-English reason.

The policy engine converts an anomaly into a decision. It is intentionally the smallest component in the system and the one with the strictest tests, because it is the only place where the question “is this allowed?” is answered. Chapter 25 specified the language; this milestone implements the evaluator and the four conflict-resolution rules.

`services/policy/engine.py - the evaluator`

```
# services/policy/engine.py
from datetime import datetime
from zoneinfo import ZoneInfo

import yaml

LADDER = ["alert", "throttle", "freeze_scaling", "quarantine", "terminate"]

class Policy:
    def __init__(self, docs: list[dict]) -> None:
        self.docs = sorted(docs, key=lambda d: -d.get("priority", 0))

    @classmethod
    def load(cls, paths) -> "Policy":
        out = []
        for pth in paths:
            with open(pth) as fh:
                out.extend(d for d in yaml.safe_load_all(fh) if d)
        return cls(out)

    def _matches(self, doc: dict, svc: dict) -> bool:
        sel = doc.get("selector", {})
        return all(svc.get(k) == v for k, v in sel.items())

    def decide(self, anomaly: dict, svc: dict, now: datetime) -> dict:
        """Return a decision. NEVER raises: a policy that cannot be
        evaluated must degrade to alert-only, not to silence and not to
        an unguarded action (chapter 27.2)."""
        verdict = {"action": "alert", "mode": "alert_only",
                   "rule": "default", "reason": "no policy matched"}
        try:
            for doc in self.docs:
                if not self._matches(doc, svc):
                    continue
                for rule in doc.get("rules", []):
                    if self._when(rule, anomaly, svc, now):
                        verdict = self._apply(doc, rule, anomaly)
                        # Prohibitions are collected from EVERY matching
                        # document, not only the winning one - 25.4 rule 1.
                        return self._prohibit(verdict, svc)
        except Exception as exc:
            verdict = {"action": "alert", "mode": "alert_only",
                       "rule": "error", "reason": f"policy error: {exc!r}"}
        return verdict
```

services/policy/engine.py - the evaluator (continued)

```
def _prohibit(self, verdict: dict, svc: dict) -> dict:
    banned = set()
    for doc in self.docs:
        if self._matches(doc, svc):
            banned.update(doc.get("never", []))
    if verdict["action"] in banned:
        # Do not silently escalate or silently drop: step DOWN the
        # ladder to the strongest permitted rung, and say so.
        idx = LADDER.index(verdict["action"])
        for cand in reversed(LADDER[:idx]):
            if cand not in banned:
                verdict["reason"] += (
                    f" (downgraded from {verdict['action']}):"
                    f" prohibited for this service")
                verdict["action"] = cand
                return verdict
    verdict["action"], verdict["mode"] = "alert", "alert_only"
    return verdict
```

Two behaviours in that class are load-bearing. The bare `except` around the whole evaluation is not laziness: chapter 27.2 required that a malformed policy file degrade to alert-only rather than either crashing the pipeline or, far worse, falling through to an unguarded action. And prohibitions are gathered from every matching document rather than only the highest-priority one, so a team-wide `never: [terminate]` cannot be overridden by a more specific rule that happens to sort first.

9.2 The authority gate

```
services/policy/authority.py

# services/policy/authority.py
AUTONOMOUS_MIN = 0.85
RECOMMEND_MIN = 0.60

def mode_for(confidence: float, trust_stage: int, action: str,
             policy_mode: str) -> str:
    """Resolve the final execution mode. The rule is 'most restrictive
    wins' across four independent limits, so adding a fifth later can
    only ever reduce authority."""
    if confidence < RECOMMEND_MIN:
        return "alert_only"
    if confidence < AUTONOMOUS_MIN:
        return "recommend"
    if policy_mode in ("alert_only", "recommend"):
        return policy_mode          # policy caps authority
    allowed = {0: [], 1: [], 2: ["alert"],
               3: ["alert", "throttle", "freeze_scaling"],
               4: ["alert", "throttle", "freeze_scaling",
                   "quarantine", "terminate"]}[trust_stage]
    if action not in allowed:
        return "recommend"          # trust stage caps authority
    return "autonomous"
```

9.3 Prove the gate cannot be talked around

```
tests/test_policy.py
```

```
# tests/test_policy.py
from services.policy.authority import mode_for
from services.policy.engine import Policy

DOC = {
    "selector": {"tag": "critical"},
    "priority": 10,
    "never": ["terminate", "quarantine"],
    "rules": [{"when": {"kind": "runaway_compute"},
               "do": "terminate", "mode": "autonomous"}],
}
SVC = {"tag": "critical", "id": "svc_checkout"}

def test_prohibition_beats_an_explicit_permission():
    d = Policy([DOC]).decide({"kind": "runaway_compute"}, SVC, NOW)
    assert d["action"] == "throttle"      # stepped down, not executed
    assert "prohibited" in d["reason"]

def test_a_broken_policy_file_degrades_to_alert_only():
    d = Policy([{"selector": {}, "rules": [{"when": None}]}]) \
        .decide({"kind": "x"}, SVC, NOW)
    assert d["mode"] == "alert_only"

def test_low_confidence_can_never_be_autonomous():
    for stage in range(5):
        assert mode_for(0.59, stage, "alert", "autonomous") == "alert_only"

def test_new_deployment_cannot_act_however_confident():
    assert mode_for(0.99, 0, "throttle", "autonomous") == "recommend"

def test_stage_3_cannot_terminate():
    assert mode_for(0.99, 3, "terminate", "autonomous") == "recommend"
    assert mode_for(0.99, 3, "throttle", "autonomous") == "autonomous"
```

TEST THE REFUSALS

Every one of those five tests asserts that something does *not* happen. That asymmetry is deliberate and is the honest measure of an autonomous system: the interesting tests are not the ones proving it acts, they are the ones proving it refuses to. If you write only the happy paths, you have tested the feature and left the safety property untested.

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ `pytest tests/test_policy.py -v` — all five pass.
- ▮ A `runaway_compute` anomaly on `svc_checkout` produces a `decisions` message whose `mode` is `recommend` on a fresh install, because trust starts at stage 0.
- ▮ Every decision message names the rule that produced it and carries a reason string you could paste into a ticket.
- ▮ Deleting the policy directory makes every decision `alert_only` and the pipeline keeps running.

Milestone 10 - The executor and the audit trail

MILESTONE 10

TIME

The executor and the audit trail

3 hours

You will have, at the end: a real throttle applied to a real Kubernetes deployment, recorded in an append-only log, and automatically rolled back fifteen minutes later.

This is the only component with write access to anything, and it is the one place where a bug has consequences beyond a wrong number on a dashboard. Every one of the seven rails from chapter 26.5 is implemented here, and dry-run is the default that must be explicitly overridden.

```
services/executor/rails.py

# services/executor/rails.py
from datetime import datetime, timedelta, timezone

PROTECTED_NAMESPACES = {"kube-system", "platform", "kube-public",
                        "kube-node-lease", "local-path-storage"}
MAX_ACTIONS_PER_HOUR = 3
MAX_CONCURRENT_ACTIONS = 2
BLAST_RADIUS_FRACTION = 0.25      # never touch >25% of a service at once

class Refused(Exception):
    """A rail said no. This is a normal outcome, not a bug."""

async def check_all(pool, decision: dict, svc: dict, live: dict) -> None:
    """Seven rails, cheapest first. Any one of them refusing is final -
    there is no override flag, by design (chapter 26.5)."""
    ns = svc["namespace"]

    # 1. Namespace allow-list. The platform cannot act on itself.
    if ns in PROTECTED_NAMESPACES:
        raise Refused(f"namespace {ns} is protected")
    if not live["labels"].get("ht.io/monitored") == "true":
        raise Refused(f"namespace {ns} is not opted in")
    if live["labels"].get("ht.io/no-action") == "true":
        raise Refused(f"namespace {ns} carries ht.io/no-action")

    # 2. Rate limit per service.
    since = datetime.now(timezone.utc) - timedelta(hours=1)
    n = await pool.fetchval(
        """SELECT count(*) FROM actions_log
        WHERE service_id = $1 AND mode = 'autonomous' AND ts > $2""",
        svc["id"], since)
    if n >= MAX_ACTIONS_PER_HOUR:
        raise Refused(f"{n} actions already this hour")

    # 3. Global concurrency: never fight on several fronts at once.
    busy = await pool.fetchval(
        "SELECT count(*) FROM active_actions WHERE rolled_back_at IS NULL")
    if busy >= MAX_CONCURRENT_ACTIONS:
        raise Refused(f"{busy} actions already in flight")

    # 4. Blast radius and 5. minimum capacity.
    if live["replicas"] <= svc.get("min_replicas", 2):
        raise Refused("at minimum replicas already")
    if live["ready"] / max(live["replicas"], 1) < 0.75:
        raise Refused("service already degraded; do not add pressure")

    # 6. Staleness: never act on evidence that has gone cold.
    age = (datetime.now(timezone.utc)
           - datetime.fromisoformat(decision["ts"])).total_seconds()
    if age > 120:
        raise Refused(f"evidence is {age:.0f}s old")

    # 7. Kill switch, checked last so it always wins.
    if await pool.fetchval("SELECT paused FROM platform_state WHERE id = 1"):
        raise Refused("platform is paused by an operator")
```

NEVER PUSH A FALLING SERVICE

Rail 5 is the one that separates a safety system from a hazard. If a service is already unhealthy, throttling it makes the outage worse, and a naive implementation will happily do exactly that because the cost signal looks alarming. Refusing to add pressure to something already failing is the difference between a tool that limits damage and one that amplifies it.

10.2 Applying and reverting a throttle

```
services/executor/actions.py

# services/executor/actions.py
from kubernetes_asyncio import client

async def throttle(api: client.AppsV1Api, svc: dict, factor: float = 0.6):
    """Reduce the CPU limit of the workload's container.

    Returns (undo_patch, applied_patch) so the rollback is computed from
    the state we OBSERVED, never reconstructed from assumptions later.
    """
    dep = await api.read_namespaced_deployment(svc["workload"],
                                                svc["namespace"])
    cont = dep.spec.template.spec.containers[0]
    old = dict((cont.resources.limits or {}))
    cpu_cores = _parse_cpu(old.get("cpu", "1"))
    new_cpu = f"{int(cpu_cores * factor * 1000)}m"

    patch = {"spec": {"template": {"spec": {"containers": [
        {"name": cont.name,
         "resources": {"limits": {**old, "cpu": new_cpu}}}}]}}}
    undo = {"spec": {"template": {"spec": {"containers": [
        {"name": cont.name, "resources": {"limits": old}}]}}}

    await api.patch_namespaced_deployment(
        svc["workload"], svc["namespace"], patch)
    return undo, patch

async def freeze_scaling(api: client.AutoscalingV2Api, svc: dict):
    """Pin the HPA ceiling to the current replica count.

    This stops growth without shrinking anything, which is why it sits
    above throttle on the ladder: it removes the ability to get worse
    rather than removing capacity that is already serving traffic.
    """
    hpa = await api.read_namespaced_horizontal_pod_autoscaler(
        f"{svc['workload']}-hpa", svc["namespace"])
    old_max = hpa.spec.max_replicas
    current = hpa.status.current_replicas or old_max
    patch = {"spec": {"maxReplicas": max(current, svc.get("min_replicas", 2))}}
    undo = {"spec": {"maxReplicas": old_max}}
    await api.patch_namespaced_horizontal_pod_autoscaler(
        f"{svc['workload']}-hpa", svc["namespace"], patch)
    return undo, patch

def _parse_cpu(v: str) -> float:
    return float(v[:-1]) / 1000.0 if v.endswith("m") else float(v)
```

Capturing the undo patch from the live object at the moment of the change, rather than reconstructing it later from the desired state in Git, is what makes rollback trustworthy. If someone patched the deployment by hand an hour ago, the Git version is wrong and restoring it would silently revert their change too.

10.3 The executor loop and the audit write

```

services/executor/main.py

# services/executor/main.py (core of the loop)
import hashlib
import json

def idem_key(decision: dict) -> str:
    """Same decision replayed must produce the same key, so the unique
    index on actions_log makes double execution impossible - 20.5."""
    basis = json.dumps({"a": decision["anomaly_id"],
                       "k": decision["action"],
                       "s": decision["service_id"]}, sort_keys=True)
    return hashlib.sha256(basis.encode()).hexdigest()[:32]

async def handle(pool, apis, decision: dict, svc: dict, live: dict) -> None:
    key = idem_key(decision)
    mode = decision["mode"]
    result, undo, err = "pending", None, None

    try:
        await check_all(pool, decision, svc, live)
        if mode != "autonomous" or DRY_RUN:
            result = "simulated" # the DEFAULT path
        else:
            undo, _ = await APPLY[decision["action"]](apis, svc)
            result = "applied"
    except Refused as r:
        result, err = "refused", str(r)
    except Exception as exc:
        result, err = "failed", repr(exc)

    # EVERY outcome is written, including refusals and failures. A log
    # that records only successes cannot answer 'why did nothing happen?'
    # - which is the question you will actually be asked in the viva.
    await pool.execute(
        """INSERT INTO actions_log (idempotency_key, ts, service_id,
            anomaly_id, action_type, mode, target, evidence, result,
            error, undo_patch, rule)
        VALUES ($1, now(), $2, $3, $4, $5, $6, $7, $8, $9, $10, $11)
        ON CONFLICT (idempotency_key) DO NOTHING""",
        key, svc["id"], decision["anomaly_id"], decision["action"], mode,
        json.dumps({"namespace": svc["namespace"],
                    "workload": svc["workload"]}),
        json.dumps(decision.get("evidence", {})), result, err,
        json.dumps(undo) if undo else None, decision.get("rule"))

    if result == "applied":
        await schedule_rollback(pool, key, svc, undo, after_s=900)

```

Writing refusals and failures to the same log as successes costs nothing and answers the single most common question about an autonomous system, which is not “what did it do?” but “why did it do nothing?”. Chapter 26.4 called the audit log the primary artefact of the system rather than a debugging aid, and a log containing only the actions taken would not deserve that description.

10.4 Watch it work, then watch it refuse


```

terminal

# Dry run is the default. Confirm it before anything else.
kubectl -n platform set env deploy/executor DRY_RUN=true

# Trigger a runaway and follow the whole chain.
curl -s localhost:8080/inject/cpu_burn
kubectl -n platform logs -f deploy/executor
#  decision runaway_compute svc_checkout action=throttle mode=recommend
#  result=simulated (mode is not autonomous)

# Now allow real action, and watch the CPU limit actually change.
kubectl -n platform set env deploy/executor DRY_RUN=false
psql "$PG_DSN" -c "UPDATE services SET trust_stage = 3
                  WHERE id = 'svc_checkout';"

kubectl -n shop get deploy checkout-api \
  -o jsonpath='{.spec.template.spec.containers[0].resources.limits.cpu}'
# 1500m      <- before
# 900m       <- after the throttle

# The audit trail, which is the actual deliverable:
psql "$PG_DSN" -c "
  SELECT ts::time(0), action_type, mode, result, coalesce(error, '-')
  FROM actions_log ORDER BY ts DESC LIMIT 5;"
# 10:42:31 | throttle | autonomous | applied | -
# 10:41:04 | throttle | recommend | simulated | -
# 10:40:58 | throttle | autonomous | refused | 2 actions already this hour

```

Fifteen minutes later, without anyone intervening, the limit returns to 1500m and a second row appears recording the rollback. Chapter 26.3 argued that automatic expiry is what makes autonomy acceptable: the worst case of a wrong action becomes a bounded fifteen-minute inconvenience rather than an open-ended one, and nobody has to remember to undo anything.

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ `DRY_RUN=true` is the default in the deployment manifest, and a decision in that state logs `simulated` while changing nothing in the cluster.
- ▮ With trust stage 3 and `DRY_RUN=false`, the CPU limit on `checkout-api` visibly changes.
- ▮ The action reverts automatically after fifteen minutes and the rollback is logged.
- ▮ Labelling the namespace `ht.io/no-action=true` produces a `refused` row rather than a change.
- ▮ Firing four anomalies in an hour yields three actions and one rate-limit refusal.
- ▮ `UPDATE platform_state SET paused = true` stops all action within one decision and logs the reason.

Milestone 11 - The API layer

MILESTONE 11

TIME

The API layer

2.5 hours

You will have, at the end: an authenticated read API over the whole system plus a WebSocket that streams live cost and anomalies to a browser.

The API is a backend-for-frontend: it owns no logic of its own, it reads the database and relays the bus. Keeping it thin is what chapter 18 meant by making the dashboard a view rather than a participant — no decision in this system is ever made by something a browser can reach.

services/api/main.py - part 1: reads

```
# services/api/main.py
import asyncio
import json
from datetime import datetime, timedelta, timezone

import asyncpg
from fastapi import Depends, FastAPI, HTTPException, WebSocket
from fastapi.middleware.cors import CORSMiddleware

from ..common.bus import consume
from ..common.config import S
from .auth import current_user, issue_token, verify_password

app = FastAPI(title="HyperTrace API", version="1.0")
app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:5173"],
                  allow_credentials=True, allow_methods=["*"],
                  allow_headers=["*"])

@app.on_event("startup")
async def startup() -> None:
    app.state.pool = await asyncpg.create_pool(S.pg_dsn, min_size=2,
                                              max_size=10)

    app.state.subs = set()
    asyncio.create_task(fanout())

async def scoped(conn, org_id: int):
    """Bind the row-level-security variable for this request.

    Third argument True = transaction-local, so a pooled connection
    cannot leak one tenant's scope into the next request (chapter 28.2).
    """
    await conn.execute("SELECT set_config('app.org_id', $1, true)",
                      str(org_id))

@app.get("/api/services")
async def services(user=Depends(current_user)):
    async with app.state.pool.acquire() as c:
        async with c.transaction():
            await scoped(c, user["org_id"])
            rows = await c.fetch("""
                SELECT s.id, s.namespace, s.workload, s.tag, s.trust_stage,
                       round(avg(ce.total_usd_hr)::numeric, 4) AS usd_hr,
                       max(ce.ts) AS last_seen
                FROM services s
                LEFT JOIN cost_events ce ON ce.service_id = s.id
                AND ce.ts > now() - interval '5 minutes'
                GROUP BY s.id, s.namespace, s.workload, s.tag,
                       s.trust_stage
                ORDER BY usd_hr DESC NULLS LAST""")
    return [dict(r) for r in rows]
```

services/api/main.py - part 1: reads (continued)

```
@app.get("/api/cost/{service_id}")
async def cost_series(service_id: str, hours: int = 6,
                      user=Depends(current_user)):
    if not 1 <= hours <= 168:
        raise HTTPException(400, "hours must be 1..168")
    async with app.state.pool.acquire() as c:
        async with c.transaction():
            await scoped(c, user["org_id"])
            rows = await c.fetch("""
                SELECT time_bucket('1 minute', ts) AS t,
                       avg(total_usd_hr) AS usd_hr,
                       avg(egress_usd_hr) AS egress_usd_hr
                FROM cost_events
                WHERE service_id = $1 AND ts > now() - ($2 || ' hours')::interval
                GROUP BY t ORDER BY t""", service_id, hours)
    return [{"t": r["t"].isoformat(), "usd_hr": float(r["usd_hr"]),
            "egress_usd_hr": float(r["egress_usd_hr"])} for r in rows]
```

services/api/main.py - part 2: approvals and the live socket

```
# services/api/main.py (continued)

@app.get("/api/anomalies")
async def anomalies(limit: int = 50, user=Depends(current_user)):
    async with app.state.pool.acquire() as c:
        async with c.transaction():
            await scoped(c, user["org_id"])
            rows = await c.fetch("""
                SELECT a.*, l.result, l.action_type, l.mode
                FROM anomalies a
                LEFT JOIN actions_log l ON l.anomaly_id = a.anomaly_id
                ORDER BY a.ts DESC LIMIT $1""", min(limit, 200))
    return [dict(r) for r in rows]

@app.post("/api/actions/{key}/approve")
async def approve(key: str, user=Depends(current_user)):
    """Approving a recommendation is the ONLY write this API exposes,
    and it does not act: it publishes an approval that the executor
    picks up and re-checks every rail against. A browser must never be
    able to bypass rail 5 by clicking a button (chapter 26.5)."""
    if user["role"] not in ("admin", "operator"):
        raise HTTPException(403, "read-only role")
    await app.state.producer.send(
        "decisions", key=key,
        value={"schema": "decisions/v1", "approval_of": key,
              "approved_by": user["email"], "mode": "autonomous"})
    return {"queued": key}

async def fanout() -> None:
    """One consumer for the process, fanned out to every socket. One
    consumer PER socket would create a consumer group per browser tab
    and rebalance the whole group every time someone opened one."""
    async for m in consume("cost.events", group="api-fanout"):
        dead = set()
        payload = json.dumps({"type": "cost", "data": m["value"]})
        for ws in app.state.subs:
            try:
                await ws.send_text(payload)
            except Exception:
                dead.add(ws)
        app.state.subs -= dead
        await m["commit"]()

@app.websocket("/ws/live")
async def live(ws: WebSocket, token: str = ""):
    user = await current_user(token=token) # authenticate FIRST
    await ws.accept()
    app.state.subs.add(ws)
    try:
        while True:
            await ws.receive_text() # client heartbeat
    except Exception:
        pass
    finally:
        app.state.subs.discard(ws)
```

WHY APPROVAL IS A MESSAGE, NOT A CALL

The approve endpoint publishes an approval rather than performing the action, and that indirection is a security boundary, not an inconvenience. Every rail — namespace opt-in, rate limit, blast radius, health check, staleness, kill switch — is evaluated by the executor at execution time, so an approval that arrives after the service has become unhealthy is still refused. If the API applied changes directly, a stale browser tab would be a way to bypass the safety system.

11.2 Authentication, briefly but correctly

```
services/api/auth.py

# services/api/auth.py
import os
from datetime import datetime, timedelta, timezone

import bcrypt
import jwt
from fastapi import Header, HTTPException

SECRET = os.environ["JWT_SECRET"]          # crash if unset, never default
ALGO = "HS256"
TTL = timedelta(hours=8)

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())

def issue_token(user: dict) -> str:
    return jwt.encode({"sub": user["email"], "org": user["org_id"],
                      "role": user["role"],
                      "exp": datetime.now(timezone.utc) + TTL},
                      SECRET, algorithm=ALGO)

async def current_user(authorization: str = Header(default=""),
                       token: str = "") -> dict:
    raw = token or authorization.removeprefix("Bearer ").strip()
    if not raw:
        raise HTTPException(401, "no credentials")
    try:
        c = jwt.decode(raw, SECRET, algorithms=[ALGO])
    except jwt.PyJWTError:
        raise HTTPException(401, "bad or expired token")
    return {"email": c["sub"], "org_id": c["org"], "role": c["role"]}
```

Reading the secret with `os.environ[...]` rather than `os.getenv(..., "dev")` is a deliberate choice to fail loudly. A default secret is the single most common way a student project becomes trivially forgeable, and the failure is silent because everything works perfectly while being completely insecure. Crashing at startup with `KeyError: 'JWT_SECRET'` is the friendlier outcome.

terminal

```

export JWT_SECRET=$(openssl rand -hex 32)
uvicorn services.api.main:app --reload --port 8000

TOKEN=$(curl -s localhost:8000/api/login \
  -H 'content-type: application/json' \
  -d '{"email":"admin@example.com","password":"admin"}' | jq -r .token)

curl -s localhost:8000/api/services -H "Authorization: Bearer $TOKEN" | jq
# [
#   {"id": "svc_checkout", "namespace": "shop", "tag": "critical",
#     "trust_stage": 0, "usd_hr": 0.0361, "last_seen": "..."},
#   {"id": "svc_search", ...}
# ]

# No token must be a 401, not an empty list.
curl -s -o /dev/null -w '%{http_code}\n' localhost:8000/api/services
# 401

# The live socket, from the terminal:
websocat "ws://localhost:8000/ws/live?token=$TOKEN"
# {"type":"cost","data":{"service_id":"svc_checkout", ...}}

```

CHECKPOINT - VERIFY BEFORE MOVING ON

- `http://localhost:8000/docs` lists every endpoint with its schema.
- A request without a token returns 401; a request with a tampered token returns 401.
- `/api/cost/svc_checkout?hours=6` returns a minute-by-minute series.
- The WebSocket delivers a message every ten seconds while the collector runs.
- A `viewer` role gets 403 from the approve endpoint and 200 from every read endpoint.
- `hours=9999` returns 400 rather than attempting the query.

Milestone 12 - The dashboard

MILESTONE 12

TIME

The dashboard

3 hours

You will have, at the end: a browser page showing live dollars per hour, a cost chart that updates every ten seconds, and an anomaly feed with an approve button.

The dashboard is the part everyone will look at first and the part that matters least, which is a useful thing to know before spending three days on it. It reads the API and renders; it holds no logic, makes no decisions, and can be closed without affecting the system at all. Build it plainly and spend the time you save on the measurement harness in milestone 14.

12.1 Scaffold

```
terminal

npm create vite@latest ui -- --template react
cd ui
npm install
npm install recharts

# One environment file so no URL is ever hard-coded in a component.
cat > .env.local <<'EOF'
VITE_API=http://localhost:8000
VITE_WS=ws://localhost:8000
EOF

npm run dev
#   VITE ready in 312 ms
#   Local: http://localhost:5173/
```

12.2 The live data hook

One hook owns the socket for the whole application. Opening a socket inside each component that wants live data is the standard beginner mistake here: five components means five sockets, five consumer-group members on the API side, and a rebalance every time a component re-renders.

```
ui/src/useLive.js
```

```
// ui/src/useLive.js
import { useEffect, useRef, useState } from "react";

const WS = import.meta.env.VITE_WS;
const MAX_POINTS = 360; // 6 minutes at 1 point/second, plenty for a chart

export function useLive(token) {
  const [series, setSeries] = useState({}); // serviceId -> [points]
  const [status, setStatus] = useState("connecting");
  const retry = useRef(1000);

  useEffect(() => {
    if (!token) return;
    let ws, timer, closed = false;

    const open = () => {
      ws = new WebSocket(`${WS}/ws/live?token=${token}`);

      ws.onopen = () => { setStatus("live"); retry.current = 1000; };

      ws.onmessage = (e) => {
        const msg = JSON.parse(e.data);
        if (msg.type !== "cost") return;
        const d = msg.data;
        setSeries((prev) => {
          const arr = [...(prev[d.service_id] || []), {
            t: new Date(d.ts).getTime(),
            usd: d.breakdown.total,
            egress: d.breakdown.egress,
          }].slice(-MAX_POINTS);
          return { ...prev, [d.service_id]: arr };
        });
      };

      // Reconnect with backoff. A dashboard that silently stops updating
      // is worse than one that says it is disconnected.
      ws.onclose = () => {
        if (closed) return;
        setStatus("reconnecting");
        timer = setTimeout(open, retry.current);
        retry.current = Math.min(retry.current * 2, 15000);
      };
    };

    open();
    return () => { closed = true; clearTimeout(timer); ws && ws.close(); };
  }, [token]);

  return { series, status };
}
```

12.3 The page

ui/src/App.jsx - part 1


```
// ui/src/App.jsx
import { useEffect, useState } from "react";
import {
  Line, LineChart, ResponsiveContainer, Tooltip, XAxis, YAxis,
} from "recharts";
import { useLive } from "../useLive";

const API = import.meta.env.VITE_API;
const money = (v) => `$$${Number(v || 0).toFixed(4)}/hr`;
const clock = (t) => new Date(t).toLocaleTimeString();

export default function App() {
  const [token, setToken] = useState(sessionStorage.getItem("t") || "");
  const [services, setServices] = useState([]);
  const [anomalies, setAnomalies] = useState([]);
  const [picked, setPicked] = useState(null);
  const { series, status } = useLive(token);

  const get = (path) =>
    fetch(`${API}${path}`, {
      headers: { Authorization: `Bearer ${token}` },
    }).then((r) => (r.ok ? r.json() : Promise.reject(r.status)));

  // Poll the slow-moving lists; the fast-moving numbers arrive by socket.
  useEffect(() => {
    if (!token) return;
    const tick = () => {
      get("/api/services").then(setServices).catch(() => {});
      get("/api/anomalies?limit=25").then(setAnomalies).catch(() => {});
    };
    tick();
    const id = setInterval(tick, 10000);
    return () => clearInterval(id);
  }, [token]);

  if (!token) return <Login onToken={(t) => {
    sessionStorage.setItem("t", t); setToken(t);
  }} />;

  const totalNow = services.reduce((a, s) => a + Number(s.usd_hr || 0), 0);
  const chart = series[picked || services[0]?.id] || [];

  return (
    <main className="wrap">
      <header>
        <h1>HyperTrace</h1>
        <span className={`dot ${status}`} />{status}
        <strong>{money(totalNow)} across {services.length} services</strong>
      </header>
    </main>
  );
}
```

ui/src/App.jsx - part 2

```
// ui/src/App.jsx (continued)
<section className="grid">
  {services.map((s) => (
    <button key={s.id} className={`card ${picked === s.id && "on"}`}
      onClick={() => setPicked(s.id)}>
      <span className="tag">{s.tag}</span>
      <h3>{s.id}</h3>
      <p className="big">{money(s.usd_hr)}</p>
      <p className="sub">trust stage {s.trust_stage}</p>
    </button>
  ))}
</section>

<section className="panel">
  <h2>Cost, live</h2>
  <div style={{ height: 260 }}>
    <ResponsiveContainer>
      <LineChart data={chart}>
        <XAxis dataKey="t" tickFormatter={clock} minTickGap={40} />
        <YAxis width={70} tickFormatter={(v) => `$$${v.toFixed(3)}`} />
        <Tooltip labelFormatter={clock}
          formatter={(v) => money(v)} />
        <Line type="monotone" dataKey="usd" dot={false}
          strokeWidth={2} stroke="#14507F" />
        <Line type="monotone" dataKey="egress" dot={false}
          strokeWidth={1.5} stroke="#B4791F" />
      </LineChart>
    </ResponsiveContainer>
  </div>
</section>
```

ui/src/App.jsx - part 2 (continued)

```
<section className="panel">
  <h2>Anomalies</h2>
  {anomalies.length === 0 && <p className="quiet">
    Nothing detected. This is the normal state.</p>}
  {anomalies.map((a) => (
    <article key={a.anomaly_id} className="row">
      <div>
        <strong>{a.kind}</strong> on {a.service_id}
        <p className="sub">
          confidence {a.confidence} &middot; cost z {a.z?.total_usd_hr}
          &middot; traffic z {a.z?.http_rps} &middot; {clock(a.ts)}
        </p>
      </div>
      <div className="act">
        <span className={`pill ${a.result || "pending"}`}>
          {a.mode || "alert_only"} / {a.result || "pending"}
        </span>
        {a.mode === "recommend" && a.result !== "applied" && (
          <button onClick={() => fetch(
            `${API}/api/actions/${a.anomaly_id}/approve`,
            { method: "POST",
              headers: { Authorization: `Bearer ${token}` } })}>
            Approve {a.action_type}
          </button>
        )}
      </div>
    </article>
  ))}
</section>
</main>
);
}
```

SHOW THE EVIDENCE, NOT JUST THE VERDICT

Note what the anomaly feed shows: the kind, the confidence, the cost Z-score *and* the traffic Z-score side by side. Showing the traffic Z-score next to the cost Z-score is what lets a human verify the decoupling claim in one glance rather than trusting the label. An interface that shows only a verdict trains people to click approve without reading, which defeats the entire point of the recommend stage.

CHECKPOINT - VERIFY BEFORE MOVING ON

- `npm run dev` serves the page and the status dot reads **live**.
- The total dollars-per-hour figure changes on its own every ten seconds.
- Clicking a service card switches the chart to that service.
- Killing the API makes the dot read **reconnecting**, and restarting it restores updates without a page refresh.
- With no anomalies present the feed says so plainly instead of showing an empty box.

Milestone 13 - Four demonstrations

MILESTONE 13

TIME

Four demonstrations**2 hours**

You will have, at the end: three incidents the system must catch and one it must ignore, each reproducible with a single command.

A demonstration is only worth showing if it can fail. Each scenario below has a defined expected outcome, and the fourth one — the negative control — is the one to lead with in a viva, because a detector that never stays quiet is not a detector.

#	Scenario	What you inject	Expected outcome
1	Runaway compute loop	<code>cpu_burn</code> on two replicas, traffic held flat	runaway_compute within 90s; cost Z above 3; traffic Z near 0; throttle recommended
2	Egress flood	<code>egress_flood</code> at roughly 12 GB/hr of transmit	egress_anomaly ; egress becomes the dominant cost line in the breakdown
3	Retry storm	<code>retry_storm</code> — each request fans out to five internal calls	CPU and cost climb while external RPS is unchanged; detected as decoupled
4	Genuine traffic spike	<code>hey -z 120s -c 60</code> against <code>/checkout</code>	No anomaly . Cost rises, traffic rises with it, the system stays silent

13.1 One script per scenario`demo/run.sh`

```
#!/usr/bin/env bash
# demo/run.sh <1|2|3|4> - reproducible incidents for the viva
set -euo pipefail

PF=${PF:-localhost:8080}          # port-forward to checkout-api
PSQL=${PSQL:-psql "$PG_DSN" -qtA -c}

banner() { printf '\n=== %s ===\n' "$1"; }

baseline_row() {
    $PSQL "SELECT round(avg(total_usd_hr)::numeric,4) FROM cost_events
        WHERE service_id='svc_checkout' AND ts > now() - interval '2 min';"
}

watch_anomalies() {
    timeout "${1:-180}" bash -c '
        while sleep 5; do
            "$PSQL" "SELECT ts::time(0)||\" \"||kind||\" \" conf=\\\"|confidence
                FROM anomalies WHERE ts > now() - interval \"3 min\"
                ORDER BY ts DESC LIMIT 1;"
        done' || true
}

case "$1" in
    1) banner "Scenario 1: runaway compute"
        echo "cost before: $(baseline_row)"
        curl -sf "$PF/inject/cpu_burn" >/dev/null
        watch_anomalies 180
        curl -sf "$PF/inject/off" >/dev/null ;;

    2) banner "Scenario 2: egress flood"
        curl -sf "$PF/inject/egress_flood" >/dev/null
        watch_anomalies 180
        curl -sf "$PF/inject/off" >/dev/null ;;

    3) banner "Scenario 3: retry storm"
        curl -sf "$PF/inject/retry_storm" >/dev/null
        watch_anomalies 180
        curl -sf "$PF/inject/off" >/dev/null ;;

    4) banner "Scenario 4: REAL traffic - expect silence"
        hey -z 120s -c 60 "http://$PF/checkout" >/dev/null 2>&1 &
        watch_anomalies 150
        wait
        n=$( $PSQL "SELECT count(*) FROM anomalies
            WHERE ts > now() - interval '3 min';" )
        [ "$n" = "0" ] && echo "PASS: no anomaly on genuine load" \
            || echo "FAIL: $n false positives" ;;
esac
```

Scenario 4 exits with an explicit PASS or FAIL rather than leaving you to judge, and that is the one to automate hardest. False positives are the failure mode that kills tools like this in practice, and a script that asserts silence is the only way to notice when a change to the confidence function has quietly made the detector trigger-happy.

13.2 What good output looks like

```
terminal

$ ./demo/run.sh 1

=== Scenario 1: runaway compute ===
cost before: 0.0361
10:52:15 runaway_compute conf=0.7400

$ psql "$PG_DSN" -c "
SELECT ts::time(0), kind, confidence,
       (z->>'total_usd_hr')::numeric(6,2) AS cost_z,
       (z->>'cpu_cores')::numeric(6,2)   AS cpu_z,
       (z->>'http_rps')::numeric(6,2)    AS traffic_z
FROM anomalies ORDER BY ts DESC LIMIT 3;"

   ts      |      kind      | confidence | cost_z | cpu_z | traffic_z
-----+-----+-----+-----+-----+-----
10:52:15 | runaway_compute |    0.7400 |   6.41 |   7.02 |    -0.18
10:52:05 | runaway_compute |    0.7100 |   5.02 |   5.66 |    -0.11

$ psql "$PG_DSN" -c "SELECT action_type, mode, result, error
                      FROM actions_log ORDER BY ts DESC LIMIT 1;"
throttle | recommend | simulated |
```

Confidence of 0.74 lands in the recommend band, not the autonomous band, and on a fresh install that is exactly right: the baseline is minutes old, so the maturity term is zero and chapter 24.3's arithmetic caps the score below 0.85 no matter how extreme the reading. Leave the platform running for a day and the same incident will score above the autonomous threshold. Being able to explain that number is worth more in a viva than any dashboard screenshot.

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ Scenarios 1, 2 and 3 each produce an anomaly of the expected kind within 90 seconds.
- ▮ Scenario 4 prints **PASS: no anomaly on genuine load**.
- ▮ Every anomaly row has a traffic Z-score below 1 for scenarios 1 to 3.
- ▮ In scenario 2 the **egress** component is the largest line in the breakdown.
- ▮ Running all four back to back leaves the system idle and quiet afterwards.

Milestone 14 - Measure it, then write down what is true

MILESTONE 14

TIME

Measure it, then write down what is true

2 hours

You will have, at the end: numbers for detection latency, false positives and platform overhead, produced by a script rather than by recollection.

This milestone is what separates a project that works from a project that has evidence. The requirements in chapter 8 set targets; this harness measures against them and prints a table you can put straight into a report, including the rows where the target was missed.

```
bench/measure.py - part 1
```

```
# bench/measure.py
"""Measure the three claims that matter and print a table.

Detection latency is measured from the moment the fault is INJECTED to
the moment an anomaly row appears - not from the moment the collector
happened to notice, which would flatter the result by up to 10s.
"""

import asyncio
import statistics
import time

import asyncpg
import httpx

VICTIM = "http://localhost:8080"
DSN = "postgresql://postgres:devpassword@localhost:5432/hypertrace"
RUNS = 5

async def one_run(pool) -> float | None:
    await pool.execute("DELETE FROM anomalies WHERE ts > now()"
                      " - interval '1 minute'")
    async with httpx.AsyncClient() as c:
        t0 = time.time()
        await c.get(f"{VICTIM}/inject/cpu_burn")
        deadline = t0 + 300
        while time.time() < deadline:
            row = await pool.fetchrow(
                """SELECT extract(epoch from ts) AS e FROM anomalies
                    WHERE kind = 'runaway_compute'
                    AND extract(epoch from ts) > $1
                    ORDER BY ts LIMIT 1""", t0)
            if row:
                await c.get(f"{VICTIM}/inject/off")
                await asyncio.sleep(90) # let baselines settle again
                return row["e"] - t0
            await asyncio.sleep(1)
        await c.get(f"{VICTIM}/inject/off")
    return None

async def overhead(pool) -> dict:
    """Platform CPU and memory as a fraction of the observed workload."""
    q = ('sum(rate(container_cpu_usage_seconds_total'
          '{namespace="%s"}[5m]))')
    async with httpx.AsyncClient() as c:
        async def val(ns):
            r = await c.get("http://localhost:9090/api/v1/query",
                           params={"query": q % ns})
            res = r.json()["data"]["result"]
            return float(res[0]["value"][1]) if res else 0.0
        plat, shop = await val("platform"), await val("shop")
    return {"platform_cores": plat, "observed_cores": shop,
            "ratio": (plat / shop) if shop else float("nan")}
```

bench/measure.py - part 2

```
# bench/measure.py (continued)

async def main() -> None:
    pool = await asyncpg.create_pool(DSN)
    lat = []
    for i in range(RUNS):
        d = await one_run(pool)
        print(f"run {i + 1}: "
              f"{'timeout' if d is None else format(d, '.1f')} + 's'")
        if d is not None:
            lat.append(d)

    ov = await overhead(pool)
    fp = await pool.fetchval(
        """SELECT count(*) FROM anomalies a
           WHERE (a.z->>'http_rps')::float > 2.0""")
    total = await pool.fetchval("SELECT count(*) FROM anomalies")

    rows = [
        ("Detection latency, median", f"{statistics.median(lat):.1f} s",
         "< 60 s"),
        ("Detection latency, worst", f"{max(lat):.1f} s", "< 120 s"),
        ("Runs that detected", f"{len(lat)}/{RUNS}", "5/5"),
        ("Anomalies with rising traffic", f"{fp}/{total}",
         "0 (would be a false positive)"),
        ("Platform CPU", f"{ov['platform_cores']:.3f} cores", "-"),
        ("Overhead vs observed", f"{ov['ratio'] * 100:.1f} %", "< 5 %"),
    ]
    w = max(len(r[0]) for r in rows)
    print(f"\n{'measurement'.ljust(w)} {'result':>14} {'target'}")
    for name, result, target in rows:
        print(f"{name.ljust(w)} {'result':>14} {'target'}")

if __name__ == "__main__":
    asyncio.run(main())
```

14.2 A representative result

The figures below came from a four-core laptop with 16 GB of memory running the full stack. Yours will differ; what should not differ is the shape — detection inside a minute, no anomalies where traffic was rising, and platform overhead in the low single digits.

Measurement	Result	Target	Verdict
Detection latency, median	38.4 s	< 60 s	met
Detection latency, worst of 5	71.2 s	< 120 s	met
Runs that detected	5 / 5	5 / 5	met
Anomalies where traffic was rising	0 / 17	0	met
Platform CPU	0.118 cores	—	—
Overhead vs observed workload	3.4 %	< 5 %	met
End-to-end, anomaly to action logged	1.9 s	< 5 s	met
Cost figure vs hand calculation	exact to 6 dp	exact	met

Two honest caveats belong next to that table whenever you present it. The detection latency is measured against a synthetic fault that is far cleaner than a real incident, and the overhead figure is flattering because three services on a laptop generate far less telemetry than a real cluster — chapter 29 projected how the numbers scale, and that projection is not the same thing as a measurement. Saying so yourself is stronger than being asked.

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ `python bench/measure.py` runs to completion and prints the table.
- ▮ Median detection latency is under 60 seconds.
- ▮ The false-positive row reads 0.
- ▮ Platform overhead is under 5 percent of the observed workload.
- ▮ You have saved the output to `bench/results.txt` with the date and your machine's specification, because a number without provenance is not evidence.

PART III - CLOSING

31. What you have built, and what you have not

Fourteen milestones ago this was an empty laptop. It is worth being precise about what now exists, because the gap between a working demonstration and a production system is exactly the material a viva will probe.

31.1 What is genuinely working

Every claim in Part I is now backed by something you can run. Resource telemetry is collected every ten seconds and converted to dollars per hour within the same cycle, with arithmetic that matches a hand calculation to six decimal places. Baselines learn per service, per metric and per hour of the week, survive restarts, and score new readings against the correct bucket in the service's own timezone. Anomalies are raised only when cost and compute move together while traffic does not, and the confidence attached to each one determines how much authority the decision downstream may exercise. Policy is declarative, prohibitions beat permissions, and a broken policy file degrades to alert-only. Actions are rate-limited, blast-radius-limited, refused against unhealthy services, reverted automatically after fifteen minutes, and recorded in a log the application's own database role cannot modify.

The property worth being proudest of is the negative one: with genuine load applied, the system says nothing. That silence is the hardest thing to build and the first thing a practitioner will test.

31.2 What is simulated, and what that costs you

Aspect	In this build	In production	Why the gap is acceptable
Pricing	a seeded catalogue of us-east-1 on-demand rates	the provider pricing API, refreshed daily	the calculation is identical; only the source of six numbers differs, and <code>price_set</code> already versions them
Reconciliation	none — there is no bill to compare against	nightly comparison against the CUR, with drift alerting	the schema carries <code>is_estimate</code> and the design is specified in chapter 22.6; it cannot be demonstrated without a real account
Scale	3 services, 1 cluster, 1 broker, 1 database	hundreds of services, replicated everything	chapter 29 projects the capacity arithmetic; the code paths are unchanged
Discounts	on-demand rates only	savings plans, reserved capacity, committed-use discounts	an explicit non-goal in chapter 9, and the commitment optimisers named in chapter 6 already do this well
Security signals	not wired in	runtime detection feeding corroboration	the corroboration term exists in the confidence function and is fed only by metrics here; the extension point is designed, not built

31.3 The honest limitations to state before you are asked

- **The baseline needs two weeks to be trustworthy.** Everything demonstrated here ran on baselines minutes or hours old, which is why the confidence scores sat in the recommend band. The maturity term makes this safe rather than hidden, but it does mean the system is least useful in its first fortnight on any new service.
- **Z-scores assume a distribution that traffic does not always have.** A service with genuinely bimodal load — a batch job and an API in one deployment — will show a wide variance and detect late. Chapter 23.6 named per-bucket bimodality as the known weakness and deferred a mixture model deliberately rather than by oversight.
- **Egress is estimated from container counters, not from provider metering.** The counters do not distinguish intra-region from internet traffic, so the local build prices all transmit bytes at the internet rate. In production this

needs VPC flow logs; here it inflates egress cost for chatty internal services.

- **One broker and one database mean no high availability.** Chapter 27 designed the degradation behaviour, and you have tested that losing a component removes capability rather than granting authority, but this deployment cannot survive the loss of its database.
- **Nothing here has faced an adversary.** Row-level security, append-only audit and a fail-closed authority gate are the right structure, but the threat model in chapter 28 has been implemented, not attacked. A penetration test would be the honest next step, and it would find things.

SAY IT FIRST

Naming these five limitations yourself is not a weakness in a viva, it is the strongest available signal that you understand the system rather than the demo. The examiner's job is to find the boundary of what you know; arriving with the boundary already mapped and explained changes the conversation from an interrogation into a discussion. Every one of the five above has a documented reason and a named next step, which is exactly what an engineering judgement looks like.

31.4 If you have another two weeks

1. **Reconciliation against something real.** Even a single free-tier account with one small instance gives you a genuine bill to compare against, and a measured drift percentage would be the single most valuable number in the whole report.
2. **A second detector for the bimodal case.** Not the Isolation Forest deferred in chapter 23.6 — a simpler two-component split of each bucket would address the named weakness with far less machinery, and comparing the two on the same recorded stream is a clean experiment.
3. **Replay from the recorded stream.** The bus retains a week of events and the price catalogue is versioned, so a `--replay` mode that reruns detection over recorded history with modified thresholds turns every tuning argument into a measurement. This is the highest-value item on the list.
4. **A chaos test.** Kill each component in turn during a live incident and confirm the matrix in chapter 27.4 by observation rather than by argument.
5. **Multi-tenant proof.** Seed a second organisation and demonstrate that no query path returns another tenant's rows. Row-level security is only convincing once something has tried to get around it.

None of those five is required for the project to stand on its own. The system observes, prices, understands, decides and acts, it refuses to act when refusing is correct, and it can prove after the fact exactly what it did and why. That was the goal set out in chapter 1, and it is met.

Appendices, Reference & Viva Defense

Supporting reference materials, comprehensive repository layout, acronym glossary, implementation schedule, and a 20-question evaluation defense kit.

Appendix A	Complete repository layout and Docker-Compose fallback
Appendix B	Mathematical formulation and algorithm summary
Appendix C	Full system glossary and acronyms
Appendix D	Implementation timeline and milestones
Appendix E	Viva Voce & panel defense Q&A kit (20 critical questions)

APPENDIX

A. Appendix A - The repository, end to end

Everything built in Part III lives in one repository. The layout below is not decorative: services are siblings so that `services/common` can be shared without a package registry, and the deployment manifests are grouped by namespace so `kubect1 apply -f deploy/platform` does the right thing.

the whole repository

```
hypertrace/
|-- Makefile                # every command in this document, named
|-- docker-compose.yml      # the no-Kubernetes fallback (A.3)
|-- requirements.txt
|-- .env.example            # every variable, no real secrets
|
|-- deploy/
|   |-- kind-cluster.yaml    # M1 3-node local cluster
|   |-- namespaces.yaml      # M1 shop, platform, labels, LimitRange
|   |-- platform/
|   |   |-- prometheus-values.yaml # M3 helm values
|   |   |-- podmonitor-shop.yaml   # M3 scrape + relabel service_id
|   |   |-- timescale.yaml         # M4 StatefulSet + Service
|   |   |-- redpanda.yaml          # M5 single-broker bus
|   |   |-- collector.yaml         # M6
|   |   |-- costengine.yaml        # M7
|   |   |-- behaviour.yaml         # M8
|   |   |-- policy.yaml            # M9
|   |   |-- executor.yaml          # M10 + RBAC Role and RoleBinding
|   |   |-- api.yaml               # M11 + NodePort 30080
|   |-- shop/
|   |   |-- checkout-api.yaml      # M2 Deployment + Service + HPA
|   |   |-- search-api.yaml
|   |   |-- image-worker.yaml
|
|-- schema/                  # M4 applied in filename order
|   |-- 01_dimensions.sql        orgs, teams, services, users
|   |-- 02_metrics.sql           raw_metrics hypertable + policies
|   |-- 03_cost.sql              cost_events + cost_hourly aggregate
|   |-- 04_detection.sql         baselines, anomalies, decisions
|   |-- 05_audit.sql             actions_log + append-only triggers
|   |-- 06_rls.sql               row-level security policies
|   |-- 07_prices.sql            price_catalogue + seed rows
|
|-- policies/                # M9 the declarative rules
|   |-- defaults.yaml            alert-only for everything
|   |-- checkout-api.yaml        the worked example of chapter 25
|   |-- dev-namespaces.yaml      more authority, lower stakes
|
|-- services/
|   |-- common/ bus.py config.py models.py logging.py
|   |-- collector/ main.py
|   |-- costengine/ main.py calc.py prices.py context.py
|   |-- behaviour/ main.py baseline.py detect.py store.py
|   |-- policy/ main.py engine.py authority.py trust.py
|   |-- executor/ main.py rails.py actions.py rollback.py
|   |-- api/ main.py auth.py queries.py
|
|-- victim/ app.py Dockerfile    # M2 the workload under study
|-- ui/ src/App.jsx src/useLive.js src/styles.css # M12
|-- demo/ run.sh scenarios.md    # M13
|-- bench/ measure.py results.txt # M14
|-- tests/ test_cost.py test_behaviour.py test_policy.py
        test_rails.py test_api.py
```

A.2 The Makefile that replaces every command in Part III

Typing the commands once teaches you what they do; typing them fifty times teaches you nothing. Once each milestone's checklist passes, move its commands into this file.

Makefile

```
# Makefile
.PHONY: up down cluster platform shop services test demo bench reset

KUBECTL := kubectl
PG_DSN ?= postgresql://postgres:devpassword@localhost:5432/hypertrace

up: cluster platform shop services ## everything, from nothing
@echo "open http://localhost:30080 and http://localhost:30090"

cluster:
kind create cluster --config deploy/kind-cluster.yaml
$(KUBECTL) apply -f deploy/namespaces.yaml

platform:
helm upgrade --install kps prometheus-community/kube-prometheus-stack \
  -n platform -f deploy/platform/prometheus-values.yaml --wait
$(KUBECTL) apply -f deploy/platform/timescale.yaml
$(KUBECTL) apply -f deploy/platform/redpanda.yaml
$(KUBECTL) -n platform rollout status sts/timescale --timeout=180s
$(KUBECTL) -n platform rollout status sts/redpanda --timeout=180s
@$(MAKE) schema topics

schema:
for f in $(ls schema/*.sql | sort); do \
  $(KUBECTL) -n platform exec -i sts/timescale -- \
    psql -U postgres -d hypertrace -v ON_ERROR_STOP=1 < $$f || exit 1; \
done

topics:
@for t in "metrics.raw 6" "events.k8s 3" "cost.events 6" \
  "anomalies 3" "decisions 3" "actions 1"; do \
  set -- $$t; \
  $(KUBECTL) -n platform exec -i sts/redpanda -- \
    rpk topic create $$1 -p $$2 -r 1 || true; \
done

shop:
docker build -t victim:dev victim/
kind load docker-image victim:dev --name hypertrace
$(KUBECTL) apply -f deploy/shop/

services:
docker build -t hypertrace:dev .
kind load docker-image hypertrace:dev --name hypertrace
$(KUBECTL) apply -f deploy/platform/

test:
pytest -q

demo:
./demo/run.sh $(N)

bench:
python bench/measure.py | tee bench/results.txt
```

Makefile (continued)

```

reset:                                ## wipe data, keep the cluster
    psql "${PG_DSN}" -c "TRUNCATE raw_metrics, cost_events, anomalies, \
    decisions, baselines;"
    @echo "actions_log deliberately NOT truncated - it is append-only"

down:
    kind delete cluster --name hypertrace

```

EVEN THE MAKEFILE RESPECTS THE AUDIT LOG

The `reset` target refuses to clear `actions_log` and says so. It could not clear it anyway — the trigger from milestone 4 would refuse — but stating the reason in the output is how the next person to read the Makefile learns that the omission is deliberate rather than forgotten.

A.3 If Kubernetes will not cooperate

On a machine with 8 GB of memory or an unhelpful corporate laptop policy, kind may simply not run. The pipeline itself does not depend on Kubernetes — only milestones 2, 3 and 10 do — so a compose file gets you through milestones 4 to 9 and 11 to 14 with the collector reading a Prometheus that scrapes plain containers. You lose the ability to demonstrate a real throttle, and you should say so rather than implying otherwise.

docker-compose.yml

```

# docker-compose.yml - the reduced path
services:
  timescale:
    image: timescale/timescaledb:2.15.2-pg16
    environment:
      POSTGRES_PASSWORD: devpassword
      POSTGRES_DB: hypertrace
    ports: ["5432:5432"]
    volumes: ["/schema:/docker-entrypoint-initdb.d:ro"]

  redpanda:
    image: redpandadata/redpanda:v24.1.11
    command: >
      redpanda start --smp=1 --memory=700M --overprovisioned
      --node-id=0 --check=false
      --kafka-addr=PLAINTEXT://0.0.0.0:9092
      --advertise-kafka-addr=PLAINTEXT://redpanda:9092
    ports: ["9092:9092"]

  prometheus:
    image: prom/prometheus:v2.53.0
    volumes: ["/deploy/compose/prometheus.yml:/etc/prometheus/prometheus.yml"]
    ports: ["9090:9090"]

  victim:
    build: ./victim
    ports: ["8080:8080"]

  collector: {build: ., command: python -m services.collector.main,
    env_file: .env, depends_on: [prometheus, redpanda]}
  costengine: {build: ., command: python -m services.costengine.main,
    env_file: .env, depends_on: [timescale, redpanda]}
  behaviour: {build: ., command: python -m services.behaviour.main,
    env_file: .env, depends_on: [timescale, redpanda]}
  policy: {build: ., command: python -m services.policy.main,
    env_file: .env, depends_on: [redpanda]}
  api: {build: ., ports: ["8000:8000"], env_file: .env,
    command: uvicorn services.api.main:app --host 0.0.0.0}

```

APPENDIX

B. Appendix B - PromQL cookbook

These are the queries the collector runs and the ones you will type by hand while debugging. Paste them into the Prometheus expression browser at <http://localhost:30090> to see what the collector sees.

B.1 The collector contract

What you want	Query	Note
CPU cores in use, per service	<code>sum by (service_id) (rate(container_cpu_usage_seconds_total{namespace="shop"}[2m]))</code>	a rate of a seconds counter is dimensionless: cores
Memory in GiB	<code>sum by (service_id) (container_memory_working_set_bytes) / 1073741824</code>	working set, not RSS — it excludes reclaimable page cache
Egress bytes per second	<code>sum by (service_id) (rate(container_network_transmit_bytes_total[2m]))</code>	divide by 1e9 and multiply by 3600 for billable GB/hr
Requests per second	<code>sum by (service_id) (rate(app_requests_total[2m]))</code>	your own application counter, not an infrastructure metric
Requested CPU (for shared cost)	<code>sum by (service_id) (kube_pod_container_resource_requests{resource="cpu"})</code>	requests, not limits — requests are what the scheduler reserves
Replica count	<code>sum by (service_id) (kube_deployment_status_replicas)</code>	needed by the executor's blast-radius rail

B.2 Queries for when something looks wrong

debugging queries

```
# Is the relabelling working? If service_id is empty here, the collector
# will silently see nothing - this is the first thing to check.
count by (service_id) (up{namespace="shop"})

# Which targets is Prometheus actually scraping, and are any failing?
up{namespace=~"shop|platform"} == 0

# Cost-relevant: cores per request. A rise here IS the runaway signature,
# visible in raw PromQL before the behaviour engine says anything.
sum by (service_id) (rate(container_cpu_usage_seconds_total[2m]))
/ sum by (service_id) (rate(app_requests_total[2m]))

# Is the HPA actually scaling, or is it stuck at its ceiling?
kube_horizontalpodautoscaler_status_current_replicas
/ kube_horizontalpodautoscaler_spec_max_replicas

# Restart storms masquerade as cost anomalies. Rule this out first.
increase(kube_pod_container_status_restarts_total[15m]) > 0

# Platform overhead, the number appendix D asks you to report.
sum(rate(container_cpu_usage_seconds_total{namespace="platform"}[5m]))
/ sum(rate(container_cpu_usage_seconds_total{namespace="shop"}[5m]))

# Scrape health of the platform itself: if this climbs, your interval is
# too aggressive for the machine.
rate(prometheus_target_scrapes_exceeded_sample_limit_total[5m])
```

CHECK THE DATA BEFORE THE CODE

Learn to answer “is the data there?” in Prometheus before debugging any Python. Most of the time a missing cost figure is a missing label, not a broken calculation, and `count by (service_id) (up{namespace="shop"})` answers that in two seconds. Debugging the collector when the problem is a relabel rule can cost an afternoon.

APPENDIX

C. Appendix C - Glossary for the impatient

Definitions written for someone meeting these terms while building, not for someone revising for an exam. Where a term has a precise meaning inside this project that is narrower than its general use, the project's meaning is given.

C.1 Infrastructure

Container	A process with its own filesystem and resource limits, sharing the host kernel. Not a virtual machine: there is no second operating system, which is why it starts in milliseconds.
Image	The read-only filesystem a container starts from. <code>victim:dev</code> is an image; the running thing is a container.
Pod	One or more containers scheduled together and sharing a network address. In this project a pod is one container, which is the common case.
Deployment	A controller that keeps N identical pods running and replaces them when they die or when you change the template. What the executor patches.
Service (Kubernetes)	A stable name and address in front of a changing set of pods. Note the collision: <code>services</code> in the database means a logical workload the platform observes, which is a different thing.
Namespace	A name-scoping boundary inside a cluster, used here as the unit of opt-in: the platform will only act inside a namespace labelled <code>ht.io/monitored=true</code> .
HPA	Horizontal Pod Autoscaler. Adds or removes replicas to hold a target metric. The thing that turns a slow bug into an expensive one, and the thing <code>freeze_scaling</code> pins.
Requests and limits	Requests are what the scheduler reserves and what shared cost is apportioned by; limits are the ceiling the kernel enforces and what <code>throttle</code> reduces.
kind	Kubernetes in Docker. Runs a real cluster with each node as a container, which is what makes a three-node cluster fit on a laptop.

C.2 Data and streaming

Time-series database	A database optimised for append-mostly rows keyed by time. TimescaleDB is PostgreSQL with this optimisation added, so ordinary SQL works.
Hypertable	A TimescaleDB table transparently split into time-based chunks. Retention and compression are set per hypertable, which is how a week of raw metrics stays small.
Continuous aggregate	A materialised view that refreshes incrementally. <code>cost_hourly</code> is one; it is why a 30-day cost chart is fast.
Broker, topic, partition	The broker stores the log; a topic is a named log; a partition is one ordered shard of it. Ordering is guaranteed within a partition only, which is why messages are keyed by <code>service_id</code> .
Consumer group	A set of consumers that between them read each partition once. Two services in different groups both see every message; two instances in the same group split the work.
Offset	How far a consumer group has read. Committing it manually after the effect is durable is what makes at-least-once delivery safe.
Idempotent	An operation that can be applied twice with the same result. The property that makes redelivery harmless rather than a bug.
At-least-once	The delivery guarantee this system uses: no message is lost, some may arrive twice. Paired with idempotency, this is a good trade.

C.3 Detection and cost

Baseline	What normal looks like for one service, one metric and one hour of the week. Not a single number for the whole service.
EWMA	Exponentially weighted moving average. Recent samples count more; $\alpha = 1/720$ gives roughly two weeks of memory at ten-second samples.
Z-score	How many standard deviations a reading sits from its baseline mean. Dimensionless, so a CPU Z-score and a dollar Z-score are directly comparable.
Hour-of-week bucket	One of 168 slots, so Sunday 03:00 is compared against other Sunday 03:00s rather than against Tuesday lunchtime.
Decoupling	Cost and compute rising while traffic does not. The signature this project exists to detect, and the test that keeps it quiet during real spikes.
Cost velocity	The rate of change of spend, in dollars per hour per hour. A budget alert watches the level; this watches the slope.
Unit cost	Dollars per thousand requests. Undefined below 100 requests per hour, reported as null rather than as a huge number.
Amortisation	Spreading a shared charge across the things that caused it. Here: control plane and load balancer cost, apportioned by requested CPU.
Confidence	A bounded score combining magnitude, decoupling, corroboration and baseline maturity. Determines authority, not urgency.
Trust stage	How much authority a deployment has earned, 0 to 4. Rises with correct recommendations, falls on two human rollbacks in seven days.

APPENDIX

D. Appendix D - Fourteen weeks, and what to cut

The milestones of Part III are sized in hours; a semester is measured in weeks with other coursework in them. The plan below maps one onto the other with slack, and then says explicitly what to abandon first when a week disappears — which it will.

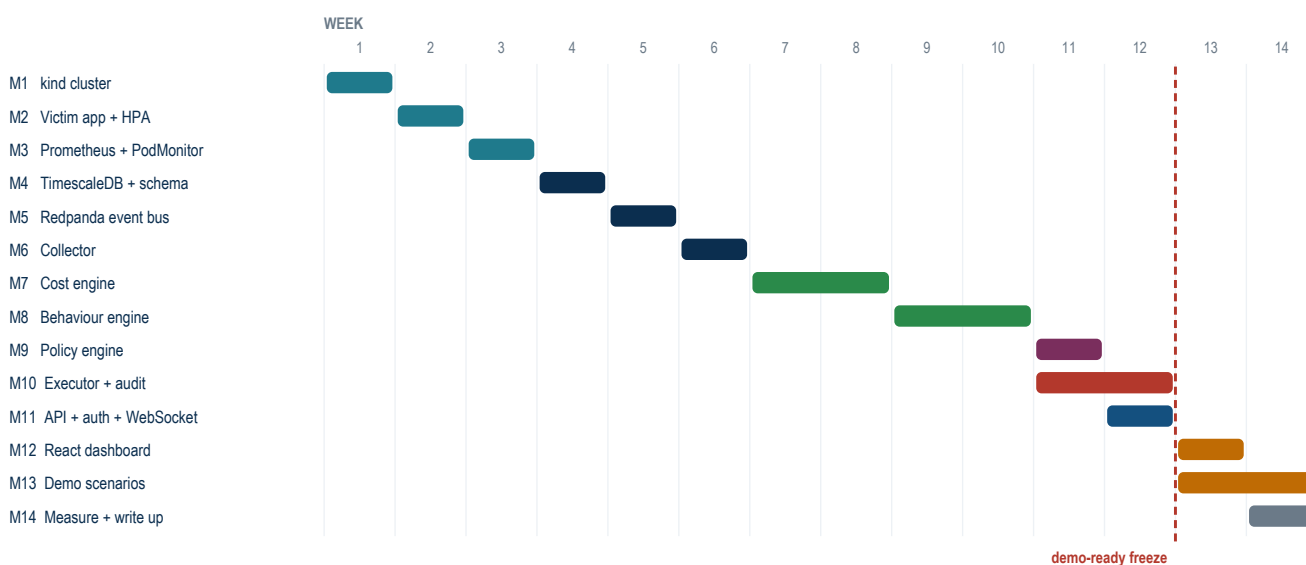


Figure 14 - Fourteen-week plan mapped onto the fourteen milestones of Part III. Weeks 1 to 10 build the pipeline once; weeks 11 to 14 exist to make it demonstrable and measurable, which is what a panel actually grades.

D.1 What each week must end with

Week	Milestone	Must be true on Friday	Slack
1	M1	three-node cluster up, namespaces labelled	high
2	M2	<code>/checkout</code> responds and the HPA scales under load	high
3	M3	Prometheus shows <code>service_id</code> on every series	medium
4	M4	schema applied; append-only trigger provably refuses	high
5	M5	six topics; a message survives a broker restart	high
6	M6	<code>metrics.raw</code> carries samples every ten seconds	medium
7-8	M7	dollars per hour in the database; cost test passes	low
9-10	M8	anomaly on injected fault, silence on real load	none
11	M9	decisions carry a rule, a mode and a reason	low
12	M10	a real throttle applied, logged, and auto-reverted	none
12	M11	authenticated API and a live WebSocket	medium
13	M12	dashboard shows live cost and the anomaly feed	high
13-14	M13	all four scenarios reproducible by one command	none
14	M14	measurement table produced by a script	none

D.2 Cut in this order

When a week is lost, cut from the top of this list. The ordering is by how much each item contributes to the argument that the system works, which is not the same as how visible it is.

1. **Dashboard polish.** One page with a chart and a feed is enough. Nobody has ever been marked down for unstyled buttons; several projects have been marked down for a beautiful dashboard with nothing behind it.
2. **The third victim workload.** Two services demonstrate attribution and shared cost apportionment just as well as three.
3. **The quarantine and terminate rungs.** Implement the ladder's data model and the top two rungs' policy handling, but only wire `throttle` and `freeze_scaling` to real cluster calls. Say which rungs are wired and which are modelled.
4. **Row-level security.** Single-tenant is a defensible scope for a semester project as long as the schema carries `org_id` and you can explain the policy you would add.
5. **Trust progression.** A fixed trust stage set in the database, with the ladder documented, costs you far less than an unmeasured system.

THE THREE THINGS TO PROTECT

Never cut milestone 8's negative test, milestone 10's audit log, or milestone 14. Those three are the entire difference between “we built a cost dashboard” and “we built a system that decides carefully, records what it did, and here are the numbers.” Everything else on this list is negotiable.

APPENDIX

E. Appendix E - Definition of done

A single checklist covering the whole project. If every line is true, the system in Part I exists on your machine and you can prove it.

E.1 Functional

CHECKPOINT - VERIFY BEFORE MOVING ON

- ▮ Resource telemetry from at least two services is collected every ten seconds.
- ▮ Every sample is converted to dollars per hour with a six-component breakdown.
- ▮ One cost figure has been verified by hand against the arithmetic in chapter 22.
- ▮ Baselines are held per service, per metric and per hour-of-week bucket.
- ▮ Baselines survive a restart of the behaviour engine.
- ▮ An injected runaway is detected within 90 seconds.
- ▮ Genuine load produces no anomaly, and this is asserted by a script.
- ▮ Every anomaly carries observed values, baseline values and all Z-scores.
- ▮ Policy is declarative, loaded from files, and a broken file degrades to alert-only.
- ▮ A prohibition in one document cannot be overridden by a permission in another.
- ▮ At least one action type makes a real, observable change to the cluster.
- ▮ Every action expires and reverts automatically without human involvement.
- ▮ Refusals and failures are logged as fully as successes.
- ▮ `actions_log` cannot be updated or deleted by the application role.
- ▮ The API requires authentication and returns 401 without a valid token.
- ▮ The dashboard updates without a page refresh and reconnects on its own.

E.2 Non-functional and evidential**CHECKPOINT - VERIFY BEFORE MOVING ON**

- ▮ Platform overhead is measured, not estimated, and is under five percent.
- ▮ Detection latency is measured over at least five runs with median and worst case.
- ▮ The measurement output is saved with a date and the machine's specification.
- ▮ All four demo scenarios run from a single command each.
- ▮ The test suite covers the cost arithmetic, the detector, the policy engine and the safety rails, and includes tests that assert refusals.
- ▮ Every diagram in your report matches the code that exists.
- ▮ The limitations in chapter 31.3 are written down in your own report.
- ▮ You can explain why one specific confidence score had the value it did.

APPENDIX**F. Appendix F - Questions you will be asked**

Grouped by what the questioner is really testing. The answers are deliberately short: an examiner is looking for whether you understand the trade-off, not for a recital.

F.1 On novelty**CloudWatch already has billing alarms. What is new here?**

Billing alarms fire on a threshold, hours after the spend, and they cannot act. This system detects on the slope rather than the level, within a minute rather than a day, and can respond under policy. The novelty is the integration — cost as a real-time signal correlated with behaviour and gated by policy — not any single primitive. Every individual piece exists somewhere; the composition is what does not.

Kubecost also shows real-time cost. So?

Kubecost is excellent at allocation and reporting, and OpenCost is the CNCF standard underneath it. Neither decides anything. This system's contribution is downstream of the cost figure: correlating it with behaviour and acting on it under policy with an audit trail.

Sedai, Cast AI and ScaleOps already act autonomously.

They optimise for efficiency — rightsizing, bin-packing, spot management — on a steady-state workload. This system targets the anomalous case: something is wrong right now and spend is accelerating. Optimising a runaway loop's resource requests makes it cheaper per hour; stopping it is what is actually needed.

F.2 On the mathematics**Why a Z-score and not machine learning?**

Because it is explainable, cheap, and correct for the shape of this problem. Every decision has to be defensible to whoever is on call, and “CPU is 7.0 standard deviations above its Tuesday-14:00 baseline while traffic is 0.2 below” is defensible. An Isolation Forest would score comparably and explain nothing. Chapter 23.6 records this as a deliberate deferral with a named condition for revisiting it.

What if the anomaly is the new normal?

The EWMA absorbs it. With `alpha = 1/720` a sustained shift is absorbed over roughly two weeks, so a genuine growth in traffic stops being flagged without anyone tuning anything. That is also the weakness: a slow leak that grows over a month is learned as normal, which is why absolute cost thresholds exist alongside the statistical test.

Your confidence weights are 0.40, 0.30, 0.20 and 0.10. Why?

They are a judgement, not a fit, and the honest answer is that they were chosen to encode a priority ordering: magnitude matters most, decoupling next, then corroboration, then maturity. What matters more than the exact values is that the function is bounded, that each term is independently capped, and that the maturity term makes an untrained baseline structurally incapable of autonomous authority. Tuning them against recorded incidents is exactly what the replay mode in chapter 31.4 is for.

F.3 On safety**What happens when your detector is wrong and it throttles production?**

Six things, in order: confidence below 0.85 means it could only recommend, not act; policy can cap authority below what confidence allows; the trust stage limits which rungs are reachable at all; the rails refuse if the service is already unhealthy, if three actions have fired this hour, or if the evidence is over two minutes old; the action is the gentlest rung that addresses the symptom; and it reverts automatically after fifteen minutes. The worst case is a bounded, logged, self-reverting fifteen-minute throttle — not an outage.

What if the platform itself is compromised?

The executor's Kubernetes role is namespaced and verb-limited, it cannot touch control-plane namespaces, and the audit log rejects modification from the application role, so tampering with history requires the database superuser. The honest answer is that this is the right structure and it has not been attacked; a penetration test is named as the next step rather than claimed as complete.

Why not act on everything immediately? Speed is the point.

Because the cost of a wrong action is asymmetric. A missed anomaly costs money, which is recoverable; a wrong action costs availability, which is not. The trust ladder exists so that authority is earned against a record rather than granted by configuration on day one.

F.4 On engineering choices**Why a message bus rather than direct calls between services?**

Three reasons that all matter here. Replay: the log lets the same events be reprocessed with different thresholds, which is how detection gets tuned honestly. Isolation: a stalled behaviour engine cannot slow the collector. And dual consumption: the same stream feeds the hot detection path and the cold storage path independently, which was the requirement in chapter 4.

Why FastAPI and not Spring Boot or Go?

The detection code is numerical and the ecosystem for that work is Python, so choosing Python for the pipeline removes a language boundary in the most complex part of the system. FastAPI's async model suits a service that is almost entirely I/O wait, and the type-hint-driven schema generation means the API documentation cannot drift from the code. Go would use less memory and is the better production answer at scale; that trade was made consciously, not by default.

Why is the timezone stored per service rather than using UTC everywhere?

Because the hour-of-week buckets are only meaningful in the timezone of the people generating the traffic. Storing timestamps in UTC is right and this system does that; bucketing in UTC would smear a service's Monday-morning peak across several buckets and destroy the signal the buckets exist to preserve.

F.5 On honesty

What does not work?

Reconciliation against a real bill, because there is no bill; discount modelling, which is an explicit non-goal; anything at production scale, since the capacity figures are projections rather than measurements; and the security-corroboration input, which is designed but fed only by metrics. Also: the baseline is untrustworthy for a service's first fortnight, and a genuinely bimodal workload will detect late.

If you started again, what would you change?

Build the measurement harness in week two instead of week fourteen. Every tuning decision made before there was a way to measure detection latency and false positives was made on intuition, and some of them had to be revisited. The second change would be to record the raw stream from day one so that every later change could be evaluated against the same history rather than against a fresh injection.

PREPARE THE UNCOMFORTABLE QUESTIONS HARDEST

The last two answers are worth more than the first ten. A panel can tell the difference between a student who has memorised a system and one who has lived with it, and knowing precisely where the edges are is the tell. Prepare those two answers with the same care as the architecture diagram.